UG_31240275@ics.tanta.edu.eg
UG_31240275@ics.tanta.edu.eg
UG_31240275@ics.tanta.edu.eg

Artificial Intelligence

A Course Text for Undergraduates

CS313: Introduction to Artificial Intelligence

Fall Edition

رومانی صبحی رزق جرجس مطر

2025/2026

2025/2026

2025/2026

Agents · Search · Logic · Learning

Companion volume to the weekly lecture schedule

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Preface

رومانی صبحی رزق جرجس مطر

This book accompanies a one-semester undergraduate course in artificial intelligence. It is organized into sixteen chapters, one per week of the term, moving from rational agents and search through logic, probabilistic reasoning, decision-making, machine learning, and the ethical questions raised by deployed AI systems.

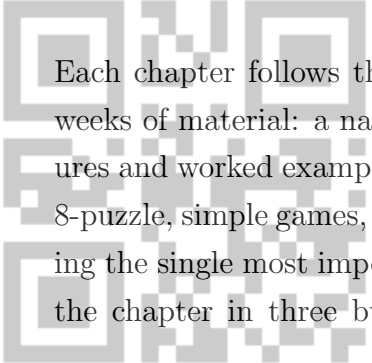
Each chapter is self-contained enough to read on its own, but the chapters build on one another: search underlies planning, planning underlies decision-theoretic reasoning, and probabilistic and learning-based methods recur throughout. Every chapter ends with a set of exercises ranging from short conceptual checks to small implementation tasks. Two appendices collect the programming assignments and the term project guidelines used in the course, and two quick-reference sheets at the back of the book summarize the algorithms and formulas introduced along the way.

The material draws on several standard references in the field, most heavily Russell and Norvig's *Artificial Intelligence: A Modern Approach*, which remains the closest thing the field has to a common textbook, along with Sutton and Barto on reinforcement learning, Bishop and Goodfellow, Bengio, and Courville on machine learning and deep learning, and Koller and Friedman on probabilistic graphical models. Full citations appear in the bibliography.

2025/2026 2025/2026 2025/2026

This text is intended as a support for lectures, not a replacement for them: proofs are sketched rather than belabored, and the emphasis throughout is on the ideas and algorithms a student needs in order to read the primary textbook chapters, complete the assignments, and reason clearly about what AI systems can and cannot do.

How to Use This Book



Each chapter follows the same structure to make navigation predictable across sixteen weeks of material: a narrative development of the week's core ideas, illustrated with figures and worked examples drawn from a small set of running scenarios (route-finding, the 8-puzzle, simple games, small Bayesian networks, gridworlds); a boxed *Key Idea* highlighting the single most important claim in the chapter; a boxed *Chapter Summary* recapping the chapter in three bullet points; and an exercise set moving from short conceptual

questions to a substantial implementation task tied to the corresponding programming assignment in Appendix A.

Students preparing for an exam are directed first to Chapter 16's synthesis material and the exam-preparation table therein, then to the two quick-reference sheets at the very end of the book, and finally back to individual chapters' Key Idea and Chapter Summary boxes for targeted review. Students working on the term project (Appendix B) will find the end-to-end case study in Appendix F a useful template for how to decompose an applied problem into the right combination of techniques from across the book. Mathematical prerequisites are reviewed in Appendix C and software tooling in Appendix D, both intended as references to return to as needed rather than reading assignments in their own right.

Course Staff

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Course Syllabus

CS313: Introduction to Artificial Intelligence

Credits	3
Term	Fall
Level	Undergraduate (Junior/Senior)
Lecture	Tue/Thu, 90 minutes
Lab/Recitation	1 hour weekly
Instructor	TBD
Office hours	2 hours/week + by appointment

Course Description

An introduction to the concepts, techniques, and applications of artificial intelligence. The course covers how intelligent agents represent knowledge, search for solutions, reason under uncertainty, and learn from data. Students implement core algorithms from scratch, evaluate them empirically, and examine the societal implications of deploying AI systems.

Prerequisites

- Data Structures and Algorithms
- Discrete Mathematics
- Introductory Probability and Statistics (or concurrent enrollment)
- Basic linear algebra recommended
- Programming proficiency in Python

Learning Outcomes

By the end of this course, students will be able to:

1. Formulate real-world problems as search, constraint satisfaction, or decision problems.
2. Implement and compare uninformed, informed, and adversarial search algorithms.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

3. Represent knowledge using logical and probabilistic formalisms and perform inference over them.
4. Build, train, and evaluate supervised and unsupervised learning models, including neural networks.
5. Apply reinforcement learning to sequential decision problems.
6. Assess AI systems for bias, reliability, privacy risk, and appropriate use.
7. Communicate technical results in written reports and oral presentations.

Required Materials

- **Textbook:** Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed.
- **Supplementary:** Selected chapters from Goodfellow, Bengio & Courville, *Deep Learning* (free online); course notes posted weekly; this book.
- **Software:** Python 3.11+, NumPy, scikit-learn, PyTorch, Jupyter. No paid tools required (see Appendix D).
- **Compute:** Departmental GPU cluster access provided; Google Colab free tier is sufficient as a fallback.

Grading

2025/2026 2025/2026 2025/2026

Component	Weight
Final exam	50%
Midterm exam	25%
Term project	15%
Programming assignments, labs, quizzes, participation	10%

Scale: A $\geq$ 93, A$-$ $\geq$ 90, B+ $\geq$ 87, B $\geq$ 83, B$-$ $\geq$ 80, C+ $\geq$ 77, C $\geq$ 73, C$-$ $\geq$ 70, D $\geq$ 60, F $<$ 60.

Programming Assignments

Full specifications appear in Appendix A.

#	Topic	Due
P1	Search agents: BFS, UCS, A* on a pathfinding/puzzle domain	Week 4
P2	Adversarial search: minimax, alpha-beta, evaluation functions	Week 6
P3	Probabilistic inference: Bayesian network + particle filter	Week 9
P4	Supervised learning: implement logistic regression and a decision tree, benchmark against scikit-learn	Week 11
P5	Neural networks or reinforcement learning (student choice)	Week 14

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Assignments are individual unless stated otherwise. Autograders run on submission; the highest of three submissions counts.

رومانی صبحی رزق جرجس مطر

Term Project

Teams of 2–3. Students choose an application problem, select and implement appropriate AI methods, evaluate empirically, and discuss limitations and ethical considerations. Guidelines and the grading rubric appear in Appendix B.

- Week 7: Proposal (1–2 pages)
- Week 11: Progress checkpoint (in-lab demo)
- Week 15: Final presentation (10 min) and report (6–8 pages)

2025/2026 2025/2026 2025/2026

Grading weights: technical execution 40%, evaluation rigor 25%, report and presentation 25%, ethical analysis 10%.

Weekly Schedule

Each week of the course corresponds to the chapter of this book with the same number. The Reading column lists the matching chapters of Russell & Norvig.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Week	Topic	Reading
1	What is AI? Intelligent agents, environments, rationality; history and current landscape	Ch. 1–2
2	Uninformed search: BFS, DFS, uniform-cost, iterative deepening	Ch. 3.1–3.4
3	Informed search: greedy, A*, heuristic design and admissibility; local search	Ch. 3.5–4.1
4	Constraint satisfaction: backtracking, arc consistency, heuristics	Ch. 6
5	Adversarial search: minimax, alpha-beta pruning, expectimax, Monte Carlo tree search	Ch. 5
6	Logical agents: propositional logic, inference, resolution, SAT solvers	Ch. 7
7	First-order logic and knowledge representation; **Midterm exam**	Ch. 8–9
8	Uncertainty and probability; Bayes' rule; Bayesian networks	Ch. 12–13
9	Inference in Bayes nets; temporal models, HMMs, particle filtering	Ch. 13–14
10	Decision theory, utility, MDPs, value and policy iteration	Ch. 15–16
11	Machine learning foundations: generalization, overfitting, evaluation, linear and logistic regression, decision trees	Ch. 19
12	Neural networks: perceptrons, backpropagation, regularization, CNNs	Ch. 21
13	Reinforcement learning: Q-learning, SARSA, function approximation	Ch. 22
14	Language and perception: n-grams, word embeddings, transformers and large language models; computer vision overview	Ch. 23–25
15	AI ethics and safety: bias, fairness, transparency, labor and environmental impact, governance. Project presentations	Ch. 27–28
16	**Final exam** (cumulative, emphasis on weeks 8–15)	—

2025/2026 2025/2026 2025/2026

Course Policies

Late work. Each student has four late days for the semester, usable in whole-day increments, no more than two on any one assignment. Beyond that, late work loses 20% per day and is not accepted more than three days late. Extensions for documented illness or emergency are granted without penalty; contact the instructor as early as you can.

Collaboration. Discussing concepts and debugging strategies with classmates is encouraged. Writing code or prose together, or sharing solutions, is not, except on team project work. Cite every person, website, and tool that contributed to your submission.

Use of AI tools. Generative AI assistants may be used for concept explanation, debugging help, and editing. They may not be used to produce solution code or written analysis you submit as your own. Every assignment requires a short disclosure statement describing which tools you used and how. Exams are closed-tool. Undisclosed use is treated as an academic integrity violation.

Academic integrity. Violations are reported to the university office of academic in-

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

tegrity and typically result in a zero for the work and a possible course failure. If you are unsure whether something is permitted, ask before submitting.

Accessibility. Students with documented disabilities should contact the disability services office; accommodations will be implemented promptly and confidentially. Please notify the instructor as early in the term as possible.

Attendance. Lecture attendance is not graded, but lab participation is. More than three unexcused lab absences reduces the participation component.

Communication. Course questions go to the discussion forum so everyone benefits; personal matters go to email. Expect a response within one business day.

Wellbeing. This course is demanding, and asking for help early is the single best predictor of finishing well. Campus counseling and academic support services are available to all students at no cost.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Contents

Preface ii

Course Syllabus iv

1 What Is AI? Agents, Environments, and Rationality 1

1.1 Defining Artificial Intelligence . . . 1
1.2 The Agent Function and Agent Program . . . 2
1.3 Rationality, Omniscience, and Autonomy . . . 2
1.4 Task Environments and PEAS . . . 3
1.5 Agent Program Types . . . 4
1.6 Environment Programs and the Agent–Environment Contract . . . 5
1.7 Multi-Agent Environments . . . 6
1.8 Benchmarks and How the Field Measures Progress . . . 7
1.9 Looking Ahead . . . 7

2 Uninformed Search 10

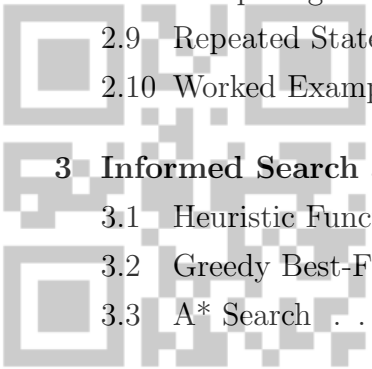
2.1 Problem-Solving Agents . . . 10
2.2 Evaluating Search Strategies . . . 11
2.3 Breadth-First Search . . . 11
2.4 Uniform-Cost Search . . . 12
2.5 Depth-First Search . . . 12
2.6 Iterative Deepening Search . . . 13
2.7 Bidirectional Search . . . 13
2.8 Comparing the Strategies . . . 14
2.9 Repeated States and Graph Search . . . 14
2.10 Worked Example: Route-Finding . . . 14

3 Informed Search and Heuristics 17

3.1 Heuristic Functions . . . 17
3.2 Greedy Best-First Search . . . 17
3.3 A* Search . . . 17

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر



UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

3.4 Designing Heuristics 18
3.5 Memory-Bounded Variants 19
3.6 Local Search: Hill Climbing and Simulated Annealing 19
3.7 Worked Example: Straight-Line Distance as a Heuristic 20

4 Constraint Satisfaction Problems 23
4.1 CSP Formulation 23
4.2 Backtracking Search 24
4.3 Constraint Propagation 24
4.4 Structure and Decomposition 25
4.5 Local Search for CSPs 25
4.6 Worked Example: AC-3 Propagation 26

5 Adversarial Search and Games 28
5.1 Games as Search Problems 28
5.2 Minimax 28
5.3 Alpha-Beta Pruning 29
5.4 Depth-Limited Search and Evaluation Functions 30
5.5 Expectimax and Stochastic Games 30
5.6 Monte Carlo Tree Search 31
5.7 Worked Example: Expectimax with a Chance Node 31

2025/2026 2025/2026 2025/2026

6 Logical Agents: Propositional Logic 34
6.1 Knowledge-Based Agents 34
6.2 Syntax and Semantics of Propositional Logic 34
6.3 Inference 35
6.3.1 Resolution 35
6.3.2 Forward and Backward Chaining 35
6.3.3 The DPLL Algorithm and Satisfiability 35
6.4 Limitations of Propositional Logic 37
6.5 Worked Example: Resolution Refutation 37

7 First-Order Logic and Knowledge Representation 39
7.1 Why First-Order Logic 39
7.2 Syntax and Semantics 39
7.3 Knowledge Engineering 40
7.4 Inference in First-Order Logic 40
7.4.1 Unification and Generalized Modus Ponens 40
7.4.2 Resolution in First-Order Logic 40
7.5 Ontological Commitments and Representing Change 41

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

7.6 Worked Example: A Small Family-Relations Knowledge Base 41

8 Uncertainty and Bayesian Networks 44
8.1 Why Probability 44
8.2 Basic Probability Axioms 44
8.3 Bayes' Rule 44
8.4 Independence and Conditional Independence 45
8.5 Bayesian Networks 45
8.6 Exact Inference 46
8.7 Constructing Bayesian Networks 47
8.8 Worked Example: Variable Elimination by Hand 47

9 Approximate Inference and Temporal Probabilistic Models 50
9.1 Sampling Methods for Bayesian Networks 50
9.2 Temporal Models 51
9.3 Hidden Markov Models 52
9.4 Particle Filtering 52
9.5 Worked Example: Comparing Filtering Methods 53

10 Decision Theory and Markov Decision Processes 56
10.1 Utility Theory 56
10.2 Sequential Decision Problems 56
10.3 Value Iteration 57
10.4 Policy Iteration 58
10.5 Partial Observability 58
10.6 Worked Example: One Bellman Backup by Hand 59

11 Machine Learning Foundations 61
11.1 The Learning Problem 61
11.2 Generalization, Overfitting, and Evaluation 61
11.3 Linear Regression 62
11.4 Logistic Regression 63
11.5 Decision Trees 64
11.6 Model Selection and the No Free Lunch Theorem 65

12 Neural Networks 67
12.1 From Perceptrons to Multilayer Networks 67
12.2 Forward Propagation 67
12.3 Backpropagation 68
12.4 Training in Practice 69

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

12.5 Convolutional Neural Networks 69
12.6 Loss Functions and Output Layers 70

13 Reinforcement Learning 72
13.1 The Reinforcement Learning Problem 72
13.2 Model-Based versus Model-Free Learning 72
13.3 Temporal-Difference Learning 73
13.4 Q-Learning 73
13.5 SARSA 73
13.6 Function Approximation 74
13.7 Policy Search Methods 74

14 Language and Perception: From N-grams to Transformers 76
14.1 Language Models 76
14.2 Word Embeddings 77
14.3 Recurrent Networks and Their Limits 78
14.4 Attention and the Transformer 79
14.5 Large Language Models 80
14.6 Perception: A Brief Overview 80
14.7 Worked Example: Estimating a Bigram Model 81

15 AI Ethics, Safety, and Governance 83
15.1 Why Ethics Belongs in a Technical Course 83
15.2 Bias and Fairness 83
15.3 Robustness, Reliability, and Safety 84
15.4 Privacy 85
15.5 Transparency and Explainability 85
15.6 Labor, Environmental, and Economic Impact 86
15.7 Governance and Regulation 86
15.8 A Practical Checklist 86

16 Synthesis: How the Pieces Fit Together 89
16.1 One Course, Several Threads 89
16.2 The Search Thread 90
16.3 The Knowledge and Reasoning Thread 91
16.4 The Uncertainty Thread 91
16.5 The Learning Thread 91
16.6 A Cross-Chapter Comparison Table 91
16.7 Full-Course Exam Preparation Table 92
16.8 Where This Leaves You 93

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

A Programming Assignments **96**
A.1 P1: Search Agents (Chapters 2–3) . . . 96
A.2 P2: Adversarial Search (Chapter 5) . . . 97
A.3 P3: Probabilistic Inference (Chapters 8–9) . . . 97
A.4 P4: Supervised Learning (Chapter 11) . . . 98
A.5 P5: Neural Networks or Reinforcement Learning (Chapters 12–13) . . . 98
A.6 Common Grading Rubric . . . 99
A.7 Common Pitfalls Across Assignments . . . 99

B Term Project Guidelines **101**
B.1 Overview . . . 101
B.2 Milestones . . . 101
B.2.1 Proposal (1–2 pages) . . . 102
B.2.2 Checkpoint . . . 102
B.2.3 Final Report and Presentation . . . 102
B.3 Grading Rubric . . . 102
B.4 Suggested Project Areas . . . 102
B.5 Common Pitfalls . . . 103
B.6 Two Worked Project Sketches . . . 103
B.6.1 Example: Route Recommendation with Learned Preferences . . . 103
B.6.2 Example: Cliff-Walking Agent with Risk-Sensitive Policies . . . 104

C Mathematical Background **105**
C.1 Linear Algebra . . . 105
C.1.1 Vectors and Dot Products . . . 105
C.1.2 Matrices and Matrix-Vector Products . . . 105
C.1.3 Norms . . . 106
C.2 Calculus and Optimization . . . 106
C.2.1 Derivatives and Gradients . . . 106
C.2.2 The Chain Rule . . . 106
C.2.3 Convexity . . . 107
C.3 Probability Review . . . 107
C.3.1 Random Variables and Distributions . . . 107
C.3.2 Expectation and Variance . . . 107
C.3.3 The Chain Rule of Probability . . . 108
C.4 Dimensionality Reduction: A Brief Look at PCA . . . 108
C.5 Big-O Notation Review . . . 109
C.6 Notation Summary . . . 110

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

D Python and Tooling Primer **112**
D.1 Environment Setup . . . 112
D.2 NumPy Essentials . . . 112
D.3 Working with scikit-learn . . . 113
D.4 A Minimal PyTorch Training Loop . . . 113
D.5 Plotting with Matplotlib . . . 114
D.6 Reproducibility Checklist . . . 114
D.7 Debugging Numerical Code . . . 115
D.8 Version Control for Assignments . . . 115
D.9 Common Error Messages and What They Usually Mean . . . 116

E Solutions and Hints to Selected Exercises **117**

F End-to-End Case Study: Building a Maze-Solving Agent **123**
F.1 Stage 1: Known Maze, Deterministic Movement . . . 123
F.2 Stage 2: Unknown Maze, Must Be Explored . . . 124
F.3 Stage 3: Noisy Sensors, Uncertain Position . . . 124
F.4 Stage 4: Unknown Reward Structure, Learned by Trial and Error . . . 125
F.5 What This Progression Illustrates . . . 126

Glossary **127**

2025/2026 2025/2026 2025/2026
Quick Reference Sheet I: Algorithms **132**

Quick Reference Sheet II: Formulas and Notation **134**

Bibliography **137**

Index **138**

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

List of Figures

رومانی صبحی رزق جرجس مطر

1.1 The basic agent–environment interaction loop. The agent receives percepts through sensors, updates its internal state, selects an action via its agent program, and executes it through actuators; the resulting change in the environment produces the next percept. 2

1.2 A taxonomy of agent program designs, in order of increasing capability. Each later type subsumes the reasoning of the types before it: a learning agent, for instance, can in principle acquire any of the other four behaviors through experience. 4

1.3 The environment simulation loop used throughout this book to evaluate agents. The feedback arrow on the right indicates that this loop runs until a terminal condition (goal reached, episode length exceeded, or agent failure) is met. 6

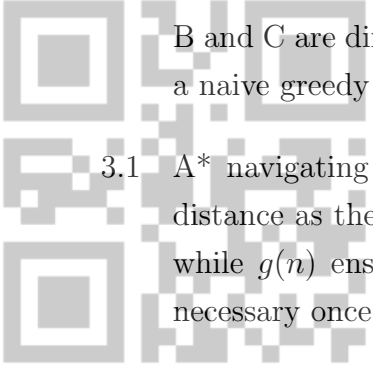
2.1 A search tree rooted at start state A. Shaded nodes B and C have been expanded; the goal, G, has just been generated. The order in which nodes such as D, E, F, and G are discovered and expanded is what distinguishes BFS, DFS, and their variants. 11

2.2 Illustrative comparison (branching factor $b = 10$) of cumulative nodes generated by iterative deepening across all its restarts versus nodes generated by a single BFS pass to the same depth. The overhead of IDS's repeated shallow re-exploration is small relative to the cost of the final, deepest layer, which dominates both curves. 13

2.3 A small road network. Edge labels give road distances (path costs). Cities B and C are directly connected, giving uniform-cost search a shortcut that a naive greedy choice at A would miss. 14

3.1 A* navigating a grid with an obstacle (dark region), using Manhattan distance as the heuristic. The heuristic guides expansion toward the goal while $g(n)$ ensures the detour around the obstacle is still recognized as necessary once the direct route is blocked. 18

2025/2026 2025/2026 2025/2026



UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

3.2 Illustrative node expansions for a moderately difficult 8-puzzle instance under uniform-cost search versus A* with two admissible heuristics. Manhattan distance dominates misplaced-tiles, so A* using it never expands more nodes, and empirically expands dramatically fewer. 18

3.3 A 1-D objective landscape illustrating why hill climbing can become trapped: starting near the left peak, a hill-climbing agent never discovers the higher peak on the right because every step toward it initially decreases the objective. Simulated annealing's willingness to accept occasional downhill moves allows escape from such traps. 20

3.4 A* with straight-line-distance heuristic. The dashed line shows the straight-line distance from A to the goal, which lower-bounds every real route and is therefore admissible. 20

4.1 The constraint graph for map-coloring Australia's mainland states and territories plus Tasmania. Each edge is a binary constraint requiring the two adjacent regions to take different colors; Tasmania (T) is disconnected from the mainland graph, so it can take any color independent of the rest. 23

4.2 A fragment of backtracking search for map coloring. After fixing WA=Red, trying NT=Green leaves SA with no legal color (both its neighbors, WA and NT, would occupy Red and Green), forcing a backtrack; trying NT=Blue instead leaves SA=Green as a valid continuation. MRV would have preferred to assign the most constrained variable, often SA, earlier to detect such failures sooner. 24

4.3 AC-3 trace for the chain X–Y–Z. Because $D_Z = \{1\}$, arc consistency forces $Y \neq 1$, shrinking D_Y to $\{2, 3\}$; this change is re-queued back onto the arc (X, Y), but since $D_X = \{1, 2, 3\}$ still has a compatible partner in $\{2, 3\}$ for every value, no further pruning occurs at X. A final assignment (e.g., X=1, Y=2, Z=1) can then be found with no backtracking. 26

5.1 Minimax values backed up through a depth-2 game tree. At the MIN level, node A takes $\min(3, 12) = 3$ and node B takes $\min(2, 4) = 2$; at the MAX root, the value is $\max(3, 2) = 3$, so MAX should choose the branch leading to node A. 29

5.2 Alpha-beta pruning on the same tree as Figure 5.1. The dashed edge to 12 under node A is irrelevant once MIN's first child there is examined, since MIN would never choose 12 over a smaller value already available. Under node B, once its first child evaluates to 2, MAX already has $\alpha = 3$ from node A, so B cannot possibly exceed 3 and its remaining child is pruned entirely without being evaluated. 29

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

5.3 Expectimax tree: MAX chooses between two actions, each landing on a chance node with two equally likely outcomes. The left chance node has expected value $0.5(3) + 0.5(9) = 6$; the right has expected value $0.5(5) + 0.5(5) = 5$. MAX therefore prefers the left action despite its higher variance, since expectimax cares only about expected value. 32

6.1 A resolution refutation: resolving $P \vee Q$ with $\neg Q \vee R$ (on complementary literal Q) yields $P \vee R$; resolving that with $\neg P$ (on complementary literal P) yields the empty clause, proving the original set of clauses – together with the negated query used to seed $\neg P$ – is unsatisfiable, and hence that R was entailed. 35

6.2 The satisfiability phase transition for random 3-SAT: as the clause-to-variable ratio crosses approximately 4.27, instances shift from almost always satisfiable to almost always unsatisfiable, and solver runtime peaks sharply near this threshold, where instances are hardest to decide either way. 36

6.3 A breeze percept at the square right of Start implies (via $B \Leftrightarrow P_{\text{adjacent}}$) that at least one neighboring square contains a pit; resolution over the accumulated sentences from several squares can narrow down, or in some cases fully determine, which specific square is the culprit. 36

6.4 Resolution refutation deriving the empty clause from $KB \wedge \neg\alpha$, proving $KB \models R$. Each step resolves two clauses on a complementary pair of literals ($P/\neg P$, then $Q/\neg Q$, then $R/\neg R$), eliminating one variable per step. 37

7.1 The knowledge engineering process. The dashed feedback arrow reflects the iterative nature of the process: testing queries against early vocabulary choices routinely reveals gaps or ambiguities that send the engineer back to step 3. 40

7.2 A fragment of a general upper ontology, of the kind used to organize knowledge across many task-specific domains before domain-specific categories (e.g., “Purchase” as a kind of Event) are added beneath it. 41

7.3 A small family tree. Arrows point from parent to child. Ana and Ben share both parents; Cleo shares only Pedro. 42

8.1 Posterior probability of a rare disease (prior 1%) as repeated independent test results accumulate, each test being 95% accurate. A single positive result raises belief substantially but leaves real doubt; a second independent positive result nearly eliminates that doubt, illustrating how Bayesian updating combines evidence multiplicatively rather than additively. . . . 45

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

8.2 The classic Burglary–Earthquake–Alarm Bayesian network. Burglary and Earthquake are root causes assumed marginally independent of each other; both influence Alarm; and Alarm's state influences whether John or Mary calls, with John's and Mary's calls conditionally independent of each other given Alarm. 46

8.3 Query time growth for enumeration versus variable elimination on a network whose treewidth stays small as more nodes are added (a common case for networks with sparse, chain- or tree-like structure). Enumeration's exponential growth in the number of variables makes it impractical well before variable elimination's more moderate growth becomes a concern. 47

9.1 Convergence of a sampling-based estimate toward the true posterior probability as sample count increases. Early estimates fluctuate substantially; the estimate stabilizes once enough samples have been drawn, illustrating the general law-of-large-numbers behavior underlying all sampling-based approximate inference. 51

9.2 A temporal model unrolled over time: hidden states X_0, X_1, X_2, X_3 evolve via the transition model, and each hidden state emits a noisy observation E_t via the sensor model. This structure underlies hidden Markov models, Kalman filters, and dynamic Bayesian networks alike. 51

9.3 A trellis diagram for a two-state HMM. Gray arrows show all possible transitions between consecutive time steps; the highlighted blue path is the single most likely state sequence recovered by the Viterbi algorithm, which finds this path by keeping only the best-scoring incoming edge into each node rather than summing over all of them as filtering does. 52

9.4 Particle filter evolution during robot localization. Particles start uniformly distributed over the map, reflecting complete initial uncertainty, and concentrate around the true position as sensor evidence accumulates over successive time steps. 53

9.5 Illustrative convergence of estimation error as the number of samples or particles grows. Both approximate methods converge toward the true posterior as sample count increases, consistent with the law of large numbers, but neither provides the exact-answer guarantee of enumeration or variable elimination at any finite sample size. 54

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

10.1 The classic 4×3 grid-world MDP. The agent starts at S, each non-terminal move costs a small living reward (often negative), the top-right cell gives a terminal reward of $+1$, and the cell below it gives a terminal reward of -1; the black square is an impassable obstacle. The optimal policy near the -1 cell is sensitive to the size of the living-cost penalty, as explored in the exercises. 57

10.2 Typical convergence of value iteration on the gridworld of Figure 10.1: the maximum change in value across all states shrinks geometrically with each sweep, a direct consequence of the Bellman update being a contraction mapping with contraction rate governed by γ. 58

10.3 A belief state: a probability distribution over which underlying MDP state the agent actually occupies, given everything it has observed so far. A POMDP agent chooses actions based on this whole distribution, not a single most-likely state, since acting only on the mode can discard useful information about the remaining uncertainty. 59

11.1 The classic bias–variance / generalization curve. Training error decreases monotonically with model complexity, but test error follows a U-shape: too little complexity underfits (high bias), too much overfits (high variance), and the best generalization occurs at an intermediate complexity, here illustrated near degree 3. 62

11.2 5-fold cross-validation: the data is split into five folds, and the model is trained and evaluated five times, holding out a different fold as the test set each time. The final performance estimate is the average across all five folds. 62

11.3 Linear regression fit to a small dataset. The line minimizes the sum of squared vertical distances (residuals) between predicted and observed y values across all training points. 63

11.4 The logistic sigmoid function. Its output is interpreted as $P(y = 1 \mid x)$; the classifier predicts class 1 whenever $\sigma(z) > 0.5$, equivalently whenever $z = w^\top x > 0$, which is a linear condition on x. 64

11.5 A decision tree for the classic "PlayTennis" dataset, splitting first on Outlook, and further on Humidity or Wind depending on the branch. Each internal node tests a single attribute; each leaf assigns a class label. . . . 64

12.1 Three common activation functions. Sigmoid and tanh saturate for large $|z|$, contributing to vanishing gradients in deep networks; ReLU does not saturate for $z > 0$, which is a major reason it largely replaced sigmoid/tanh in hidden layers of deep networks. 67

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

12.2 A fully connected feedforward network with one hidden layer: three input units, four hidden units, and two output units. Every unit in each layer connects to every unit in the next, each connection carrying its own trainable weight. 68

12.3 Typical training and validation loss curves. Training loss decreases monotonically, but validation loss begins to rise again once the network starts overfitting, motivating early stopping at the point where validation loss is minimized. 69

12.4 A typical CNN architecture for image classification: alternating convolution and pooling layers progressively reduce spatial resolution while increasing feature depth, followed by fully connected (FC) layers producing class scores (here, 10 classes). 69

13.1 The reinforcement learning loop. Unlike the MDP setting of Chapter 10, the agent does not know $P(s' \mid s, a)$ or R in advance; it must estimate a good policy purely from the stream of states, actions, and rewards it experiences. 72

13.2 The cliff-walking gridworld. Q-learning tends to learn the shortest path directly along the cliff edge, since it learns the value of the optimal policy regardless of exploration; SARSA tends to learn a safer path further from the cliff, because its value estimates incorporate the risk of an ϵ-greedy exploratory step causing a fall. 73

13.3 Learning curves for Q-learning and SARSA on cliff-walking (Figure 13.2). Q-learning converges to a higher asymptotic reward (the shorter path) but the policy it learns occasionally falls off the cliff during ϵ-greedy exploration, so its *online* performance during training can be worse than SARSA's despite Q-learning's higher-value final policy. 74

14.1 Illustrative perplexity as a function of n-gram order. Training perplexity keeps falling as n grows (the model memorizes longer and longer exact phrases), but test perplexity bottoms out and then rises, because higher-order n-grams are seen too rarely in training to estimate reliably – a direct instance of the overfitting pattern from Chapter 11's bias–variance discussion. 77

14.2 A schematic (dimension-reduced) view of word embeddings. The offset vector from "man" to "king" is approximately parallel to the offset from "woman" to "queen," illustrating how relational structure (here, royalty) can emerge as a roughly consistent direction in embedding space purely from co-occurrence statistics, with no explicit relational supervision provided during training. 78

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

14.3 An RNN unrolled over four time steps. Each hidden state h_t depends on the current input x_t and the previous hidden state h_{t-1}, so information from x_1 can in principle influence h_4, but only by passing through three intermediate transformations, each of which can attenuate (vanishing gradient) or amplify (exploding gradient) the signal. 78

14.4 Self-attention from the word "sat" (as query) to every word in the sentence (as keys), with line opacity representing attention weight. Unlike the RNN in Figure 14.3, every word is reachable in a single step, with weight determined by learned relevance rather than by sequential distance. 79

14.5 One transformer block. Residual ("skip") connections around both the attention and feedforward sublayers, combined with normalization, are what make it practical to stack many such blocks without the vanishing-gradient problems that plague very deep or very long RNNs. 79

14.6 Schematic illustration of an empirical *scaling law*: downstream performance tends to improve smoothly and predictably as training compute (a combination of model size and data) increases, across several orders of magnitude, though the curve eventually flattens and the relationship can vary across task types. 80

2025/2026 2025/2026 2025/2026

15.1 The machine learning pipeline of Chapter 11, annotated with a representative failure mode that can be introduced at each stage. A model can be technically correct with respect to its training objective and still cause harm if the harm was introduced earlier in the pipeline, which is why auditing the whole pipeline, not just the trained model, is necessary. . . . 83

15.2 A representation gap: the training data's group composition does not match the population the system is actually deployed on. Group C is under-represented by more than half relative to the deployment population, which typically translates into worse model performance for that group even without any explicit protected attribute ever being used as a feature. 84

15.3 Schematic illustration of distributional shift: a model's accuracy on live traffic can degrade steadily after deployment as the real-world data distribution drifts away from the training distribution, even though nothing about the model itself has changed. Monitoring for this kind of drift is a core part of responsible deployment, not a one-time validation step. . . . 85

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

16.1 Dependency map of the book. The search thread (left, Chapters 2–5), the knowledge/reasoning thread (center, Chapters 6–7), and the uncertainty thread (right, Chapters 8–10) all feed into the learning thread (Chapters 11–14), which in turn motivates the ethics discussion (Chapter 15) that closes the course. 90

A.1 Timeline of the five programming assignments across the 16-week course, aligned with the chapters they draw on. 96

B.1 Term project timeline over the 16-week semester. Team formation and topic selection happen in the first two weeks; the proposal is due at Week 7; the in-lab checkpoint at Week 11; and the final presentation and report at Week 15, matching the weekly schedule in the course syllabus. 101

C.1 Two vectors x and w with angle θ between them. The dot product $w^\top x$ measures how aligned the two vectors are, independent of any coordinate system. 105

C.2 Gradient descent on $f(w) = w^2 + 0.5$. At the point shown, the negative gradient points back toward the minimum; taking repeated small steps in this direction converges to $w = 0$, the function's minimizer. 106

C.3 Two Gaussian distributions with the same mean but different variances. Variance quantifies how spread out a distribution is around its expectation, independent of where that expectation is located. 108

C.4 PCA on a 2-D dataset with correlated features. The first principal component (PC1) is the direction of greatest variance; the second (PC2), orthogonal to the first, captures most of what remains. Projecting onto PC1 alone would preserve most of the structure visible in this data while reducing it to a single dimension. 109

C.5 Growth rates of common complexity classes. Exponential growth, $O(2^n)$ or more generally $O(b^n)$, is what makes exhaustive search over large state spaces (Chapters 2–5) infeasible without pruning or heuristics, since it quickly dwarfs every polynomial alternative regardless of the constant factors involved. 110

F.1 Stage 1: a fully known, deterministic maze. A* with Manhattan distance (Chapter 3) finds the shortest path around both walls in milliseconds; there is no uncertainty to reason about yet. 123

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

F.2 Stage 2: the agent has only explored the shaded region near its start. A model-based reflex agent (Chapter 1) that maintains a partial map and re-plans with A* every time new wall information is observed – an approach sometimes called "repeated A*" – handles this setting well as long as the maze does not change while being explored. 124

F.3 Stage 3: a particle-filter-style belief over the agent's true position (Chapter 9), with opacity representing probability mass. The agent should choose actions that make sense across this whole distribution – for instance, favoring an action safe under all likely positions – rather than acting as if the single most likely cell were certainly correct. 125

F.4 Stage 4: learning curve for a Q-learning agent (Chapter 13) discovering the maze and the reward location purely through trial-and-error exploration. Early episodes involve extensive random wandering; as the action-value estimates improve, the agent converges toward a near-optimal number of steps, approaching (but not necessarily matching exactly) the Stage-1 A* solution length, since some residual exploration continues under an ϵ-greedy policy. 125

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

List of Tables

رومانی صبحی رزق جرجس مطر

1.1 Task environment classification for four example domains. 3

2.1 Trace of uniform-cost search on Figure 2.3 from A to G. Each row shows the node popped for expansion and the frontier immediately afterward, ordered by $g(n)$. 15

3.1 $f(n) = g(n)+h(n)$ values during A* search on Figure 3.4, with straight-line distances $h(\text{A}) = 6.4$, $h(\text{B}) = 3.2$, $h(\text{C}) = 4.1$, $h(\text{G}) = 0$. 21

4.1 Illustrative effect of constraint propagation on backtracking search node counts, for CSPs of increasing size drawn from the same family of random instances. 25

2025/2026 2025/2026 2025/2026

5.1 Rough branching factors and typical game lengths for several classic games, illustrating why exhaustive minimax is feasible for some and requires MCTS-style approximation for others. 31

6.1 Truth table for the material conditional $P \Rightarrow Q$, frequently a point of confusion since it is true whenever P is false, regardless of Q. 34

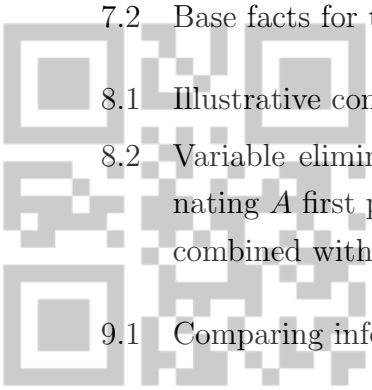
7.1 Common natural-language quantifier patterns and their correct first-order translations; a frequent student error is pairing $\forall$ with $\wedge$ or $\exists$ with $\Rightarrow$, both of which produce sentences with unintended, usually vacuous or overly weak, truth conditions. 39

7.2 Base facts for the family tree in Figure 7.3. 42

8.1 Illustrative conditional probability table for the Alarm node in Figure 8.2. 46

8.2 Variable elimination trace for $P(C)$ on the chain $A \to B \to C$. Eliminating A first produces an intermediate factor over B alone, which is then combined with $P(C \mid B)$ and summed over B. 48

9.1 Comparing inference methods introduced in Chapters 8–9. 53



UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

10.1 How the optimal policy near the penalty cell in Figure 10.1 changes as the living-cost reward $R(s)$ varies, illustrating the qualitative shift from risk-averse to risk-tolerant behavior. 59

15.1 A stylized example showing why equal accuracy across two groups does not imply equal treatment. Both groups have 90% overall accuracy, but Group B suffers a much higher false-negative rate – relevant if a false negative means, e.g., a missed loan approval or missed medical diagnosis. 84

15.2 A starting checklist for evaluating a deployed or proposed AI system, organized by this chapter's sections. 87

16.1 The shared representation/objective/algorithm pattern, instantiated across four representative techniques from the course. 92

16.2 A chapter-by-chapter exam preparation summary. 93

A.1 Default grading rubric applied to each programming assignment. 99

C.1 Common mathematical notation used throughout this book. 110

D.1 Frequently encountered errors and their typical causes. 116

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

List of Algorithms

1 Breadth-First Search 12
2 AC-3 25
3 Alpha-Beta (MAX node) 30
4 Value Iteration 57

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 1

What Is AI? Agents, Environments, and Rationality

1.1 Defining Artificial Intelligence

Artificial intelligence can be defined along two axes: whether a system is meant to think or act, and whether it is judged against human performance or an ideal standard of correctness. This gives four traditional framings: systems that think like humans (cognitive modeling), systems that think rationally (logic-based reasoning), systems that act like humans (the Turing test tradition), and systems that act rationally. Modern AI, and this course, is organized primarily around the last of these: an intelligent agent is one that acts to achieve the best expected outcome given what it knows and perceives.

2025/2026 2025/2026 2025/2026

Historically the field has moved through several distinct emphases. Early work (1950s–1960s) focused on general-purpose reasoning and search, exemplified by programs that proved theorems or played simple games. The 1970s–1980s saw a shift toward encoding expert knowledge directly (expert systems), which succeeded in narrow domains but proved brittle and expensive to maintain. From the 1990s onward, the field increasingly turned to statistical and learning-based methods, a trend that accelerated sharply after 2012 with the resurgence of neural networks trained on large datasets, and again after 2017 with the transformer architecture (Chapter 14) enabling large language models. Each shift did not discard earlier ideas so much as add a new layer: search (Chapters 2–5) and logic (Chapters 6–7) remain the right tools for many problems even in an era dominated by learned models.

Key Idea

An agent is anything that perceives its environment through *sensors* and acts upon that environment through *actuators*. A *rational agent* selects, for every possible percept sequence, the action expected to maximize its performance measure, given

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

its built-in knowledge and whatever it has perceived so far.

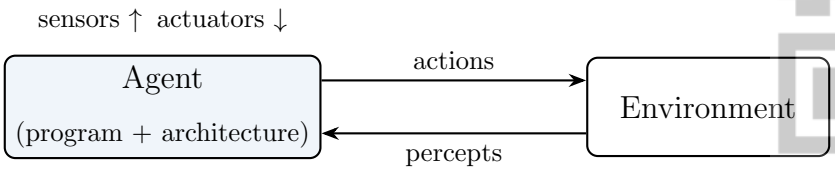


Figure 1.1: The basic agent–environment interaction loop. The agent receives percepts through sensors, updates its internal state, selects an action via its agent program, and executes it through actuators; the resulting change in the environment produces the next percept.

1.2 The Agent Function and Agent Program

An agent's behavior is described abstractly by an *agent function* mapping percept sequences to actions. In practice this function is implemented by an *agent program* running on some physical architecture. The agent function may be exponentially large or infinite (if percept sequences are unbounded), so the program is what actually determines the system's behavior, typically by maintaining an internal state and updating it as new percepts arrive.

2025/2026 2025/2026 2025/2026

It is worth being precise about the distinction: the agent *function* is a mathematical object, a lookup table in principle, mapping every possible percept history to an action. No one implements an agent this way, because the table is far too large even for simple environments. The agent *program* is the actual, compact implementation – a piece of code – whose input/output behavior realizes the desired agent function without ever storing the full table explicitly. Much of this book is about designing agent programs whose behavior approximates rational agent functions using tractable amounts of computation and memory.

1.3 Rationality, Omniscience, and Autonomy

Rationality is not omniscience. A rational agent maximizes *expected* performance given the information available; it is not blamed for actions that turn out badly because of information it had no way of obtaining. Rationality also does not require perfection of execution (an agent could act rationally with respect to a plan while a component of the plan fails), and it does not preclude gathering more information before acting or learning from experience – both are often rational. An agent is *autonomous* to the extent that its behavior is determined by its own experience rather than solely by built-in knowledge.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Worked Example

Consider an agent crossing a street. A rational agent looks both ways before crossing; if it is rational and looks both ways and sees no traffic, and is then struck by a vehicle running a red light at high speed from an obscured angle, this does not retroactively make the agent's decision irrational. Rationality is evaluated against the information available *at the time of the decision*, not against the outcome.

1.4 Task Environments and PEAS

Every agent design begins with specifying the task environment: Performance measure, Environment, Actuators, Sensors (PEAS). Task environments are classified along several dimensions that determine which techniques apply:

- **Fully vs. partially observable**: can sensors access the complete state relevant to the decision at each point?
- **Single-agent vs. multi-agent**: does another agent's behavior need to be modeled, and is it cooperative or adversarial?
- **Deterministic vs. stochastic**: is the next state determined by the current state and action, or is there uncertainty?

2025/2026 2025/2026 2025/2026

- **Episodic vs. sequential**: are experiences divided into independent episodes, or does each action affect future decisions?
- **Static vs. dynamic**: can the environment change while the agent deliberates?
- **Discrete vs. continuous**: are states, time, percepts, and actions discrete or continuous?
- **Known vs. unknown**: does the agent know the rules governing the environment's transitions and rewards?

Table 1.1: Task environment classification for four example domains.

Environment	Observable	Agents	Deterministic	Episodic
Chess (with clock)	Fully	Multi	Deterministic	Sequential
Poker	Partially	Multi	Stochastic	Sequential
Autonomous driving	Partially	Multi	Stochastic	Sequential
Image classification	Fully	Single	Deterministic	Episodic
Crossword solving	Fully	Single	Deterministic	Sequential

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

1.5 Agent Program Types

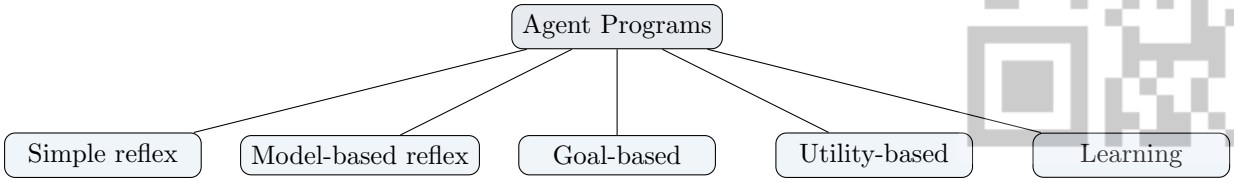


Figure 1.2: A taxonomy of agent program designs, in order of increasing capability. Each later type subsumes the reasoning of the types before it: a learning agent, for instance, can in principle acquire any of the other four behaviors through experience.

- **Simple reflex agents** select actions based only on the current percept, using condition–action rules of the form "if condition then action." They are fast and easy to implement but fail whenever the correct decision depends on unobserved state – for example, a simple reflex vacuum agent cannot remember which rooms it has already cleaned.

- **Model-based reflex agents** maintain internal state that tracks aspects of the world not currently visible, updated using a model of how the world evolves and how the agent's own actions affect it. This internal state lets the agent behave sensibly even under partial observability, at the cost of needing an accurate transition model.

- **Goal-based agents** additionally consider future consequences of actions to decide whether they achieve a goal, often requiring search or planning (Chapters 2–5) to find an action sequence that reaches the goal. Goal-based agents are more flexible than reflex agents because the goal can be changed without rewriting the condition–action rules.

- **Utility-based agents** generalize goals to a utility function, allowing trade-offs among competing objectives (speed vs. safety, cost vs. quality) and handling uncertainty via expected utility (Chapter 10), rather than a binary goal-achieved/not-achieved distinction.

- **Learning agents** improve their performance element over time using feedback from a critic that evaluates outcomes against a fixed performance standard, guided by a problem generator that suggests exploratory actions the agent would not otherwise take. Every other agent type in this taxonomy can, in principle, be the "performance element" that a learning agent improves.

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Worked Example

A robot vacuum cleaner: the performance measure might reward cleanliness and battery life; the environment is a room with furniture and dirt; the actuators are wheels and a suction unit; the sensors are bump sensors, dirt sensors, and possibly a camera. The environment is partially observable (the layout beyond sensor range is unknown), stochastic (wheels can slip), sequential (past cleaning affects what remains), and dynamic if people are moving furniture while it operates. A simple reflex design ("if dirt sensed, vacuum; if bumped, turn") performs adequately in a single small room, but a model-based design that tracks which areas have already been cleaned performs substantially better in a multi-room home, since it avoids repeatedly revisiting already-clean areas at the expense of neglected ones.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

1.6 Environment Programs and the Agent–Environment Contract

It is useful to formalize the agent–environment interaction as a simulation loop, since this is how virtually every agent in this book is eventually tested. An environment program maintains the true state of the world (possibly hidden from the agent), exposes a percept to the agent at each time step, accepts an action in return, and updates its internal state accordingly, typically also computing a running performance score. Figure 1.3 depicts this loop as an explicit simulation, distinguishing the environment's true state from the percept the agent actually receives – a distinction that matters a great deal once we reach partially observable environments in Chapters 8–10.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

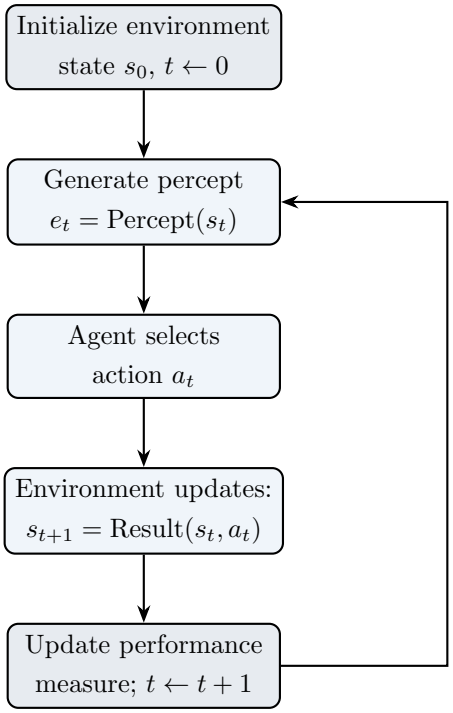


Figure 1.3: The environment simulation loop used throughout this book to evaluate agents. The feedback arrow on the right indicates that this loop runs until a terminal condition (goal reached, episode length exceeded, or agent failure) is met.

This separation between s_t (true state) and e_t (percept) is what makes partial observability precise: an environment is fully observable exactly when e_t determines s_t uniquely; otherwise the agent must maintain a belief state, as discussed in Chapter 8 onward.

2025/2026 2025/2026 2025/2026

1.7 Multi-Agent Environments

Many important environments contain more than one agent. When other agents' goals are aligned with the agent's own (cooperative multi-agent), the environment can often still be treated as effectively single-agent by treating other agents as an unpredictable but non-adversarial part of the environment's dynamics. When other agents' goals directly oppose the agent's own (competitive multi-agent, as in two-player zero-sum games), rationality must account for the other agent's own rational behavior – an agent that ignores this and simply maximizes expected utility over a fixed environment model can be exploited, which is precisely the motivation for the adversarial search techniques developed in Chapter 5. Between these extremes lie mixed-motive environments (negotiation, resource allocation, traffic), which this course touches on only briefly but which are the subject of game theory and multi-agent systems more broadly.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

1.8 Benchmarks and How the Field Measures Progress

Because "intelligence" is difficult to define operationally, the field has historically made progress by agreeing on specific, measurable benchmark tasks and tracking performance on them over time: game-playing benchmarks (chess, Go, poker), the ImageNet image-classification benchmark that catalyzed the deep learning resurgence of the 2010s, and, more recently, broad suites of language and reasoning tasks used to evaluate large language models. A running theme in this course is that progress on a benchmark is not the same as progress on the underlying capability the benchmark was designed to measure – a distinction worth keeping in mind particularly when reading claims of AI systems "passing" a given test, since tests designed for humans do not automatically transfer their intended meaning when applied to a system with a very different failure profile than a human.

1.9 Looking Ahead

The rest of this book develops techniques for building rational agents across increasingly rich task environments: deterministic single-agent search (Chapters 2–4), adversarial environments (Chapter 5), logical and probabilistic knowledge representation (Chapters 6–9), sequential decision-making under uncertainty (Chapter 10), and learning-based agents (Chapters 11–14), closing with the societal context in which these agents are deployed (Chapter 15). Figure 1.2's taxonomy recurs throughout: search algorithms implement the goal-based agent's internal deliberation; Bayesian networks and MDPs implement the utility-based agent's reasoning under uncertainty; and the learning chapters show how the performance element of any of these agent types can itself be learned from data rather than hand-specified.

2025/2026 2025/2026 2025/2026

Chapter Summary

- An agent perceives via sensors and acts via actuators; rationality means maximizing expected performance given available information, not achieving perfect outcomes or omniscience.
- Task environments are classified along seven dimensions (PEAS, observability, agent count, determinism, episodicity, dynamics, discreteness, and known/unknown), which together determine which techniques in the rest of this book apply.
- Agent programs range from simple reflex to model-based, goal-based, utility-based, and learning agents, each adding capability at the cost of additional

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

required knowledge or computation.

Exercises

1.1 Give a PEAS description for (a) an autonomous highway driving system and (b) an automated essay-grading system. For each, classify the environment along all seven dimensions in Section 1.4.

1.2 Explain why rationality does not imply omniscience. Construct a short example in which a rational agent's action leads to a bad outcome through no fault of its decision-making.

1.3 Is a thermostat an agent under the definition given here? Is it a rational agent? Justify your answer using the PEAS framework and Figure 1.1.

1.4 Classify each of the following environments as fully or partially observable, deterministic or stochastic, and episodic or sequential: chess with a clock, poker, a self-checkout scanner, and image classification of a single photo. Compare your answers to Table 1.1.

1.5 For each of the five agent program types in Figure 1.2, give one task environment where that type is sufficient and one where it fails, explaining why.

1.6 Explain, using the vacuum-cleaner example, why a model-based reflex agent can outperform a simple reflex agent even though neither one plans ahead or considers explicit goals.

1.7 Briefly describe one historical shift in AI methodology mentioned in Section 1.1 (from reasoning/search, to expert systems, to statistical learning, to large-scale neural networks) and explain what limitation of the prior approach motivated it.

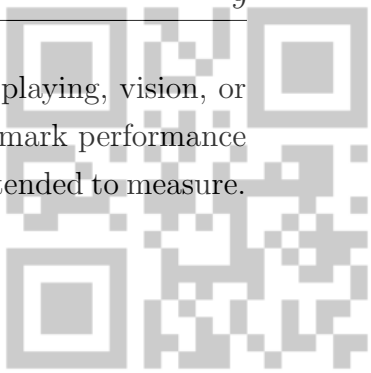
1.8 Using Figure 1.3, explain precisely where partial observability enters the simulation loop, and why an agent operating in a partially observable environment cannot, in general, implement its agent function as a simple lookup from the current percept alone.

1.9 Give an example of a mixed-motive multi-agent environment (neither purely cooperative nor purely zero-sum) and explain why neither the model-based/utility-based agent framework nor the pure minimax framework of Chapter 5 is fully adequate for it on its own.

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

1.10 Choose a real AI benchmark you are familiar with (e.g., a game-playing, vision, or language benchmark) and discuss one way in which strong benchmark performance might fail to indicate the broader capability the benchmark was intended to measure.



رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 2

Uninformed Search

2.1 Problem-Solving Agents

Many tasks can be cast as a *search problem* defined by: a state space, an initial state, a set of actions available in each state, a transition model, a goal test, and a path cost function. A *solution* is a sequence of actions leading from the initial state to a goal state; an *optimal* solution has the lowest path cost among all solutions.

Formulating a problem well is itself a skill. A common source of difficulty is choosing the wrong level of *abstraction*: a state representation that is too detailed (e.g., tracking every joint angle of a robot arm when only the position of its gripper matters for the task) makes the search space needlessly large, while one that is too coarse can omit information the goal test or transition model actually needs. A good abstraction discards details irrelevant to the task while preserving everything the search needs to reason correctly about costs and goal satisfaction.

Key Idea

Search algorithms explore a state space by maintaining a *frontier* of nodes generated but not yet expanded, and an *explored set* of nodes already expanded. Algorithms differ chiefly in how they choose which frontier node to expand next.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

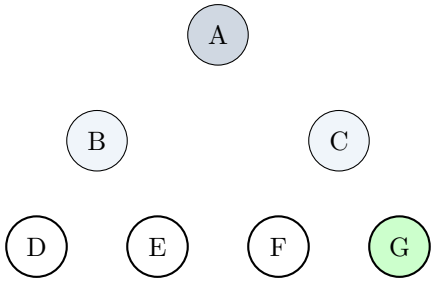


Figure 2.1: A search tree rooted at start state A. Shaded nodes B and C have been expanded; the goal, G, has just been generated. The order in which nodes such as D, E, F, and G are discovered and expanded is what distinguishes BFS, DFS, and their variants.

رومانی صبحی رزق جرجس مطر

2.2 Evaluating Search Strategies

Search strategies are compared on four criteria: *completeness* (does it always find a solution if one exists?), *optimality* (does it find the lowest-cost solution?), *time complexity*, and *space complexity*, typically expressed in terms of branching factor b, solution depth d, and maximum depth m. Space complexity is often the binding practical constraint: a search that could in principle run for hours if given enough memory instead fails immediately once available RAM is exhausted, which is why several algorithms in this chapter exist specifically to trade time for memory.

2.3 Breadth-First Search

Breadth-first search (BFS) expands the shallowest unexpanded node, using a FIFO queue for the frontier. It is complete and, when all step costs are equal, optimal, but has time and space complexity $O(b^{d+1})$, making memory the binding constraint in practice: with branching factor 10 and solution depth 10, BFS must in principle hold on the order of 10^{10} nodes in memory, well beyond what a typical machine can provide.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Algorithm 1 Breadth-First Search

```
frontier ← FIFO queue containing initial node
explored ← ∅
while frontier is not empty do
    node ← Pop(frontier)
    if GoalTest(node) then return solution
    end if
    Add node to explored
    for each action in Actions(node) do
        child ← ChildNode(node, action)
        if child not in explored or frontier then
            Add child to frontier
        end if
    end for
end while
return failure
```

2.4 Uniform-Cost Search

Uniform-cost search (UCS) expands the node with the lowest path cost $g(n)$ from the start, using a priority queue. It is complete and optimal for any step costs ≥ 0, generalizing BFS (which is UCS with unit costs). Its complexity is expressed as $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where C^* is the optimal cost and ϵ the minimum step cost. Two refinements matter in practice: UCS tests for a goal only when a node is *popped* for expansion, not when it is generated, and it must handle the case where a cheaper path to an already-frontier node is discovered, updating that node's cost rather than adding a duplicate.

2.5 Depth-First Search

Depth-first search (DFS) expands the deepest unexpanded node, using a LIFO stack. It requires only $O(bm)$ space, since it only needs to remember the single path from the root to the current node plus the unexpanded siblings along that path, but it is neither complete (it can loop forever in infinite or cyclic spaces) nor optimal, since it commits to the first solution path it finds regardless of cost. *Depth-limited search* bounds the depth to avoid infinite descent at the cost of incompleteness if the limit is too shallow.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

2.6 Iterative Deepening Search

Iterative deepening depth-first search (IDS) runs depth-limited search with increasing limits $0, 1, 2, \ldots$ until a solution is found. It combines DFS's space efficiency, $O(bd)$, with BFS's completeness and optimality (under uniform costs), at the cost of re-generating shallow nodes repeatedly.

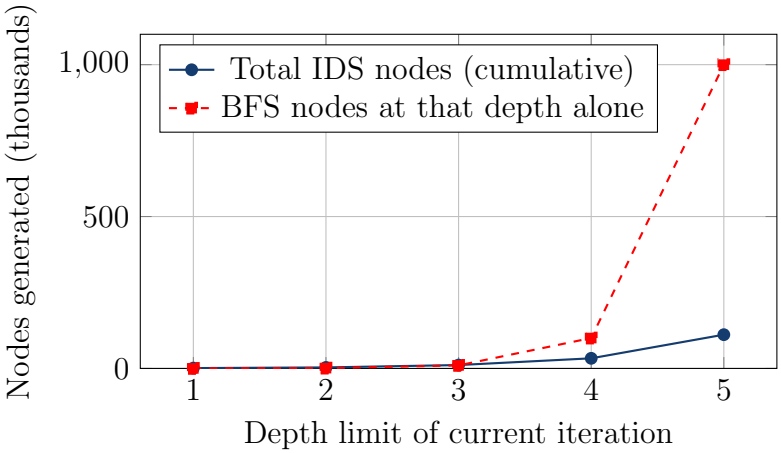


Figure 2.2: Illustrative comparison (branching factor $b = 10$) of cumulative nodes generated by iterative deepening across all its restarts versus nodes generated by a single BFS pass to the same depth. The overhead of IDS's repeated shallow re-exploration is small relative to the cost of the final, deepest layer, which dominates both curves.

2025/2026 2025/2026 2025/2026

The overhead is asymptotically negligible because the search tree is dominated by its last level: for branching factor b, the number of nodes at depth d is roughly $b - 1$ times the total number of nodes at all shallower depths combined, so repeating the shallow levels costs comparatively little, as shown in Figure 2.2.

2.7 Bidirectional Search

When the goal state is known explicitly and actions are reversible, bidirectional search runs two simultaneous searches – forward from the start and backward from the goal – meeting in the middle. This reduces time complexity to roughly $O(b^{d/2})$, a substantial saving, at the cost of needing an explicit goal state, reversible actions, and an efficient way to check whether the two frontiers have met.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

2.8 Comparing the Strategies

Strategy	Complete?	Optimal?	Time	Space
BFS	Yes	Yes*	$O(b^{d+1})$	$O(b^{d+1})$
UCS	Yes	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	same
DFS	No	No	$O(b^m)$	$O(bm)$
IDS	Yes	Yes*	$O(b^d)$	$O(bd)$
Bidirectional	Yes*	Yes*	$O(b^{d/2})$	$O(b^{d/2})$

*optimal only under uniform step costs; conditions apply as noted in text.

2.9 Repeated States and Graph Search

State spaces are frequently graphs rather than trees: the same state can be reached by multiple action sequences (for example, in a grid, moving right then up reaches the same square as moving up then right). Tree search, which re-explores such states every time they are reached, can be exponentially wasteful or even fail to terminate in the presence of cycles. *Graph search* fixes this by maintaining an explored set and refusing to re-expand any state already in it, at the cost of the extra memory needed to store that set – a trade-off that is almost always worthwhile for problems with significant state repetition.

2025/2026 2025/2026 2025/2026

2.10 Worked Example: Route-Finding

A classic illustration is planning a driving route between two cities on a road map, where states are cities, actions correspond to roads connecting them, the transition model is deterministic (taking a road leads to the city at its other end), and path cost is the total distance traveled. Figure 2.3 shows a small road network; Table 2.1 traces uniform-cost search from city A to city G on this network, illustrating how the frontier's priority queue evolves as nodes are expanded in order of increasing $g(n)$.

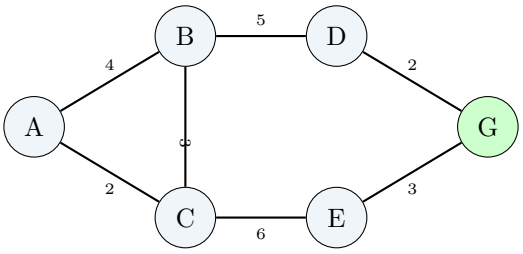


Figure 2.3: A small road network. Edge labels give road distances (path costs). Cities B and C are directly connected, giving uniform-cost search a shortcut that a naive greedy choice at A would miss.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Table 2.1: Trace of uniform-cost search on Figure 2.3 from A to G. Each row shows the node popped for expansion and the frontier immediately afterward, ordered by $g(n)$.

Step	Node expanded (g)	Frontier after expansion
1	A (0)	B(4), C(2)
2	C (2)	B(4), E(8) [via C, C–B edge gives B(5), worse, discarded]
3	B (4)	D(9), E(8)
4	E (8)	D(9), G(11)
5	D (9)	G(11) [D–G gives G(11), tie, first one kept]
6	G (11)	**goal reached, cost 11**

Note that the path A–C–E–G, discovered by following lowest cumulative cost at each step, achieves the same total cost (11) as A–C–B–D–G in this particular example; the trace also illustrates the necessary bookkeeping step (line 4 of Algorithm 2.1's generalization to UCS) of discarding a newly discovered but more expensive route to an already-frontier node.

Chapter Summary

- Search algorithms differ chiefly in frontier ordering: FIFO (BFS), priority by $g(n)$ (UCS), LIFO (DFS), or repeated bounded DFS (IDS).
- BFS and UCS are complete and optimal (under appropriate cost conditions) but memory-hungry; DFS is memory-efficient but neither complete nor optimal; IDS combines the best of both.
- Graph search (with an explored set) avoids the redundant or infinite re-exploration that plain tree search suffers in state spaces with repeated states or cycles.

Exercises

2.1 Trace BFS and DFS on the tree in Figure 2.1, extended with two additional children under D, showing the frontier contents at each step.

2.2 Prove that uniform-cost search is optimal when all step costs are strictly positive. Give a counterexample showing what can go wrong if negative step costs are allowed.

2.3 Using the reasoning behind Figure 2.2, derive an expression for the ratio of nodes generated by IDS to nodes generated by a single BFS pass to the same depth, as a function of b.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

2.4 Under what specific conditions is bidirectional search guaranteed to be both complete and correct? Give an example problem where it cannot be applied.

2.5 Explain the difference between tree search and graph search, and construct a small state space (as a diagram) where tree search fails to terminate but graph search succeeds.

2.6 **Implementation.** Implement BFS, UCS, and IDS for the 8-puzzle. Report nodes expanded and solution depth for five random solvable instances, and discuss which algorithm you would choose in practice and why.

2.7 Using Figure 2.3 and Table 2.1, verify by hand that uniform-cost search's tie-breaking choice at step 5 does not affect optimality, and explain in general why UCS can be indifferent among multiple optimal-cost paths without sacrificing correctness.

2.8 Modify the road network in Figure 2.3 so that greedy behavior at the start node (always choosing the cheapest single edge) leads to a suboptimal overall path, and confirm using a UCS trace that UCS still finds the true optimum.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 3

Informed Search and Heuristics

3.1 Heuristic Functions

An *informed* search strategy uses problem-specific knowledge beyond the problem definition, encoded in a heuristic function $h(n)$ estimating the cost from node n to the nearest goal. A heuristic is *admissible* if it never overestimates the true cost, and *consistent* (or monotone) if, for every node n and successor n' generated by action a, $h(n) \leq c(n, a, n') + h(n')$. Every consistent heuristic is admissible; the converse does not hold.

3.2 Greedy Best-First Search

Greedy best-first search expands the node that appears closest to the goal, ordering the frontier by $h(n)$ alone. It is fast in practice but is neither complete (in infinite spaces) nor optimal, since it ignores the cost already incurred and can be lured down an expensive path that merely looks promising at each step.

3.3 A* Search

Key Idea

A* search orders the frontier by $f(n) = g(n) + h(n)$, the estimated cost of the cheapest solution through n. With an admissible heuristic (for tree search) or a consistent heuristic (for graph search), A* is both complete and optimal.

The optimality proof relies on the fact that a consistent heuristic makes $f(n)$ non-decreasing along any path, so the first time A* expands a goal node, no cheaper path to that goal remains on the frontier. A* expands every node with $f(n) < C^*$, and no node

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

with $f(n) > C^*$, giving it a strong optimality guarantee among algorithms using the same heuristic information: no other algorithm using the same h and expanding search from the same start node can guarantee fewer node expansions while remaining complete and optimal.

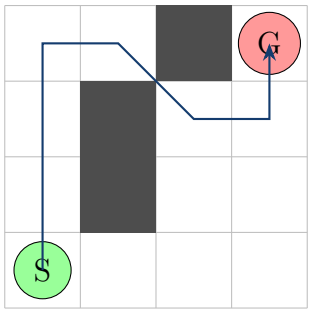


Figure 3.1: A* navigating a grid with an obstacle (dark region), using Manhattan distance as the heuristic. The heuristic guides expansion toward the goal while $g(n)$ ensures the detour around the obstacle is still recognized as necessary once the direct route is blocked.

3.4 Designing Heuristics

Good heuristics are often derived from *relaxed problems* – versions of the original problem with fewer restrictions on actions. For the 8-puzzle, the number of misplaced tiles is admissible (it relaxes "a tile can only move to an adjacent empty square" to "a tile can move anywhere"), and Manhattan distance is a tighter admissible heuristic (relaxing only the empty-square restriction). When multiple admissible heuristics $h_1, \ldots, h_k$ are available, $h(n) = \max(h_1(n), \ldots, h_k(n))$ is also admissible and dominates each individually.

2025/2026 2025/2026 2025/2026

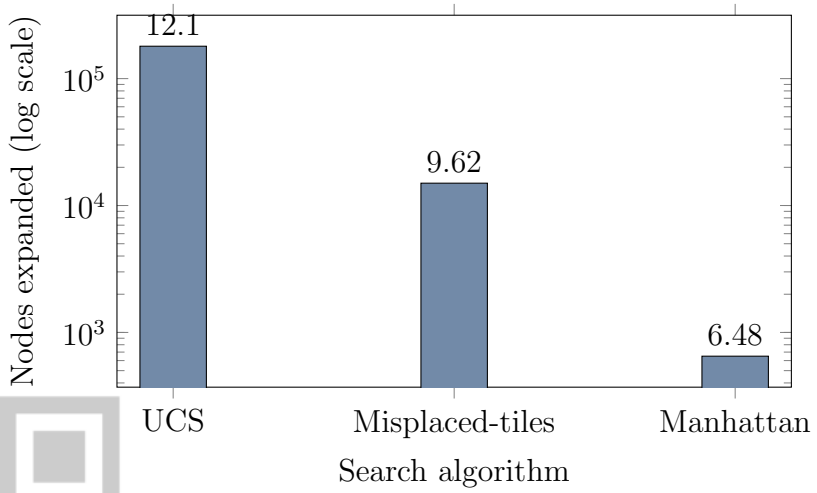


Figure 3.2: Illustrative node expansions for a moderately difficult 8-puzzle instance under uniform-cost search versus A* with two admissible heuristics. Manhattan distance dominates misplaced-tiles, so A* using it never expands more nodes, and empirically expands dramatically fewer.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Worked Example

For the 8-puzzle, Manhattan distance dominates the misplaced-tiles heuristic: it is always at least as large, so A* using Manhattan distance never expands more nodes. Figure 3.2 shows the effect empirically: on a moderately difficult instance, Manhattan distance can reduce node expansions by more than two orders of magnitude relative to uninformed search, and by more than an order of magnitude relative to the weaker misplaced-tiles heuristic.

رومانی صبحی رزق جرجس مطر

3.5 Memory-Bounded Variants

A* can exhaust memory since it retains every generated node. *Iterative-deepening A** (IDA*) instead performs a series of depth-first searches bounded by increasing f-cost thresholds, achieving A*'s optimality with linear space. *Recursive best-first search* (RBFS) and *Simplified Memory-bounded A** (SMA*) offer further trade-offs between memory use and node re-expansion, with SMA* dropping the least promising leaf node from memory when full and backing up its value into its parent so that information is not entirely lost.

2025/2026 2025/2026 2025/2026

3.6 Local Search: Hill Climbing and Simulated Annealing

When the path to the goal is irrelevant and only the goal state matters (as in many optimization problems), local search methods operate on a single current state, moving to neighboring states. *Hill climbing* always moves to the best neighbor but can become stuck at local maxima, plateaus, or ridges. *Simulated annealing* allows occasional moves to worse states, accepted with probability $e^{\Delta E/T}$, with temperature T decreasing over time; this allows escape from local maxima and converges to a global optimum under a sufficiently slow cooling schedule.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

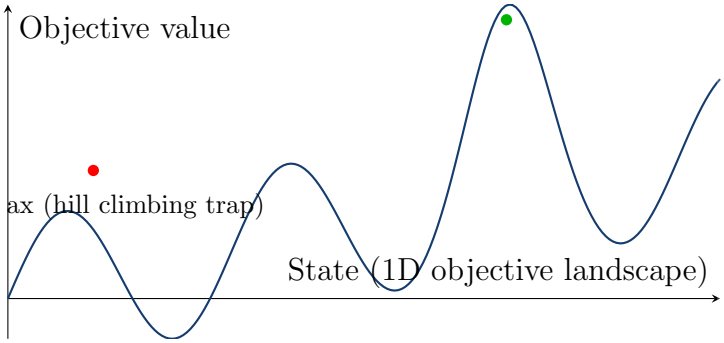


Figure 3.3: A 1-D objective landscape illustrating why hill climbing can become trapped: starting near the left peak, a hill-climbing agent never discovers the higher peak on the right because every step toward it initially decreases the objective. Simulated annealing's willingness to accept occasional downhill moves allows escape from such traps.

Genetic algorithms maintain a population of candidate states and combine them via selection (favoring higher-fitness individuals), crossover (combining parts of two parent states), and mutation (random perturbation), drawing an explicit analogy to biological evolution as a search process over a landscape of possible solutions.

3.7 Worked Example: Straight-Line Distance as a Heuristic

For route-finding on a map (Section 2.7's example), a natural admissible heuristic is the straight-line (Euclidean) distance to the goal, since no legal route can ever be shorter than a straight line between two points. Figure 3.4 shows A* using this heuristic on a small network, and Table 3.1 traces the resulting $f(n) = g(n) + h(n)$ values driving node selection.

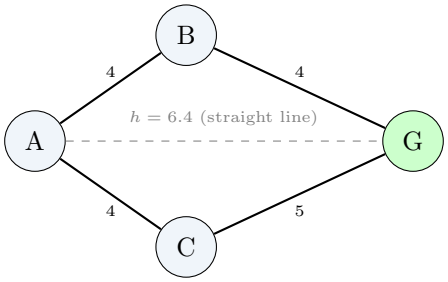


Figure 3.4: A* with straight-line-distance heuristic. The dashed line shows the straight-line distance from A to the goal, which lower-bounds every real route and is therefore admissible.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Table 3.1: $f(n) = g(n) + h(n)$ values during A* search on Figure 3.4, with straight-line distances $h(\text{A}) = 6.4$, $h(\text{B}) = 3.2$, $h(\text{C}) = 4.1$, $h(\text{G}) = 0$.

Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
A	0	6.4	6.4
B (via A)	4	3.2	7.2
C (via A)	4	4.1	8.1
G (via B)	8	0	8.0
G (via C)	9	0	9.0

A* expands B before C (lower f), then finds G via B with $f = 8.0$, which is lower than continuing to explore via C ($f = 8.1$ for C itself, already worse); the algorithm correctly identifies A–B–G, cost 8, as optimal without ever needing to fully expand C.

Chapter Summary

- Heuristics estimate remaining cost to the goal; admissibility (never overestimating) and consistency (a stronger, monotone condition) determine what optimality guarantees a heuristic-guided search retains.
- A* orders the frontier by $f(n) = g(n) + h(n)$ and is both complete and optimal given an admissible (tree) or consistent (graph) heuristic, while expanding no more nodes than any other optimal algorithm using the same heuristic.
- Good heuristics come from relaxed problems; memory-bounded variants (IDA*, SMA*) trade some efficiency for linear space; local search methods (hill climbing, simulated annealing, genetic algorithms) trade completeness guarantees for scalability when only the goal state, not the path, matters.

Exercises

3.1 Prove that if h_1 and h_2 are both admissible, then $h(n) = \max(h_1(n), h_2(n))$ is admissible. Is $h_1 + h_2$ admissible in general? Justify.

3.2 Show that Manhattan distance is a consistent heuristic for the 8-puzzle.

3.3 Explain why a consistent heuristic guarantees that A* graph search never needs to re-expand a node with a better path found later.

3.4 Using Figure 3.1 as a template, construct a grid with two obstacles where greedy best-first search finds a strictly worse path than A*, and explain the mechanism.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

3.5 **Implementation.** Extend your Chapter 2 8-puzzle solver with A* using both the misplaced-tiles and Manhattan-distance heuristics. Reproduce a chart like Figure 3.2 using your own measured node counts.

3.6 Explain how simulated annealing's acceptance criterion changes qualitatively as $T \to 0$, and why this matters for convergence, with reference to Figure 3.3.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 4

Constraint Satisfaction Problems

4.1 CSP Formulation

A constraint satisfaction problem (CSP) is defined by a set of variables $X_1, \ldots, X_n$, each with a domain D_i of possible values, and a set of constraints specifying allowable combinations of values for subsets of variables. Casting a problem as a CSP – rather than a general search problem – exposes structure that generic algorithms can exploit, such as constraint propagation and problem decomposition, because the state is factored into named variables rather than an opaque, problem-specific representation.

2025/2026 2025/2026 2025/2026

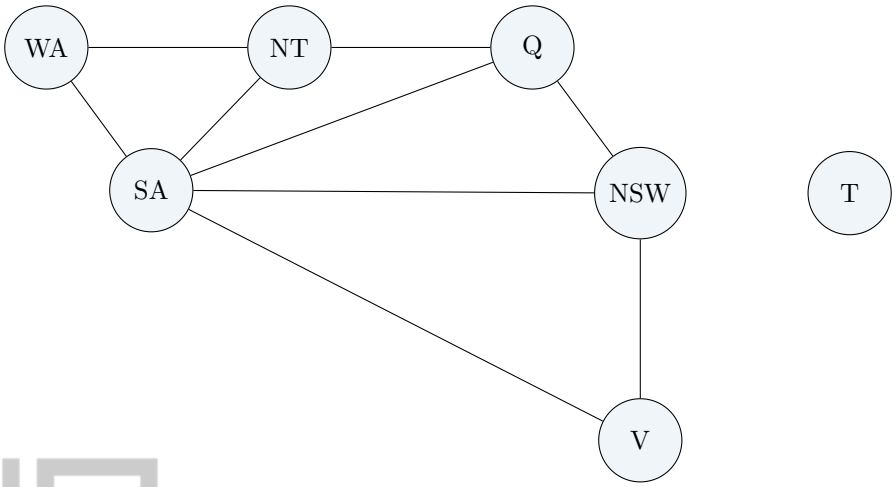


Figure 4.1: The constraint graph for map-coloring Australia's mainland states and territories plus Tasmania. Each edge is a binary constraint requiring the two adjacent regions to take different colors; Tasmania (T) is disconnected from the mainland graph, so it can take any color independent of the rest.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

4.2 Backtracking Search

Because CSPs have a factored state representation, a generic backtracking algorithm suffices: assign values to variables one at a time, checking constraints as each assignment is made, and backtrack when no legal value remains for the current variable. Uninformed backtracking is greatly improved by three general-purpose heuristics:

- **Minimum remaining values (MRV)**: choose the variable with the fewest legal values remaining, failing fastest so that dead ends are discovered as early in the search as possible.
- **Degree heuristic**: among ties, choose the variable involved in the most constraints on unassigned variables, useful as a tie-breaker for MRV and as the initial variable choice before any domains have been reduced.
- **Least constraining value**: order value choices to rule out the fewest values in neighboring variables, preserving future flexibility for the rest of the search.

2025/2026 2025/2026

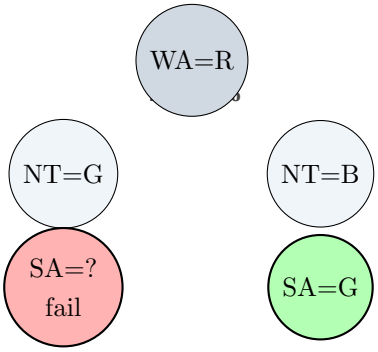


Figure 4.2: A fragment of backtracking search for map coloring. After fixing WA=Red, trying NT=Green leaves SA with no legal color (both its neighbors, WA and NT, would occupy Red and Green), forcing a backtrack; trying NT=Blue instead leaves SA=Green as a valid continuation. MRV would have preferred to assign the most constrained variable, often SA, earlier to detect such failures sooner.

4.3 Constraint Propagation

Arc consistency strengthens backtracking by removing values from domains that could never appear in a consistent solution. An arc $X_i \to X_j$ is consistent if, for every value in D_i, some value in D_j satisfies the constraint between them. The AC-3 algorithm enforces arc consistency across the whole CSP by maintaining a queue of arcs to recheck whenever a domain shrinks, running in $O(cd^3)$ time for c binary constraints and domain size d.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Algorithm 2 AC-3

```
queue ← all arcs in the CSP
while queue is not empty do
    (X_i, X_j) ← Pop(queue)
    if Revise(X_i, X_j) then
        if D_i is empty then return failure
        end if
        Add (X_k, X_i) to queue for every neighbor X_k of X_i, k ≠ j
    end if
end while
return true
```

Running arc consistency once before search, or after every assignment during search (a technique called *maintaining arc consistency*, MAC), typically reduces the search tree dramatically compared to plain backtracking, though it does not by itself guarantee a solution: it detects local inconsistency, not global unsatisfiability, except in special cases such as tree-structured CSPs.

Table 4.1: Illustrative effect of constraint propagation on backtracking search node counts, for CSPs of increasing size drawn from the same family of random instances.

Variables	Plain backtracking	+ Forward checking	+ AC-3 (MAC)
10	340	95	40
20	12,500	1,100	210
30	480,000	9,200	890

4.4 Structure and Decomposition

CSPs whose constraint graph forms a tree can be solved in time linear in the number of variables, by ordering variables so each has at most one parent, enforcing arc consistency from leaves to root, and then assigning values top-down with no backtracking needed. General graphs can sometimes be reduced to trees via *cutset conditioning* (instantiating a small set of variables to break cycles, then solving the remaining tree-structured problem for each possible assignment to the cutset) or *tree decomposition* into a tree of connected subproblems, each small enough to solve independently.

4.5 Local Search for CSPs

CSPs can also be solved by local search over complete assignments, using the *min-conflicts* heuristic: repeatedly select a conflicted variable and reassign it to the value minimizing

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

the number of violated constraints. Min-conflicts solves large, structured problems such as n-queens for n in the millions almost instantaneously, though it offers no completeness or optimality guarantee – it can, in principle, plateau or cycle without ever finding a solution even when one exists, which systematic backtracking search never does.

4.6 Worked Example: AC-3 Propagation

Consider three variables X, Y, Z with domains $D_X = \{1, 2, 3\}$, $D_Y = \{1, 2, 3\}$, $D_Z = \{1\}$, and constraints $X \neq Y$, $Y \neq Z$. Figure 4.3 shows how AC-3 propagates the tight constraint on Z back through Y and then X, pruning both domains before any search takes place.

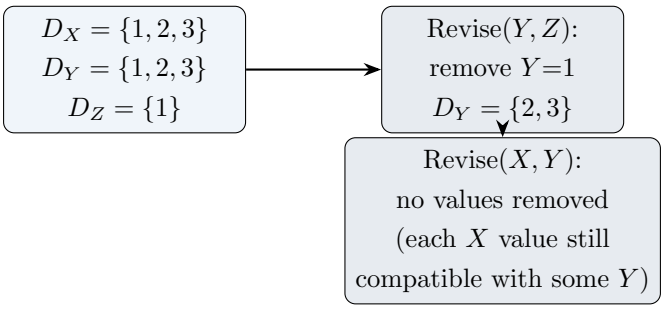


Figure 4.3: AC-3 trace for the chain X–Y–Z. Because $D_Z = \{1\}$, arc consistency forces $Y \neq 1$, shrinking D_Y to $\{2, 3\}$; this change is re-queued back onto the arc (X, Y), but since $D_X = \{1, 2, 3\}$ still has a compatible partner in $\{2, 3\}$ for every value, no further pruning occurs at X. A final assignment (e.g., X=1, Y=2, Z=1) can then be found with no backtracking.

Chapter Summary

- CSPs expose factored structure (variables, domains, constraints) that general search cannot exploit, enabling MRV, degree, and least-constraining-value heuristics.
- Constraint propagation (arc consistency, AC-3) prunes domains before or during search, often reducing the tree dramatically, though it detects only local, not global, inconsistency in general graphs.
- Tree-structured CSPs are solvable in linear time; general graphs can sometimes be decomposed toward tree structure; local search via min-conflicts scales to very large CSPs at the cost of completeness guarantees.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Exercises

4.1 Using Figure 4.1, formulate map coloring for Australia as a CSP with three colors, specifying variables, domains, and constraints explicitly. Is a 2-coloring possible? Justify from the graph structure.

4.2 Extend Figure 4.2 by showing what happens if the degree heuristic, rather than an arbitrary order, is used to choose the first variable.

4.3 Prove that enforcing arc consistency on a tree-structured CSP, followed by a single top-down pass, always finds a solution if one exists.

4.4 Explain why AC-3's worst-case time complexity is $O(cd^3)$, and identify which part of the algorithm dominates this bound.

4.5 Using the pattern in Table 4.1, explain qualitatively why the gap between plain backtracking and MAC widens as problem size grows.

4.6 **Implementation.** Implement backtracking search with MRV, degree heuristic, and forward checking for Sudoku. Compare node counts against plain backtracking on the same set of puzzles, and report at least one puzzle solved only by the improved version within a reasonable time limit.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 5

Adversarial Search and Games

5.1 Games as Search Problems

A deterministic, fully observable, two-player zero-sum game can be modeled as a search problem with an initial state, a set of legal actions (moves) per player, a transition model, a terminal test, and a utility function assigning a value to terminal states from one player's perspective. Unlike single-agent search, the second player is an adversary actively working against the first, which motivates *minimax* reasoning rather than simple cost minimization: the searching agent must plan for the opponent's best response at every level, not merely for the shortest or cheapest path to a terminal state.

2025/2026 2025/2026 2025/2026

5.2 Minimax

Key Idea

The minimax value of a node is the utility the maximizing player can guarantee assuming the minimizing player plays optimally:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal} \\ \max_a \text{Minimax}(\text{Result}(s,a)) & \text{if MAX to move} \\ \min_a \text{Minimax}(\text{Result}(s,a)) & \text{if MIN to move} \end{cases}$$

Minimax is computed by depth-first traversal of the full game tree, in $O(b^m)$ time and $O(bm)$ space for branching factor b and maximum depth m. This is intractable for games such as chess (branching factor around 35, game length around 80 ply), motivating both pruning and depth-limited evaluation.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

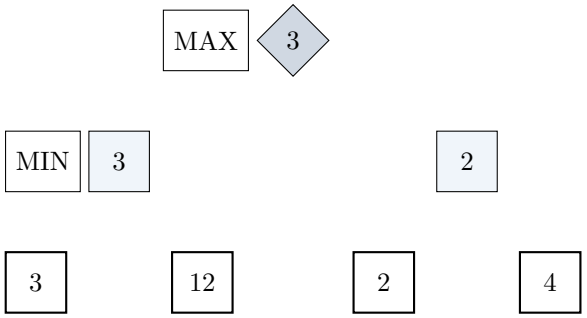


Figure 5.1: Minimax values backed up through a depth-2 game tree. At the MIN level, node A takes $\min(3, 12) = 3$ and node B takes $\min(2, 4) = 2$; at the MAX root, the value is $\max(3, 2) = 3$, so MAX should choose the branch leading to node A.

رومانی صبحی رزق جرجس مطر

5.3 Alpha-Beta Pruning

Alpha-beta pruning computes the same result as minimax while ignoring branches that cannot influence the final decision. It maintains α, the best value MAX can guarantee so far along the path, and β, the best value MIN can guarantee, pruning a branch whenever $\alpha \geq \beta$. With a good move-ordering heuristic, alpha-beta reduces the effective branching factor from b to roughly $\sqrt{b}$, doubling the depth searchable in the same time.

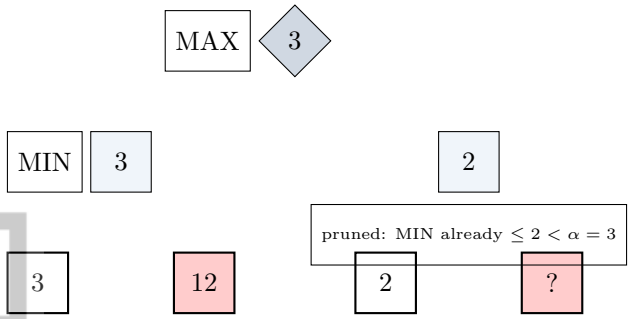


Figure 5.2: Alpha-beta pruning on the same tree as Figure 5.1. The dashed edge to 12 under node A is irrelevant once MIN's first child there is examined, since MIN would never choose 12 over a smaller value already available. Under node B, once its first child evaluates to 2, MAX already has $\alpha = 3$ from node A, so B cannot possibly exceed 3 and its remaining child is pruned entirely without being evaluated.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Algorithm 3 Alpha-Beta (MAX node)

```
function MaxValue(state, α, β)
    if Terminal(state) then return Utility(state)
    end if
    v ← −∞
    for each action a do
        v ← max(v, MinValue(Result(state, a), α, β))
        if v ≥ β then return v                                  ▷ prune
        end if
        α ← max(α, v)
    end for
    return v
end function
```

5.4 Depth-Limited Search and Evaluation Functions

Since most games are too deep to solve exactly, practical agents apply a cutoff test and estimate the value of non-terminal states with an *evaluation function* – typically a weighted sum of features (material, mobility, king safety in chess, and so on) tuned by hand or learned from data, as in modern systems that combine deep neural network evaluation with tree search. Iterative deepening is commonly combined with alpha-beta to use available time flexibly and to improve move ordering across iterations, since the best move found at a shallower depth is a strong candidate to try first at the next depth.

5.5 Expectimax and Stochastic Games

Games with an element of chance (dice, card draws) add *chance nodes* to the tree, whose value is the expectation over outcomes weighted by their probabilities:

$$\text{Expectimax}(s) = \sum_{r} P(r)\,\text{Expectimax}(\text{Result}(s, r))$$

Because expectation is sensitive to the exact utility values (not just their order), evaluation functions for expectimax must be a positive linear transformation of true utilities, unlike in pure minimax where any order-preserving transformation suffices – a subtlety that trips up many first implementations of expectimax evaluation functions.

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

5.6 Monte Carlo Tree Search

For games with very high branching factors, such as Go (branching factor over 200), MCTS replaces exhaustive expansion with repeated random simulations (*playouts*) from promising states, using the results to iteratively grow and bias the search tree via a selection policy such as Upper Confidence Bound applied to Trees (UCT):

$$\mathrm{UCT}(n) = \frac{w_n}{v_n} + C\sqrt{\frac{\ln v_p}{v_n}}$$

where w_n/v_n is node n's average simulated outcome, v_p is its parent's visit count, and the second term rewards under-explored branches. MCTS requires no hand-crafted evaluation function, only a way to simulate games to completion, and underlies systems that combine it with learned policy and value networks.

Table 5.1: Rough branching factors and typical game lengths for several classic games, illustrating why exhaustive minimax is feasible for some and requires MCTS-style approximation for others.

2025/2026 2025/2026 2025/2026

Game	Branching factor	Game length (ply)
Tic-Tac-Toe	≈ 4	≤ 9
Checkers	≈ 7	≈ 70
Chess	≈ 35	≈ 80
Go	≈ 250	≈ 150

5.7 Worked Example: Expectimax with a Chance Node

Figure 5.3 shows a small expectimax tree for a dice-based game, where MAX chooses between two actions, each leading to a chance node representing a coin flip (probability 0.5 each outcome).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

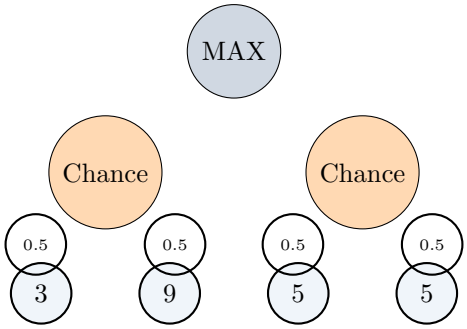


Figure 5.3: Expectimax tree: MAX chooses between two actions, each landing on a chance node with two equally likely outcomes. The left chance node has expected value $0.5(3) + 0.5(9) = 6$; the right has expected value $0.5(5) + 0.5(5) = 5$. MAX therefore prefers the left action despite its higher variance, since expectimax cares only about expected value.

This example also illustrates the sensitivity to utility scale mentioned in Section 5.5: if the leaf values were instead passed through a monotonic but concave transformation (e.g., $\sqrt{\cdot}$, reflecting risk aversion), the left branch's expected value would become $0.5\sqrt{3} + 0.5\sqrt{9} \approx 2.37$, while the right becomes $\sqrt{5} \approx 2.24$ – the ranking survives this particular transformation, but not every monotonic transformation preserves it, which is precisely why expectimax evaluation functions must be a *positive linear* transformation of true utilities, not merely order-preserving as in pure minimax.

Chapter Summary

- Minimax computes the value a rational MAX player can guarantee against an optimal adversary; alpha-beta pruning computes the identical result while ignoring provably irrelevant branches.
- Practical game agents combine depth-limited search with a learned or hand-crafted evaluation function, since exhaustive search is infeasible for most non-trivial games.
- Expectimax extends minimax to stochastic games via expectation over chance outcomes; Monte Carlo tree search replaces exhaustive expansion with random simulation for very high branching factors.

Exercises

5.1 Compute the minimax value of a small game tree of depth 3 and branching factor 2 that you construct, showing values at every node, following the style of Figure 5.1.

5.2 On the tree from Figure 5.2, verify by hand that reversing the child order under node B changes which branches get pruned, even though the final backed-up value

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

is unchanged.

5.3 Explain why alpha-beta pruning's benefit depends on move ordering, referencing Table 5.1 to argue why move ordering matters more for chess than for Tic-Tac-Toe.

5.4 Explain why expectimax requires evaluation functions that preserve the actual magnitude of utilities, using a small example where a monotonic but non-linear transformation changes the chosen move.

5.5 Using Table 5.1, explain why exhaustive minimax search to the end of the game is feasible for Tic-Tac-Toe but not for Go, and why MCTS is a natural fit for the latter.

5.6 **Implementation.** Implement minimax with alpha-beta pruning and a simple evaluation function for Tic-Tac-Toe or Connect Four. Measure nodes expanded with and without pruning at several search depths, and verify that both produce identical move choices.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 6

Logical Agents: Propositional Logic

6.1 Knowledge-Based Agents

A knowledge-based agent maintains a *knowledge base* (KB) of sentences in some formal language and decides on actions by querying the KB via logical inference, rather than through hard-coded condition–action rules. This separates *what* the agent knows from *how* it reasons, allowing new knowledge to be added declaratively without rewriting the agent's control logic each time.

6.2 Syntax and Semantics of Propositional Logic

Propositional logic sentences are built from proposition symbols and the connectives $\neg$, $\wedge$, $\vee$, $\Rightarrow$, and $\Leftrightarrow$. A *model* assigns a truth value to every proposition symbol; a sentence is *true* in a model if it evaluates to true under the standard truth tables for the connectives.

Table 6.1: Truth table for the material conditional $P \Rightarrow Q$, frequently a point of confusion since it is true whenever P is false, regardless of Q.

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Key Idea

A sentence α is *entailed* by a knowledge base KB, written $KB \models \alpha$, if α is true in every model in which KB is true. Entailment is the formal notion underlying sound logical inference.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

6.3 Inference

An inference algorithm is *sound* if it derives only entailed sentences, and *complete* if it can derive every entailed sentence. *Model checking* decides entailment by enumerating all models, which is correct but exponential in the number of symbols: for n proposition symbols there are 2^n models to check.

6.3.1 Resolution

Sentences converted to *conjunctive normal form* (CNF, a conjunction of clauses, each a disjunction of literals) support the *resolution* inference rule: from clauses containing complementary literals ℓ and $\neg\ell$, derive a new clause combining the remaining literals. Resolution is refutation-complete for propositional logic: $KB \models \alpha$ if and only if $KB \wedge \neg\alpha$ resolves to the empty clause.

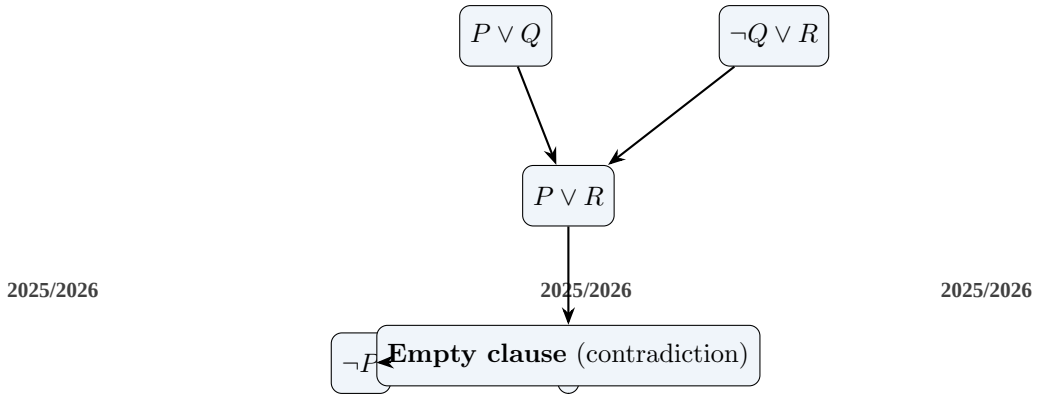


Figure 6.1: A resolution refutation: resolving $P \vee Q$ with $\neg Q \vee R$ (on complementary literal Q) yields $P \vee R$; resolving that with $\neg P$ (on complementary literal P) yields the empty clause, proving the original set of clauses – together with the negated query used to seed $\neg P$ – is unsatisfiable, and hence that R was entailed.

6.3.2 Forward and Backward Chaining

When the KB consists of *Horn clauses* (at most one positive literal per clause), inference is tractable. *Forward chaining* repeatedly applies rules whose premises are known until no new facts follow, running in linear time in the size of the KB. *Backward chaining* works from a query, recursively checking whether its premises can be established, and underlies most logic programming systems.

6.3.3 The DPLL Algorithm and Satisfiability

Determining propositional satisfiability (SAT) is NP-complete, but the Davis–Putnam–Logemann–Loveland (DPLL) algorithm, a backtracking search over CNF with unit propa-

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

gation and pure-symbol elimination, solves many practical instances efficiently, and forms the basis of modern industrial SAT solvers used for hardware verification, planning, and scheduling.

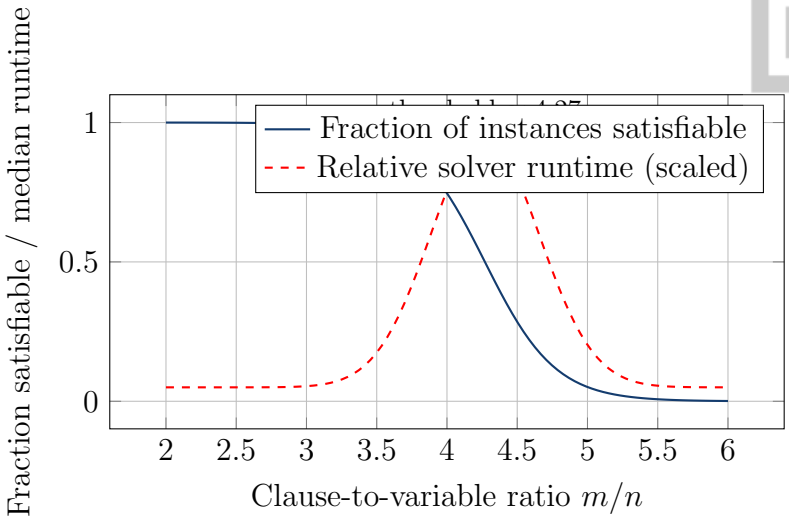


Figure 6.2: The satisfiability phase transition for random 3-SAT: as the clause-to-variable ratio crosses approximately 4.27, instances shift from almost always satisfiable to almost always unsatisfiable, and solver runtime peaks sharply near this threshold, where instances are hardest to decide either way.

Worked Example

The Wumpus World, a classic testbed, has an agent inferring safe squares in a grid from percepts (breeze, stench) using propositional sentences such as $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$, linking a breeze percept to the possible presence of pits in adjacent squares. Applying resolution or model checking to the accumulated percept sentences lets the agent prove particular squares pit-free before stepping onto them.

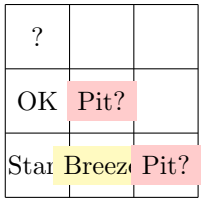


Figure 6.3: A breeze percept at the square right of Start implies (via $B \Leftrightarrow P_{\text{adjacent}}$) that at least one neighboring square contains a pit; resolution over the accumulated sentences from several squares can narrow down, or in some cases fully determine, which specific square is the culprit.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

6.4 Limitations of Propositional Logic

Propositional logic has no way to express generalizations over objects ("every square adjacent to a pit is breezy") without enumerating a separate sentence for every square. This motivates first-order logic, introduced in the next chapter, which adds objects, relations, and quantifiers.

6.5 Worked Example: Resolution Refutation

To show $KB \models \alpha$ by resolution, we show that $KB \wedge \neg\alpha$ is unsatisfiable by deriving the empty clause. Let $KB = (P \Rightarrow Q) \wedge (Q \Rightarrow R) \wedge P$, and suppose we want to show $KB \models R$. Converting to CNF gives three clauses: $(\neg P \vee Q)$, $(\neg Q \vee R)$, and (P). Negating the query adds $(\neg R)$. Figure 6.4 traces the resolution derivation of the empty clause from these four clauses.

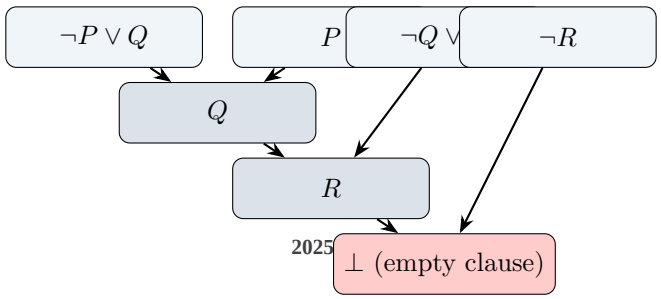


Figure 6.4: Resolution refutation deriving the empty clause from $KB \wedge \neg\alpha$, proving $KB \models R$. Each step resolves two clauses on a complementary pair of literals ($P/\neg P$, then $Q/\neg Q$, then $R/\neg R$), eliminating one variable per step.

Because the empty clause represents a contradiction, its derivation shows $KB \wedge \neg\alpha$ is unsatisfiable, and therefore $KB \models \alpha$ – exactly the refutation-completeness property stated in Section 6.4.1.

Chapter Summary

- Knowledge-based agents separate what they know (a knowledge base) from how they reason (an inference procedure), unlike hard-coded condition–action rules.
- Resolution is a sound and refutation-complete inference rule for propositional logic; forward and backward chaining give tractable inference for the Horn-clause special case.
- Propositional logic cannot generalize over objects without enumerating a separate sentence per object, motivating first-order logic in the next chapter.

2025/2026 2025 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Exercises

6.1 Convert $(P \Rightarrow Q) \wedge (Q \Rightarrow R)$ to CNF and show, via resolution (in the style of Figure 6.1), that it entails $P \Rightarrow R$.

6.2 Using Table 6.1, explain in your own words why $P \Rightarrow Q$ is considered true whenever P is false, and why this sometimes feels counterintuitive in natural language.

6.3 Explain the difference between soundness and completeness of an inference procedure, and give an example of a rule that is sound but not complete.

6.4 Show that forward chaining on a Horn-clause KB runs in time linear in the size of the KB, and explain what property of Horn clauses makes this possible.

6.5 Using propositional sentences, formalize the percept-to-inference rule linking a stench percept to the possible location of a wumpus in the Wumpus World, analogous to Figure 6.3.

6.6 Explain, referencing Figure 6.2, why random 3-SAT instances near the satisfiability threshold are particularly hard for DPLL-style solvers, while instances far from the threshold in either direction tend to be easy.

6.7 **Implementation.** Implement DPLL with unit propagation. Test it on randomly generated 3-SAT instances at several clause-to-variable ratios and reproduce a plot similar to Figure 6.2 using your own measured runtimes.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

رومانی صبحی رزق جرجس مطر

Chapter 7

First-Order Logic and Knowledge Representation

7.1 Why First-Order Logic

First-order logic (FOL) extends propositional logic with *objects*, *relations*, and *functions*, plus the quantifiers $\forall$ (universal) and $\exists$ (existential). This allows a single sentence to generalize over an entire domain: $\forall x\,\mathrm{Pit}(x) \Rightarrow \mathrm{Breezy}(\mathrm{Adjacent}(x))$ expresses a rule about every pit at once, something propositional logic could only state square by square, as illustrated at the end of Chapter 6.

2025/2026 2025/2026 2025/2026

7.2 Syntax and Semantics

A FOL model consists of a domain of objects and an interpretation mapping constant symbols to objects, predicate symbols to relations over the domain, and function symbols to functions on the domain. $\forall x\,P(x)$ is true in a model if P holds for every object in the domain; $\exists x\,P(x)$ is true if P holds for at least one object. Quantifiers interact with connectives in characteristic ways, for instance $\forall x\,P(x)$ is typically paired with $\Rightarrow$ ("for all x, if x is a dog then x barks") while $\exists x$ is typically paired with $\wedge$ ("there exists an x such that x is a dog and x is brown").

Table 7.1: Common natural-language quantifier patterns and their correct first-order translations; a frequent student error is pairing $\forall$ with $\wedge$ or $\exists$ with $\Rightarrow$, both of which produce sentences with unintended, usually vacuous or overly weak, truth conditions.

Pattern	Correct translation
"Every dog barks"	$\forall x\,\mathrm{Dog}(x) \Rightarrow \mathrm{Barks}(x)$
"Some dog is brown"	$\exists x\,\mathrm{Dog}(x) \wedge \mathrm{Brown}(x)$
"No dog flies"	$\forall x\,\mathrm{Dog}(x) \Rightarrow \neg\mathrm{Flies}(x)$

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

7.3 Knowledge Engineering

Building a FOL knowledge base for a domain involves identifying the task, assembling relevant background knowledge, choosing a vocabulary of predicates, functions, and constants, encoding general rules, encoding the specific problem instance, and posing queries to the inference procedure – an iterative process, since early representational choices are frequently revised once axioms are tested against queries.

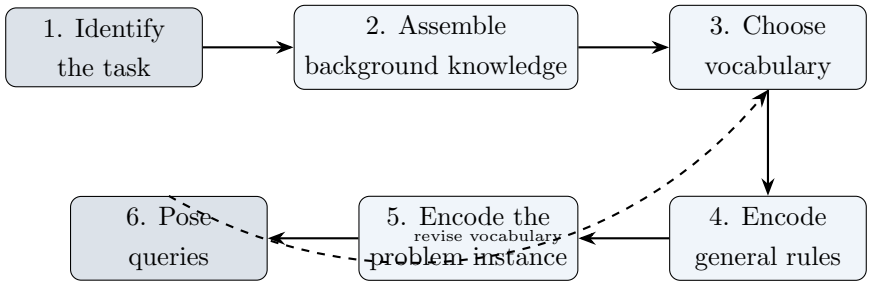


Figure 7.1: The knowledge engineering process. The dashed feedback arrow reflects the iterative nature of the process: testing queries against early vocabulary choices routinely reveals gaps or ambiguities that send the engineer back to step 3.

7.4 Inference in First-Order Logic

2025/2026 2025/2026 2025/2026

7.4.1 Unification and Generalized Modus Ponens

Unification finds a substitution that makes two sentences identical, e.g., unifying $\text{Loves}(x, \text{Mary})$ with $\text{Loves}(\text{John}, y)$ gives $\{x/\text{John}, y/\text{Mary}\}$. Unification underlies *generalized modus ponens*, which lifts propositional forward and backward chaining to first-order Horn clauses without first grounding every variable, avoiding a combinatorial blow-up that would occur if every variable had to be instantiated with every possible object before inference could begin.

7.4.2 Resolution in First-Order Logic

First-order resolution generalizes propositional resolution using unification to match complementary literals, and is refutation-complete: if a set of first-order sentences is unsatisfiable, resolution will eventually derive the empty clause, though because FOL is only semidecidable, the procedure may run forever on satisfiable inputs – a fundamental limitation, not an implementation shortcoming, following from Church and Turing's undecidability results for first-order logic.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

7.5 Ontological Commitments and Representing Change

Representing structured domains often requires *ontologies* describing categories of objects, part-whole relationships, and events.

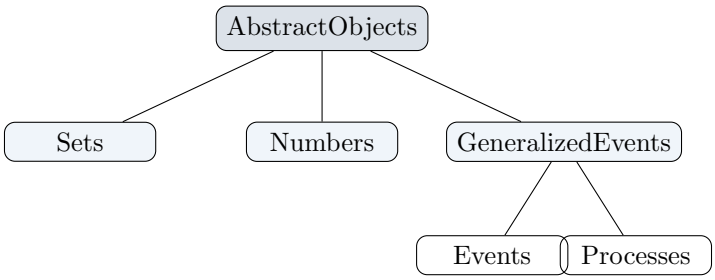


Figure 7.2: A fragment of a general upper ontology, of the kind used to organize knowledge across many task-specific domains before domain-specific categories (e.g., "Purchase" as a kind of Event) are added beneath it.

Representing change over time (needed for planning, covered next) traditionally uses *situation calculus*, where each action produces a new situation, or the more compact language of *planning domain definitions* used by modern planners, both of which must address the *frame problem*: specifying compactly what does *not* change when an action is taken, rather than listing every unaffected fact explicitly, which would make even simple domains impractical to formalize.

Worked Example

Family relationships are a standard FOL exercise: $\forall x, y\ \text{Parent}(x, y) \wedge \text{Parent}(x, z) \wedge y \neq z \Rightarrow \text{Sibling}(y, z)$ defines siblinghood in terms of a shared parent, illustrating how a handful of FOL axioms can generate an entire relational structure that would require enumerating every pair of siblings in propositional logic.

7.6 Worked Example: A Small Family-Relations Knowledge Base

Figure 7.3 shows a small family tree, which we encode in FOL using a single $\textsc{Parent}(x, y)$ predicate ("x is a parent of y") plus the definitional axiom for siblinghood from Section 7.5. Table 7.2 lists the base facts; combined with the sibling axiom, forward chaining derives $\textsc{Sibling}(\text{Ana}, \text{Ben})$ automatically, without that fact ever being stated directly.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

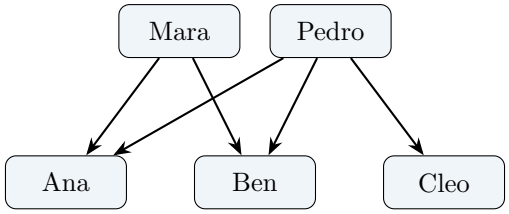


Figure 7.3: A small family tree. Arrows point from parent to child. Ana and Ben share both parents; Cleo shares only Pedro.

Table 7.2: Base facts for the family tree in Figure 7.3.

Parent(Mara, Ana), Parent(Mara, Ben)
Parent(Pedro, Ana), Parent(Pedro, Ben), Parent(Pedro, Cleo)

Applying the axiom $\forall x, y, z \;\; \text{Parent}(x, y) \wedge \text{Parent}(x, z) \wedge y \neq z \Rightarrow \text{Sibling}(y, z)$ with substitution $\{x/\text{Mara}, y/\text{Ana}, z/\text{Ben}\}$ (and separately with x/Pedro) derives Sibling(Ana, Ben) twice, via two independent shared parents, while no substitution derives Sibling(Ana, Cleo), since Ana and Cleo share no common parent in Table 7.2 – illustrating how a compact set of FOL rules generates exactly the relational structure implied by the facts, without needing a sibling fact to be stated for every pair.

Chapter Summary

- First-order logic adds objects, relations, functions, and quantifiers, letting a single sentence generalize over an entire domain.
- Unification and generalized modus ponens lift Horn-clause inference to the first-order setting without grounding every variable explicitly.
- Knowledge engineering is iterative: representational choices are typically revised once axioms are tested against real queries, and representing change over time raises the frame problem.

Exercises

7.1 Translate into FOL: "every student who takes a course passes it if they attend every lecture." Identify your predicates explicitly, and check your answer against the patterns in Table 7.1.

7.2 Find the most general unifier of Knows(John, x) and Knows(y, Mother(y)), or explain why none exists.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

7.3 Explain the frame problem and describe, at a conceptual level, one representational technique used to address it.

7.4 Using Figure 7.1 as a guide, walk through the six-step knowledge-engineering process for a small domain of your choosing (e.g., a library catalog or a recipe database), producing at least five FOL sentences.

7.5 Compare the expressive power of propositional and first-order logic using the pit/breeze example from Chapter 6: write the FOL generalization and count how many propositional sentences it would take to say the same thing for a 4×4 grid.

7.6 **Implementation.** Implement unification and forward chaining for a small first-order Horn-clause knowledge base (e.g., a family-relations or course-prerequisite domain) and demonstrate at least five non-trivial derived queries.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 8

Uncertainty and Bayesian Networks

8.1 Why Probability

Logical agents struggle in domains with uncertainty from partial observability, noisy sensors, or genuinely stochastic processes, where an agent rarely has enough information to derive a fact with certainty, only to assign it a *degree of belief*. Probability theory provides a principled way to combine such degrees of belief and update them as evidence arrives, in contrast to ad hoc numeric schemes (such as early "certainty factors") that lack a coherent semantics and can produce inconsistent conclusions.

8.2 Basic Probability Axioms

Key Idea

Probabilities obey Kolmogorov's axioms: $0 \le P(a) \le 1$; $P(\text{true}) = 1$, $P(\text{false}) = 0$; and $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$. *Conditional probability* is defined as $P(a \mid b) = P(a \wedge b)/P(b)$ when $P(b) > 0$.

8.3 Bayes' Rule

Bayes' rule follows directly from the definition of conditional probability:

$$P(h \mid e) = \frac{P(e \mid h)\, P(h)}{P(e)}$$

This lets an agent update a *prior* belief $P(h)$ into a *posterior* $P(h \mid e)$ using a *likelihood* term $P(e \mid h)$ that is often easier to specify directly (e.g., from a causal or generative model) than the posterior itself.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

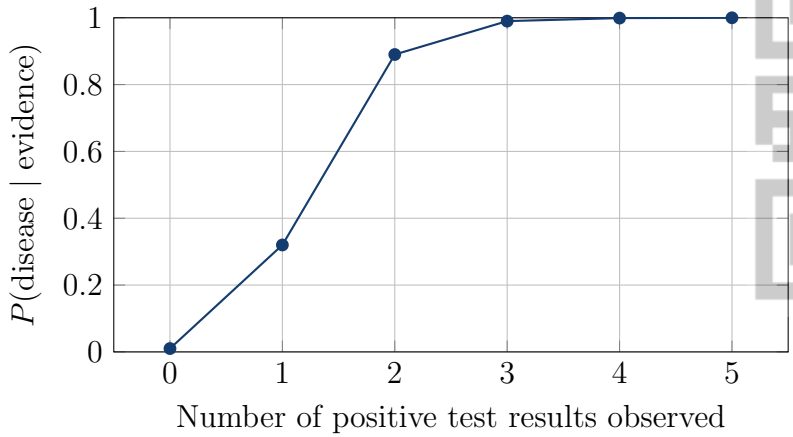


Figure 8.1: Posterior probability of a rare disease (prior 1%) as repeated independent test results accumulate, each test being 95% accurate. A single positive result raises belief substantially but leaves real doubt; a second independent positive result nearly eliminates that doubt, illustrating how Bayesian updating combines evidence multiplicatively rather than additively.

8.4 Independence and Conditional Independence

Two variables are *independent* if $P(a, b) = P(a)P(b)$; they are *conditionally independent* given a third variable c if $P(a, b \mid c) = P(a \mid c)P(b \mid c)$. Conditional independence assumptions are what make probabilistic models tractable, since a full joint distribution over n Boolean variables requires $2^n - 1$ numbers, an infeasible amount for even modest n (for $n = 30$, over a billion numbers).

8.5 Bayesian Networks

A Bayesian network is a directed acyclic graph in which each node represents a random variable and each edge represents a direct probabilistic dependency, annotated with a conditional probability table (CPT) giving $P(\text{node} \mid \text{parents})$. The network encodes the assertion that each variable is conditionally independent of its non-descendants given its parents, allowing the full joint distribution to be recovered as

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid \text{Parents}(X_i))$$

which typically requires far fewer parameters than the full joint.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

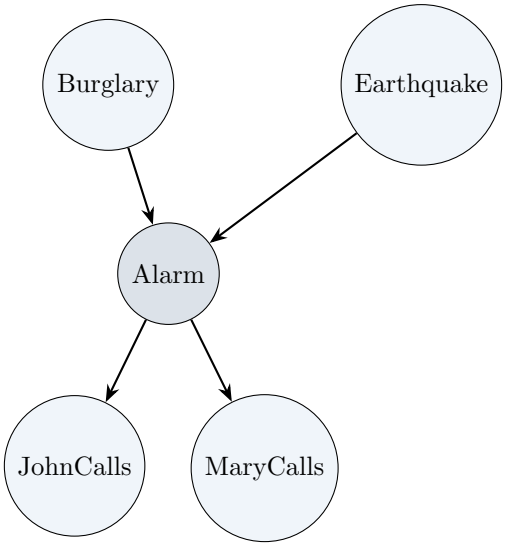


Figure 8.2: The classic Burglary–Earthquake–Alarm Bayesian network. Burglary and Earthquake are root causes assumed marginally independent of each other; both influence Alarm; and Alarm's state influences whether John or Mary calls, with John's and Mary's calls conditionally independent of each other given Alarm.

Table 8.1: Illustrative conditional probability table for the Alarm node in Figure 8.2.

Burglary	Earthquake	$P(\text{Alarm} = \text{true})$
T	T	0.95
T	F	0.94
F	T	0.29
F	F	0.001

Worked Example

Given evidence that both John and Mary called, Bayes' rule combined with the network structure in Figure 8.2 and the CPT in Table 8.1 lets us compute the posterior probability of a burglary, correctly weighting the (much less likely) explanation that an earthquake triggered the alarm instead, without ever needing to specify $P(\text{Burglary} \mid \text{JohnCalls}, \text{MaryCalls})$ directly.

8.6 Exact Inference

Enumeration answers any query by summing the joint distribution over hidden variables, but scales exponentially with the number of variables. *Variable elimination* improves on this by evaluating sums right to left, caching intermediate *factors* and eliminating variables one at a time, avoiding the repeated computation that plain enumeration incurs;

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

its complexity depends on the network's structure, remaining exponential in the worst case (specifically in the network's *treewidth*) but often far better in practice.

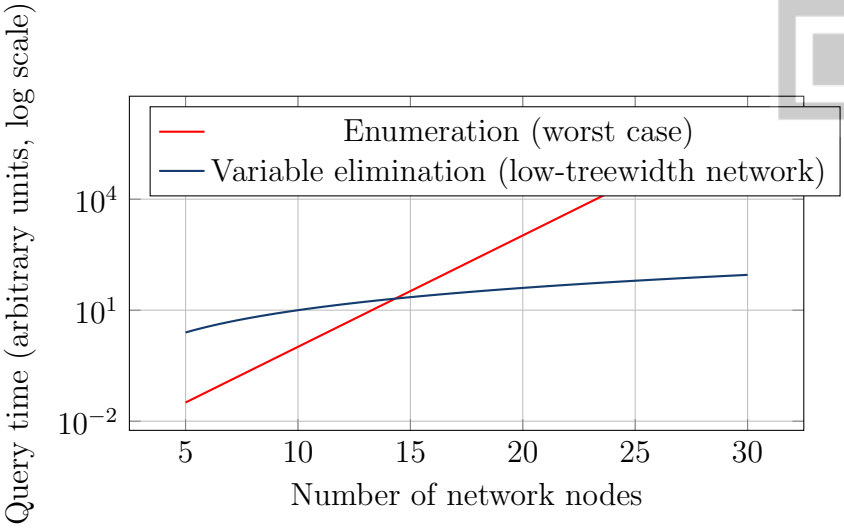


Figure 8.3: Query time growth for enumeration versus variable elimination on a network whose treewidth stays small as more nodes are added (a common case for networks with sparse, chain- or tree-like structure). Enumeration's exponential growth in the number of variables makes it impractical well before variable elimination's more moderate growth becomes a concern.

2025/2026 2025/2026 2025/2026

8.7 Constructing Bayesian Networks

A good ordering for adding nodes to a network follows causal structure: adding root causes before their effects tends to produce networks with fewer, more natural parameters than orderings that mix cause and effect arbitrarily, which can force unnaturally large CPTs to compensate for the mismatch between the chosen node order and the domain's actual causal structure.

8.8 Worked Example: Variable Elimination by Hand

Consider a three-node chain $A \to B \to C$ with $P(A)$, $P(B \mid A)$, $P(C \mid B)$ given, and suppose we want $P(C)$. Enumeration would sum the full joint $\sum_a \sum_b P(a)P(b \mid a)P(C \mid b)$ over both hidden variables at once; variable elimination instead processes one variable at a time, building and discarding intermediate factors.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Table 8.2: Variable elimination trace for $P(C)$ on the chain $A \to B \to C$. Eliminating A first produces an intermediate factor over B alone, which is then combined with $P(C \mid B)$ and summed over B.

Step	Operation	Resulting factor
1	Join $P(A)$, $P(B \mid A)$	$f_1(A, B) = P(A)P(B \mid A)$
2	Eliminate A: sum f_1 over A	$f_2(B) = \sum_a P(a)P(B \mid a) = P(B)$
3	Join $f_2(B)$, $P(C \mid B)$	$f_3(B, C) = P(B)P(C \mid B)$
4	Eliminate B: sum f_3 over B	$f_4(C) = \sum_b P(b)P(C \mid b) = P(C)$

The key saving relative to plain enumeration is that $f_2(B) = P(B)$ is computed *once* and reused, rather than being recomputed inside a nested sum for every value of C; on a chain of length n this reduces the work from exponential in n (enumeration) to linear in n (variable elimination), since each elimination step processes a factor over at most two variables regardless of chain length.

Chapter Summary

- Bayes' rule combines a prior and a likelihood into a posterior belief; conditional independence is what makes probabilistic reasoning over many variables tractable.
- Bayesian networks factor a joint distribution using a graph of conditional dependencies, requiring far fewer parameters than the full joint table.
- Exact inference by enumeration is exponential in general; variable elimination reuses intermediate factors to reduce this cost, though it remains exponential in the network's treewidth in the worst case.

Exercises

8.1 Using Figure 8.2 and Table 8.1, write out the full joint distribution as a product of conditional probabilities per the chain rule for Bayesian networks.

8.2 Show, using Bayes' rule, how a rare disease with a highly accurate test can still produce mostly false positives among those who test positive. Use concrete numbers and compare against Figure 8.1.

8.3 Explain what it means for a node to be conditionally independent of its non-descendants given its parents, and why this justifies the factored joint-distribution formula.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

8.4 Trace variable elimination on the network in Figure 8.2 to answer $P(\text{Burglary} \mid \text{JohnCalls} = \text{true})$, showing each factor created and eliminated.

8.5 Referencing Figure 8.3, explain why treewidth, not simply the number of nodes, is the right way to characterize when variable elimination remains tractable.

8.6 **Implementation.** Implement exact inference by enumeration for a Bayesian network with five or more nodes, and verify your results against a hand calculation for at least two distinct queries.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 9

Approximate Inference and Temporal Probabilistic Models

9.1 Sampling Methods for Bayesian Networks

When exact inference is too costly, *approximate inference* via sampling provides estimates that converge to the true posterior as the number of samples grows. *Direct sampling* generates complete variable assignments by sampling each variable in topological order from its conditional distribution given its already-sampled parents. *Rejection sampling* answers conditional queries by discarding samples inconsistent with the evidence, which becomes wasteful as evidence becomes less likely. *Likelihood weighting* fixes evidence variables and weights each sample by the likelihood the evidence would have received, avoiding rejection but degrading when evidence variables occur late in the sampling order. *Markov chain Monte Carlo* (MCMC) methods, such as Gibbs sampling, instead generate a sequence of samples by resampling one variable at a time conditioned on the current values of all others, forming a Markov chain whose stationary distribution is the true posterior.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

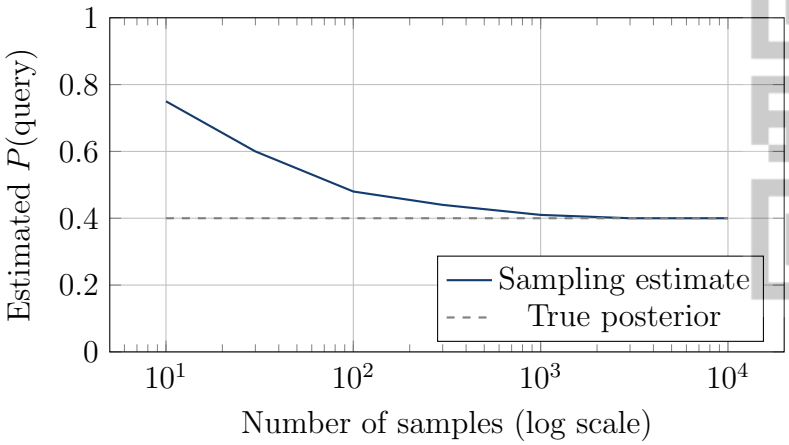


Figure 9.1: Convergence of a sampling-based estimate toward the true posterior probability as sample count increases. Early estimates fluctuate substantially; the estimate stabilizes once enough samples have been drawn, illustrating the general law-of-large-numbers behavior underlying all sampling-based approximate inference.

9.2 Temporal Models

Many domains require reasoning about a sequence of states evolving over time, observed through a sequence of noisy or partial evidence. A temporal probabilistic model assumes the *Markov property*: the current state depends only on a bounded history of previous states (typically just the immediately preceding one, the *first-order Markov assumption*), and evidence at each time step depends only on the current state (the *sensor model*).

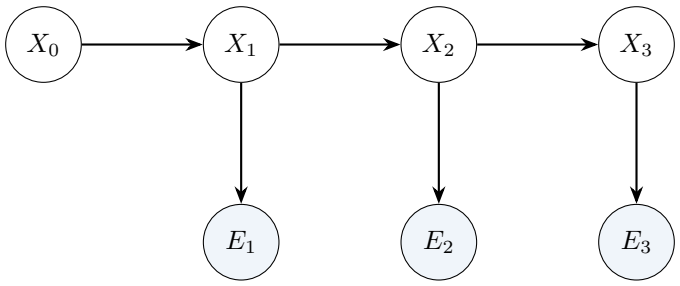


Figure 9.2: A temporal model unrolled over time: hidden states X_0, X_1, X_2, X_3 evolve via the transition model, and each hidden state emits a noisy observation E_t via the sensor model. This structure underlies hidden Markov models, Kalman filters, and dynamic Bayesian networks alike.

Key Idea

A temporal model is fully specified by a transition model $P(X_t \mid X_{t-1})$ and a sensor model $P(E_t \mid X_t)$, together with a prior $P(X_0)$. Filtering, prediction, smoothing, and most-likely-sequence tasks all reduce to inference over this factored structure.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

9.3 Hidden Markov Models

A hidden Markov model (HMM) is a temporal model with a single discrete state variable. *Filtering* computes $P(X_t \mid e_{1:t})$, the belief about the current state given all evidence so far, via the recursive update

$$P(X_{t+1} \mid e_{1:t+1}) \propto P(e_{t+1} \mid X_{t+1}) \sum_{x_t} P(X_{t+1} \mid x_t)\, P(x_t \mid e_{1:t})$$

which combines a one-step prediction with the new evidence at each step. *Smoothing* additionally uses future evidence to improve estimates of past states, via the forward–backward algorithm. *Most likely explanation* uses the Viterbi algorithm, structurally similar to filtering but taking a maximum rather than a sum at each step, to recover the single most probable state sequence.

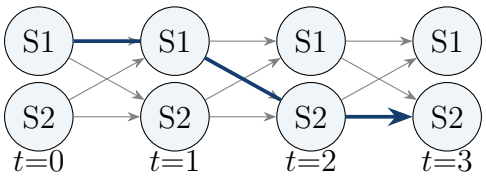


Figure 9.3: A trellis diagram for a two-state HMM. Gray arrows show all possible transitions between consecutive time steps; the highlighted blue path is the single most likely state sequence recovered by the Viterbi algorithm, which finds this path by keeping only the best-scoring incoming edge into each node rather than summing over all of them as filtering does.

2025/2026 2025/2026 2025/2026

9.4 Particle Filtering

When the state space is too large or continuous for exact HMM updates, *particle filtering* approximates the belief state with a population of weighted samples (particles). Each time step propagates particles forward through the transition model, reweights them by their likelihood under new evidence, and resamples proportionally to weight, concentrating particles in high-probability regions of the state space – a technique widely used in robot localization and tracking.

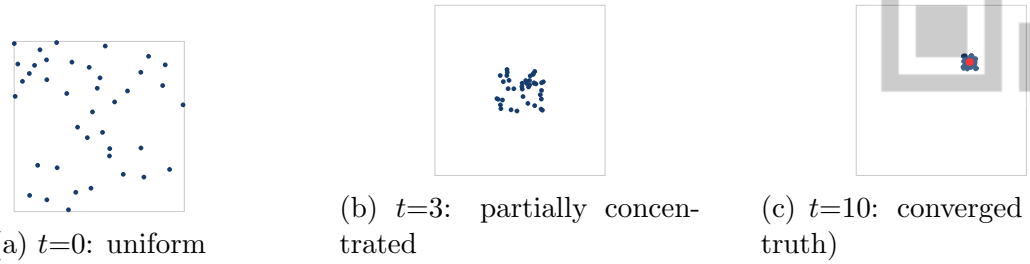
Worked Example

Robot localization: the hidden state is the robot's true position, the transition model reflects its motion commands plus noise, and the sensor model relates range-finder readings to expected readings at each candidate position. Particle filtering maintains a cloud of position hypotheses that concentrates around the true position as more sensor readings arrive, even when the initial position is completely

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

unknown, as illustrated conceptually in Figure 9.4.



(a) t=0: uniform (b) t=3: partially concentrated (c) t=10: converged (red = truth)

Figure 9.4: Particle filter evolution during robot localization. Particles start uniformly distributed over the map, reflecting complete initial uncertainty, and concentrate around the true position as sensor evidence accumulates over successive time steps.

9.5 Worked Example: Comparing Filtering Methods

Table 9.1 summarizes the trade-offs among the exact and approximate inference techniques introduced across Chapters 8–9, useful for deciding which to reach for on the term project (Appendix B) or Programming Assignment P3.

2025/2026 2025/2026 2025/2026

Table 9.1: Comparing inference methods introduced in Chapters 8–9.

Method	Exact?	Scales to	Key limitation
Enumeration	Yes	Small networks only	Exponential in variable count
Variable elimination	Yes	Moderate treewidth	Exponential in treewidth
Rejection sampling	Approximate	Any network	Wasteful for rare evidence
Likelihood weighting	Approximate	Any network	High variance for evidence deep in sampling order
Particle filtering	Approximate	Large/continuous state spaces	Particle deprivation without resampling

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

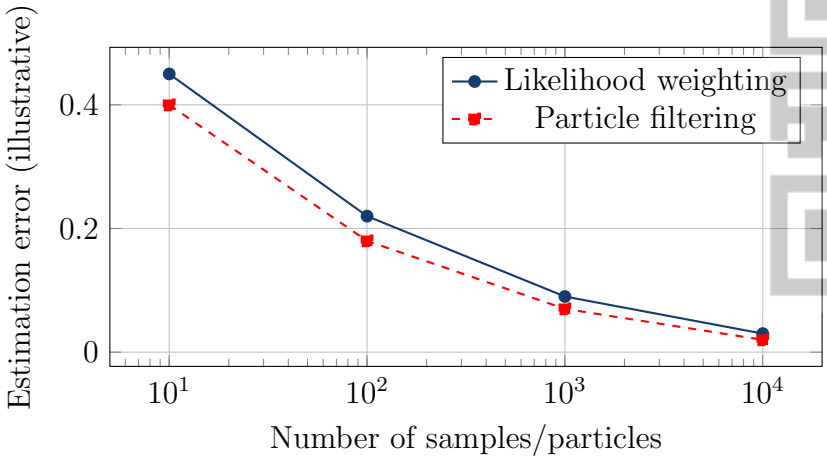


Figure 9.5: Illustrative convergence of estimation error as the number of samples or particles grows. Both approximate methods converge toward the true posterior as sample count increases, consistent with the law of large numbers, but neither provides the exact-answer guarantee of enumeration or variable elimination at any finite sample size.

Chapter Summary

- When exact inference is too costly, sampling methods (direct sampling, rejection sampling, likelihood weighting, MCMC) provide estimates that converge to the true posterior.
- Temporal models assume the Markov property and a fixed sensor model, reducing an unbounded history to a compact recursive filtering update.
- Hidden Markov models support filtering, smoothing, and most-likely-explanation via closely related recursive algorithms; particle filtering extends these ideas to large or continuous state spaces.

Exercises

9.1 Explain why rejection sampling becomes inefficient when the evidence is a rare event, and describe how likelihood weighting avoids this specific problem.

9.2 Derive the one-step filtering update for an HMM with two states and given transition and sensor models, computing $P(X_2 \mid e_1, e_2)$ from an explicit prior.

9.3 Using Figure 9.3, explain precisely how the Viterbi algorithm's computation differs from the forward algorithm's at each trellis node.

9.4 Compare filtering, smoothing, and most-likely-explanation in terms of what each computes and which algorithm (forward, forward– backward, or Viterbi) is used for each.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

9.5 Explain the role of resampling in particle filtering, and what can go wrong (particle deprivation) if it is omitted or done poorly, relating your answer to Figure 9.4.

9.6 **Implementation.** Implement the forward algorithm for filtering in a discrete HMM, and a particle filter for the same model. Compare their estimates of $P(X_t \mid e_{1:t})$ across a simulated sequence as the number of particles increases, reproducing a convergence plot similar to Figure 9.1.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

رومانی صبحی رزق جرجس مطر

Chapter 10

Decision Theory and Markov Decision Processes

10.1 Utility Theory

Rational decision-making under uncertainty requires combining beliefs (probabilities) with preferences (utilities). The *maximum expected utility* principle states that a rational agent should choose the action maximizing $\sum_s P(s \mid a)\, U(s)$. This principle can be derived from a small set of axioms of preference (orderability, transitivity, continuity, and others); an agent violating them can be shown to be exploitable through a sequence of trades that leaves it strictly worse off than when it started, a "money pump" argument that provides the normative justification for expected-utility maximization rather than any alternative decision rule.

10.2 Sequential Decision Problems

A *Markov decision process* (MDP) formalizes sequential decision-making in a fully observable, stochastic environment. It consists of a set of states S, actions $A(s)$, a transition model $P(s' \mid s, a)$, and a reward function $R(s)$ (or $R(s, a, s')$). A *policy* π maps states to actions; the goal is to find an *optimal policy* π^* maximizing expected cumulative reward, typically discounted by a factor $\gamma \in [0, 1)$ to ensure convergence over infinite horizons and to reflect a preference for sooner rewards.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

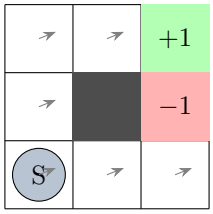


Figure 10.1: The classic 4×3 grid-world MDP. The agent starts at S, each non-terminal move costs a small living reward (often negative), the top-right cell gives a terminal reward of $+1$, and the cell below it gives a terminal reward of -1; the black square is an impassable obstacle. The optimal policy near the -1 cell is sensitive to the size of the living-cost penalty, as explored in the exercises.

Key Idea

The value of a state under policy π satisfies the Bellman equation

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, U^{\pi}(s')$$

and the optimal value function satisfies the Bellman optimality equation, replacing $\pi(s)$ with a max over actions.

10.3 Value Iteration

2025/2026 2025/2026 2025/2026

Value iteration computes the optimal value function by iterating the Bellman update

$$U_{i+1}(s) \leftarrow \max_{a} \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma\, U_i(s')\big]$$

until convergence, which is guaranteed because the Bellman update is a contraction under the discounted setting. The optimal policy is then extracted by acting greedily with respect to the converged value function.

Algorithm 4 Value Iteration

```
Initialize U(s) ← 0 for all s
repeat
    U' ← copy of U
    for each state s do
        U(s) ← max_a Σ_{s'} P(s' | s, a)[R(s, a, s') + γU'(s')]
    end for
until max_s |U(s) − U'(s)| < ε
return U
```

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

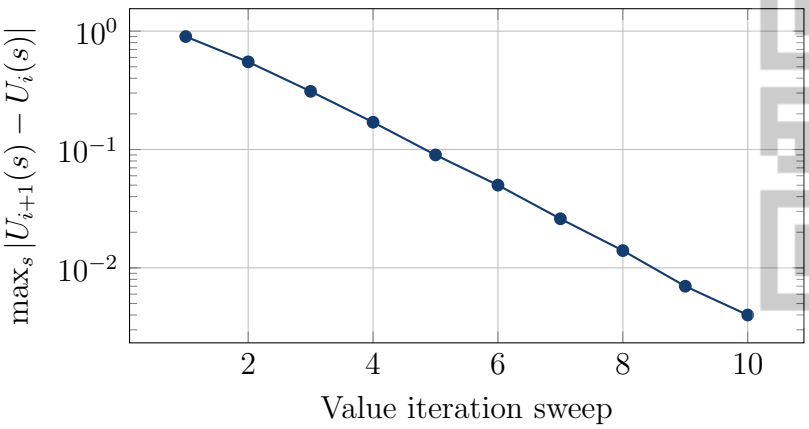


Figure 10.2: Typical convergence of value iteration on the gridworld of Figure 10.1: the maximum change in value across all states shrinks geometrically with each sweep, a direct consequence of the Bellman update being a contraction mapping with contraction rate governed by γ.

رومانی صبحی رزق جرجس مطر

10.4 Policy Iteration

2025/2026 2025/2026 2025/2026

Policy iteration alternates between *policy evaluation* (solving the linear system of Bellman equations for the current fixed policy) and *policy improvement* (updating the policy to act greedily with respect to the resulting value function), terminating when the policy no longer changes. Because policy iteration works with a finite space of deterministic policies and improves monotonically, it converges in a finite number of iterations, often fewer than value iteration requires, though each iteration can be more expensive since policy evaluation involves solving a linear system rather than a single Bellman sweep.

10.5 Partial Observability

When the state is not fully observable, the problem becomes a *partially observable MDP* (POMDP). Because the agent cannot observe the true state directly, it must act based on a *belief state* – a probability distribution over possible states, updated using the same filtering ideas introduced in Chapter 9. Solving POMDPs exactly is significantly harder than solving MDPs, since the belief state is continuous even when the underlying state space is discrete.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

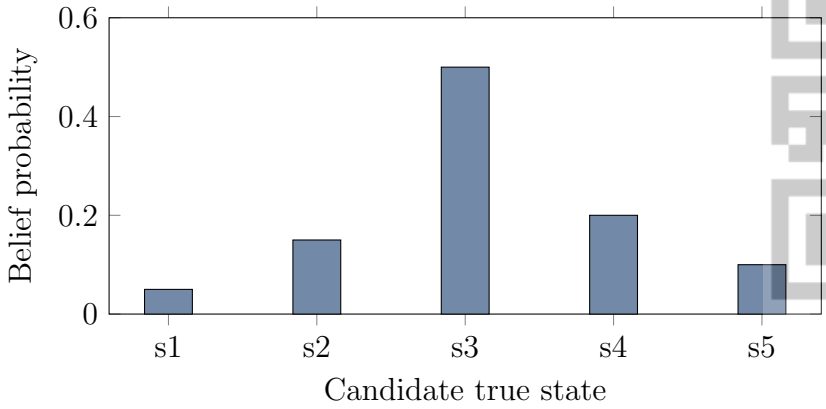


Figure 10.3: A belief state: a probability distribution over which underlying MDP state the agent actually occupies, given everything it has observed so far. A POMDP agent chooses actions based on this whole distribution, not a single most-likely state, since acting only on the mode can discard useful information about the remaining uncertainty.

Table 10.1: How the optimal policy near the penalty cell in Figure 10.1 changes as the living-cost reward $R(s)$ varies, illustrating the qualitative shift from risk-averse to risk-tolerant behavior.

Living cost	**Qualitative optimal behavior**
$R(s) = -2$ (very costly)	Agent takes the shortest path even if it means risking the -1 cell, since delay is worse than an occasional penalty.
$R(s) = -0.04$ (mild)	Agent takes the safe, slightly longer route, avoiding the cell adjacent to the -1 penalty.
$R(s) = +0.01$ (rewarding)	Agent may prefer to wander indefinitely rather than end the episode at all, since living is better than finishing.

2025/2026 2025/2026 2025/2026

10.6 Worked Example: One Bellman Backup by Hand

Consider a state s with two actions. Action a_1 leads deterministically to a neighbor with current value estimate $U(s_1) = 5$, and $R(s, a_1, s_1) = -1$ (a small living cost). Action a_2 leads with probability 0.8 to a neighbor valued at $U(s_2) = 10$ and reward -1, but with probability 0.2 the agent slips into a neighbor valued at $U(s_3) = -5$ with the same reward. Using $\gamma = 0.9$:

$$Q(s, a_1) = -1 + 0.9(5) = 3.5$$
$$Q(s, a_2) = -1 + 0.9\big[0.8(10) + 0.2(-5)\big] = -1 + 0.9(7) = 5.3$$

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

The Bellman update sets $U(s) \leftarrow \max(Q(s, a_1), Q(s, a_2)) = 5.3$, selecting a_2 despite its 20% chance of a bad outcome, because its expected value under the transition model still exceeds the safer, deterministic a_1. This single-state calculation is exactly the inner loop of Algorithm 10.1, applied simultaneously to every state in each full sweep.

Chapter Summary

- Maximum expected utility, derivable from a small set of preference axioms, is the normative basis for rational decision-making under uncertainty.
- Markov decision processes formalize sequential decisions via states, actions, transitions, and rewards, discounted over time by γ.
- Value iteration and policy iteration both compute optimal policies from the Bellman equations, trading per-iteration cost against number of iterations; partial observability (POMDPs) requires reasoning over belief states instead of raw states.

Exercises

10.1 State the axioms of utility theory and explain, for one axiom of your choosing, what kind of exploitable behavior results from violating it.

2025/2026 2025/2026 2025/2026

10.2 Derive the Bellman equation for $U^{\pi}(s)$ from the definition of expected discounted cumulative reward.

10.3 Explain why value iteration is guaranteed to converge, and how the discount factor γ affects the rate of convergence shown in Figure 10.2.

10.4 Compare value iteration and policy iteration in terms of per-iteration cost and number of iterations to convergence. When would you prefer one over the other?

10.5 Using Table 10.1 as a starting point, explain qualitatively why an MDP's optimal policy can change discontinuously as a single reward parameter is varied continuously.

10.6 **Implementation.** Implement value iteration and policy iteration for the grid-world MDP in Figure 10.1. Verify both converge to the same optimal policy, and reproduce Table 10.1 empirically by sweeping the living-cost parameter.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 11

Machine Learning Foundations

11.1 The Learning Problem

Machine learning replaces hand-coded rules with models fit to data. In *supervised learning*, we are given a training set of input–output pairs $\{(x_i, y_i)\}$ and seek a hypothesis h approximating the true underlying function; *classification* predicts discrete labels and *regression* predicts continuous values. *Unsupervised learning* seeks structure (such as clusters) in unlabeled data.

2025/2026 2025/2026 2025/2026

11.2 Generalization, Overfitting, and Evaluation

Key Idea

The goal of learning is *generalization* – accurate predictions on unseen data – not merely fitting the training set. A model that fits training data too closely, capturing noise rather than signal, is *overfit* and will generalize poorly.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

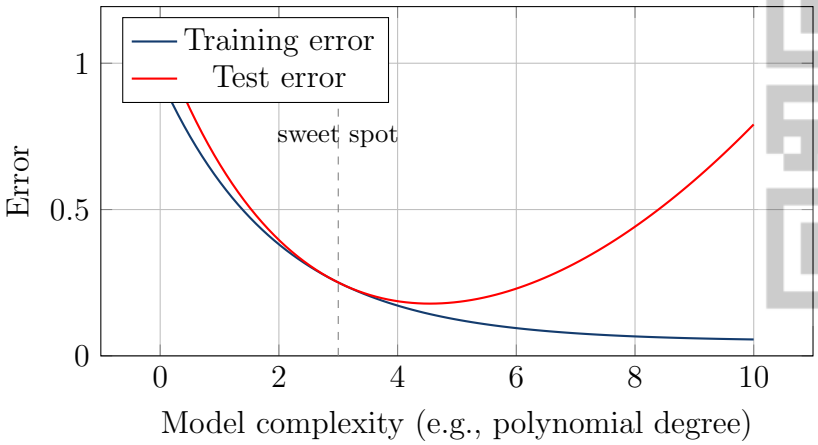


Figure 11.1: The classic bias–variance / generalization curve. Training error decreases monotonically with model complexity, but test error follows a U-shape: too little complexity underfits (high bias), too much overfits (high variance), and the best generalization occurs at an intermediate complexity, here illustrated near degree 3.

Model performance is estimated by holding out data the model never sees during training: a simple *train/test split*, or *k-fold cross-validation*, which partitions data into k folds, trains on $k-1$ and tests on the remainder k times, averaging the result for a more reliable estimate with less sensitivity to how the split happened to fall.

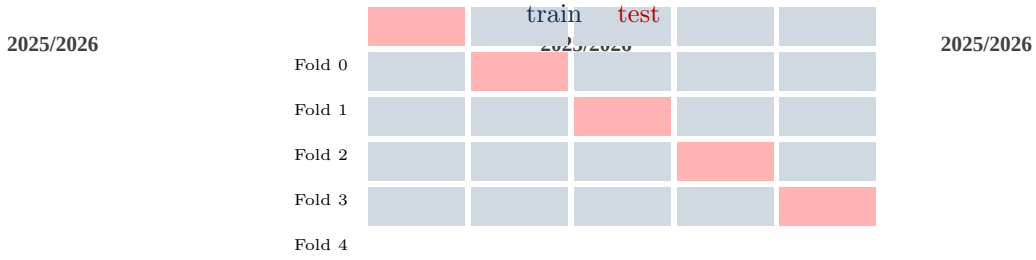


Figure 11.2: 5-fold cross-validation: the data is split into five folds, and the model is trained and evaluated five times, holding out a different fold as the test set each time. The final performance estimate is the average across all five folds.

The *bias–variance trade-off* formalizes overfitting: error decomposes into bias (systematic error from an overly simple model), variance (sensitivity to the particular training sample), and irreducible noise; increasing model complexity typically decreases bias but increases variance, exactly the pattern shown in Figure 11.1.

11.3 Linear Regression

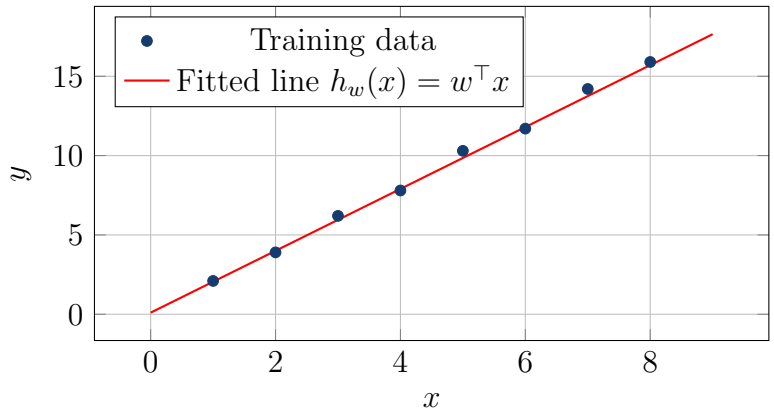
Linear regression fits $h_w(x) = w^\top x$ by minimizing squared error, $\text{Loss}(w) = \sum_i (y_i - w^\top x_i)^2$, which has a closed-form solution via the normal equations, or can be minimized

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

iteratively by *gradient descent*:

$$w \leftarrow w - \alpha \nabla_w \text{Loss}(w)$$

with learning rate α. *Regularization* (adding a penalty such as $\lambda\|w\|^2$, ridge regression, or $\lambda\|w\|_1$, lasso) discourages overly large weights and reduces overfitting, trading a small increase in training error for improved generalization.



2025/2026 2025/2026 2025/2026

Figure 11.3: Linear regression fit to a small dataset. The line minimizes the sum of squared vertical distances (residuals) between predicted and observed y values across all training points.

رومانی صبحی رزق جرجس مطر

11.4 Logistic Regression

For binary classification, logistic regression models $P(y = 1 \mid x) = \sigma(w^\top x)$ where σ is the logistic sigmoid, and fits w by maximizing the log-likelihood of the observed labels (equivalently, minimizing cross-entropy loss), typically via gradient-based methods since there is no closed-form solution. The decision boundary $P(y = 1 \mid x) = 0.5$ is linear in x, so logistic regression is a linear classifier despite the nonlinearity of σ.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

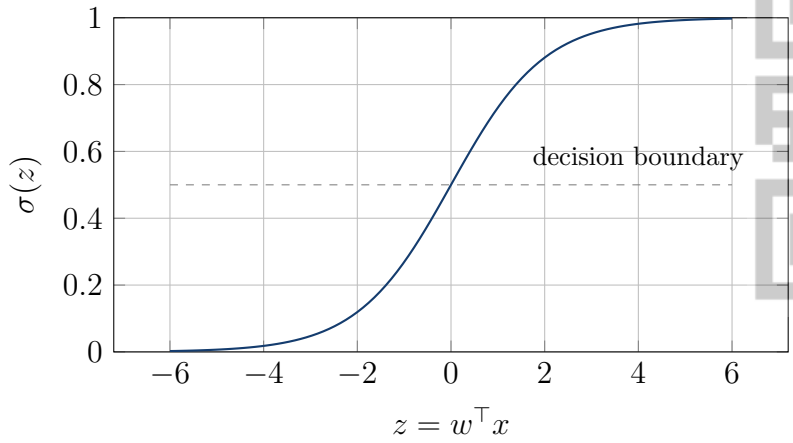


Figure 11.4: The logistic sigmoid function. Its output is interpreted as $P(y = 1 \mid x)$; the classifier predicts class 1 whenever $\sigma(z) > 0.5$, equivalently whenever $z = w^\top x > 0$, which is a linear condition on x.

11.5 Decision Trees

A decision tree classifies examples by recursively splitting on attribute tests, choosing at each node the attribute that maximizes *information gain* – the reduction in entropy of the label distribution achieved by the split:

2025/2026 2025/2026 2025/2026

$$\text{Gain}(A) = \text{Entropy}(S) - \sum_{v} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

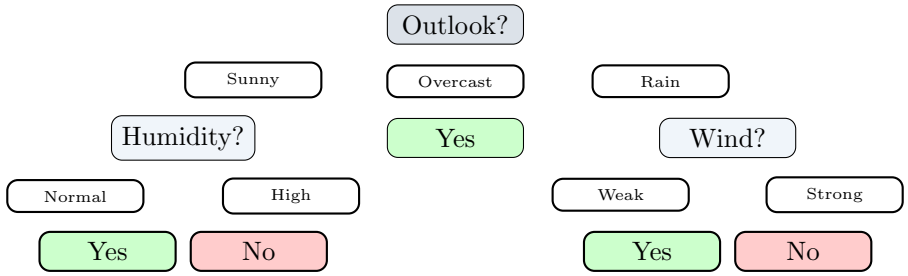


Figure 11.5: A decision tree for the classic "PlayTennis" dataset, splitting first on Outlook, and further on Humidity or Wind depending on the branch. Each internal node tests a single attribute; each leaf assigns a class label.

Fully grown trees tend to overfit; *pruning* (removing splits that do not significantly improve validation performance) or limiting tree depth controls this. *Random forests* and *gradient boosting* combine many trees to reduce variance and improve accuracy, at some cost to interpretability.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

11.6 Model Selection and the No Free Lunch Theorem

No single learning algorithm is best across all possible problems: the No Free Lunch theorem shows that, averaged over all possible data distributions, all algorithms perform equally. In practice, algorithm choice depends on the inductive bias appropriate to the actual distribution of problems encountered, which is why empirical model comparison via held-out data, not theoretical universality, guides practical algorithm selection.

Chapter Summary

- The goal of learning is generalization to unseen data, not fitting the training set; cross-validation and the bias–variance trade-off formalize how to measure and manage this.
- Linear and logistic regression fit simple, interpretable hypothesis classes via gradient-based optimization, with regularization controlling overfitting.
- Decision trees split on information gain and are prone to overfitting when grown fully; no algorithm dominates across all possible problems (No Free Lunch), so empirical comparison on held-out data guides practical algorithm choice.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Exercises

11.1 Explain the bias–variance trade-off and relate it to underfitting and overfitting using Figure 11.1 and a concrete example of model complexity (e.g., polynomial degree in regression).

11.2 Derive the gradient of the squared-error loss with respect to w for linear regression, and write out one step of gradient descent using the data in Figure 11.3.

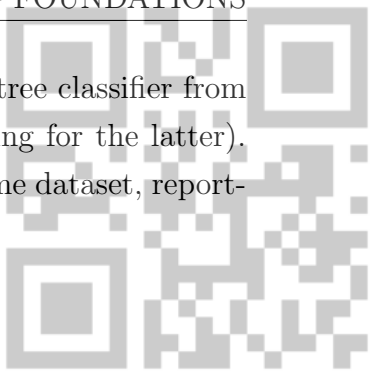
11.3 Compute the information gain for a small dataset and attribute of your choosing, showing entropy calculations at each step, and compare against the split shown in Figure 11.5.

11.4 Explain why k-fold cross-validation (Figure 11.2) gives a more reliable performance estimate than a single train/test split, and discuss the trade-off in computational cost.

11.5 Using Figure 11.4, explain why the logistic regression decision boundary is linear in x even though σ itself is a nonlinear function.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

11.6 **Implementation.** Implement logistic regression and a decision tree classifier from scratch (gradient descent for the former, information-gain splitting for the latter). Benchmark both against scikit-learn's implementations on the same dataset, reporting accuracy and discussing any discrepancies.



رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter 12

Neural Networks

رومانی صبحی رزق جرجس مطر

12.1 From Perceptrons to Multilayer Networks

A single perceptron computes $h_w(x) = \text{step}(w^\top x)$ and can only represent linearly separable functions – famously, it cannot represent XOR. Stacking units into *layers*, with a differentiable nonlinearity (sigmoid, tanh, or rectified linear unit, ReLU) replacing the hard step function, produces a *multilayer perceptron* (MLP) capable of approximating any continuous function to arbitrary accuracy given enough hidden units, a result known as the universal approximation theorem.

2025/2026 2025/2026 2025/2026

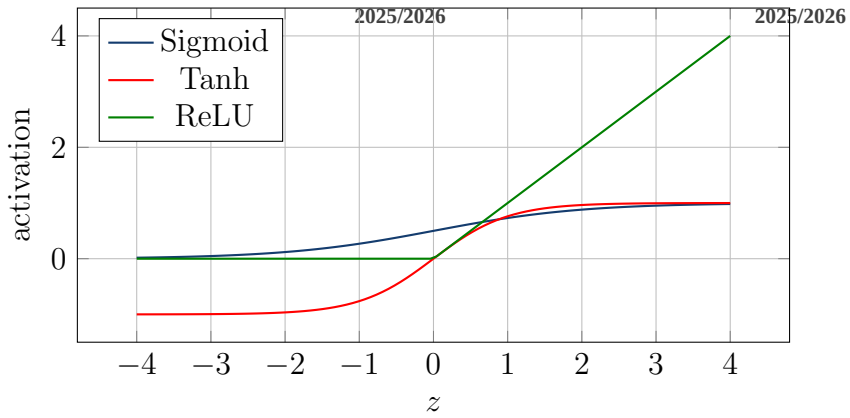


Figure 12.1: Three common activation functions. Sigmoid and tanh saturate for large $|z|$, contributing to vanishing gradients in deep networks; ReLU does not saturate for $z > 0$, which is a major reason it largely replaced sigmoid/tanh in hidden layers of deep networks.

12.2 Forward Propagation

A feedforward network computes its output by propagating the input through successive layers:

$$a^{(l)} = g\big(W^{(l)} a^{(l-1)} + b^{(l)}\big)$$

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

where g is the activation function, $W^{(l)}$ and $b^{(l)}$ are the weights and biases of layer l, and $a^{(0)} = x$. The final layer's activation, possibly with a softmax for multi-class classification, produces the network's prediction.

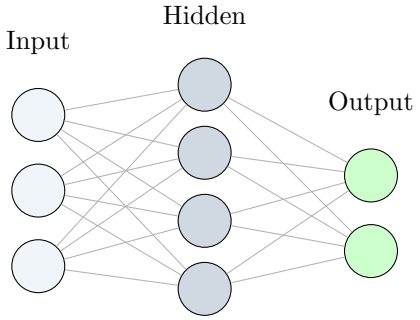


Figure 12.2: A fully connected feedforward network with one hidden layer: three input units, four hidden units, and two output units. Every unit in each layer connects to every unit in the next, each connection carrying its own trainable weight.

2025/2026 2025/2026 2025/2026

12.3 Backpropagation

Key Idea

Backpropagation computes the gradient of the loss with respect to every weight in the network efficiently, by applying the chain rule backward from the output layer, reusing intermediate quantities (error terms $\delta^{(l)}$) computed at each layer rather than recomputing derivatives from scratch for every weight.

Concretely, for output layer error $\delta^{(L)} = \nabla_a \text{Loss} \odot g'(z^{(L)})$, the error at earlier layers is computed recursively as $\delta^{(l)} = (W^{(l+1)\top}\delta^{(l+1)}) \odot g'(z^{(l)})$, and the gradient with respect to each layer's weights is $\nabla_{W^{(l)}}\text{Loss} = \delta^{(l)}a^{(l-1)\top}$. Backpropagation reduces the cost of computing all gradients from time proportional to the number of weights times the network depth (naive finite differences) to time proportional to a small constant times a single forward pass.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

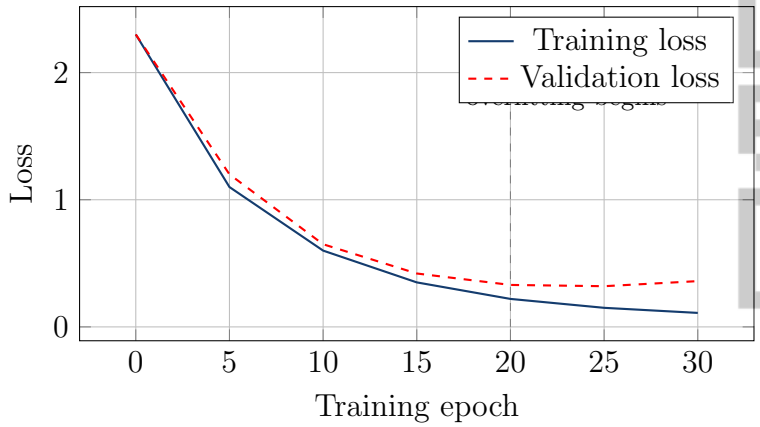


Figure 12.3: Typical training and validation loss curves. Training loss decreases monotonically, but validation loss begins to rise again once the network starts overfitting, motivating early stopping at the point where validation loss is minimized.

12.4 Training in Practice

Networks are trained by stochastic gradient descent (SGD) or adaptive variants such as Adam, updating weights on small *mini-batches* rather than the full dataset for computational efficiency and better generalization. Key practical considerations include weight initialization (poor initialization can cause vanishing or exploding gradients), learning-rate scheduling, and *regularization* techniques such as L_2 weight decay, dropout (randomly zeroing activations during training to prevent co-adaptation), and early stopping based on validation loss, as illustrated in Figure 12.3.

12.5 Convolutional Neural Networks

For grid-structured data such as images, *convolutional neural networks* (CNNs) replace fully connected layers with convolutional layers that apply small, shared filters across spatial locations. This exploits two structural properties of images: *locality* (nearby pixels are more related than distant ones) and *translation invariance* (a feature detector useful in one location is typically useful elsewhere), dramatically reducing the number of parameters compared to a fully connected network operating on raw pixels.

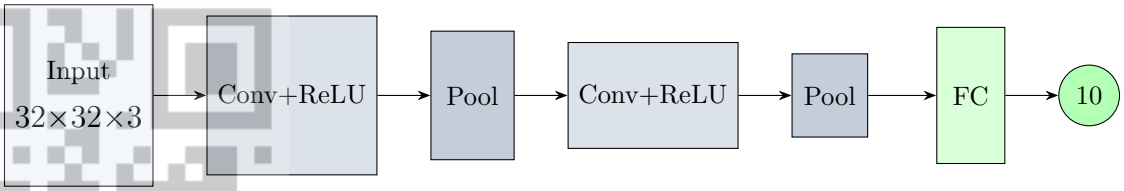


Figure 12.4: A typical CNN architecture for image classification: alternating convolution and pooling layers progressively reduce spatial resolution while increasing feature depth, followed by fully connected (FC) layers producing class scores (here, 10 classes).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Pooling layers downsample feature maps, providing a degree of robustness to small spatial shifts.

Worked Example

A CNN trained on handwritten digit recognition (MNIST) learns early-layer filters resembling edge detectors, and later layers combine these into increasingly abstract features (loops, strokes, digit parts), illustrating how depth allows hierarchical feature learning rather than requiring hand-engineered features as in classical computer vision.

12.6 Loss Functions and Output Layers

Regression networks typically use mean squared error with a linear output layer; classification networks use cross-entropy loss with a softmax output layer, which, combined, yield particularly clean gradients ($\hat{y} - y$ at the output layer) that simplify both the mathematics and the implementation of backpropagation.

Chapter Summary

- Multilayer networks with nonlinear activations overcome the single perceptron's inability to represent non-linearly-separable functions such as XOR.
- Backpropagation computes all gradients efficiently via the chain rule, propagating error terms backward from the output layer.
- Practical training relies on stochastic gradient descent variants, careful initialization, and regularization (weight decay, dropout, early stopping); convolutional networks exploit locality and translation invariance for grid-structured data such as images.

Exercises

12.1 Show that a single-layer perceptron cannot represent the XOR function, and explain, geometrically, why a hidden layer resolves this.

12.2 Derive the backpropagation update for a two-layer network with sigmoid activations and mean-squared-error loss, writing out $\delta^{(2)}$ and $\delta^{(1)}$ explicitly, referencing Figure 12.2.

12.3 Using Figure 12.1, explain the vanishing gradient problem for deep sigmoid networks and describe two practical mitigations, including why ReLU helps.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

12.4 Explain why weight sharing in convolutional layers reduces parameter count compared to a fully connected layer processing the same image, with a concrete numerical example based on Figure 12.4.

12.5 Using Figure 12.3, explain how early stopping is chosen in practice and why it is a form of regularization.

12.6 **Implementation.** Implement a feedforward neural network with backpropagation from scratch (no autodiff library) for a small classification task. Compare training curves and final accuracy against an equivalent PyTorch model, and discuss any differences you observe.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter 13

Reinforcement Learning

رومانی صبحی رزق جرجس مطر

13.1 The Reinforcement Learning Problem

Reinforcement learning (RL) addresses sequential decision-making, as in Chapter 10's MDPs, but without a known transition model or reward function: the agent must learn a good policy purely from the rewards it experiences by acting in the environment. This introduces the central tension of *exploration versus exploitation*: the agent must try actions it is uncertain about to discover their value, while also exploiting actions already known to be good.

2025/2026 2025/2026 2025/2026

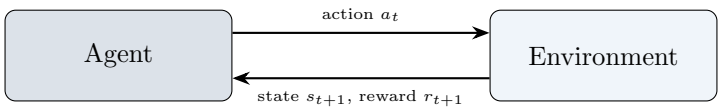
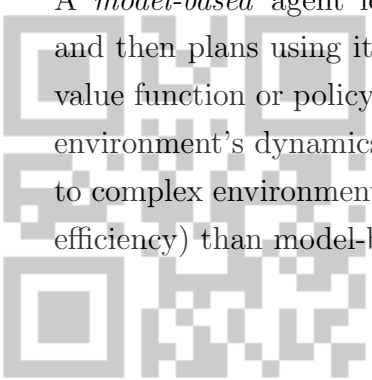


Figure 13.1: The reinforcement learning loop. Unlike the MDP setting of Chapter 10, the agent does not know $P(s' \mid s, a)$ or R in advance; it must estimate a good policy purely from the stream of states, actions, and rewards it experiences.

13.2 Model-Based versus Model-Free Learning

A *model-based* agent learns an explicit transition and reward model from experience and then plans using it (e.g., with value iteration). A *model-free* agent instead learns a value function or policy directly from experience without ever explicitly representing the environment's dynamics. Model-free methods are simpler to implement and scale better to complex environments, at the cost of typically needing more experience (lower sample efficiency) than model-based alternatives.



UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

13.3 Temporal-Difference Learning

Key Idea

Temporal-difference (TD) learning updates a value estimate using the difference between successive predictions rather than waiting for the final outcome of an episode:

$$U(s) \leftarrow U(s) + \alpha\big[R(s) + \gamma\, U(s') - U(s)\big]$$

This bootstraps from the agent's own current estimates, allowing online, incremental learning without a model of the environment.

13.4 Q-Learning

Q-learning learns an action-value function $Q(s, a)$, the expected return of taking action a in state s and acting optimally thereafter, using the update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

Because the update target uses $\max_{a'} Q(s', a')$ regardless of which action was actually taken next, Q-learning is an *off-policy* algorithm: it learns the optimal action-value function even while following an exploratory policy such as ϵ-greedy (acting greedily with probability $1 - \epsilon$ and randomly otherwise).

2025/2026 2025/2026 2025/2026

13.5 SARSA

SARSA (State–Action–Reward–State–Action) uses the update

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]$$

where a' is the action actually selected next by the current policy, making SARSA *on-policy*: it learns the value of the policy it is actually following, including its exploration.



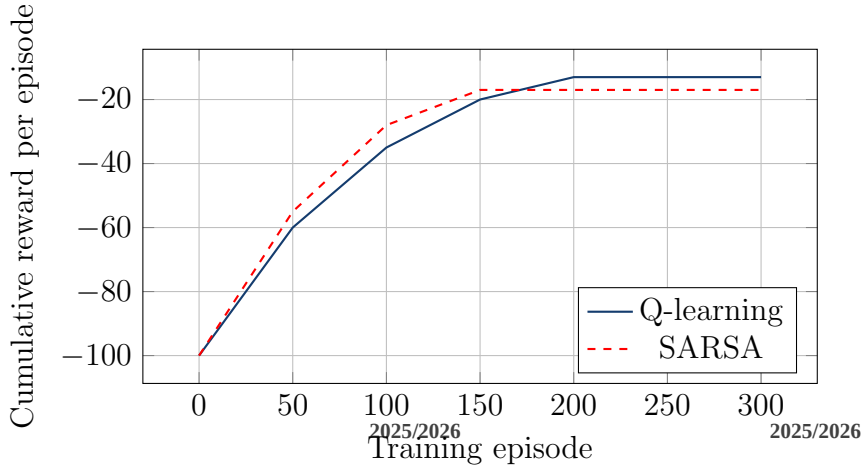
Figure 13.2: The cliff-walking gridworld. Q-learning tends to learn the shortest path directly along the cliff edge, since it learns the value of the optimal policy regardless of exploration; SARSA tends to learn a safer path further from the cliff, because its value estimates incorporate the risk of an ϵ-greedy exploratory step causing a fall.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

13.6 Function Approximation

Tabular Q-learning and SARSA require storing a value for every state–action pair, infeasible for large or continuous state spaces. *Function approximation* replaces the table with a parameterized function, such as a linear function of hand-designed features or a neural network, trained to minimize the same TD error. Combining Q-learning with a deep neural network (deep Q-networks) enabled RL agents to learn control policies directly from raw pixel input, though function approximation introduces stability challenges absent in the tabular setting, addressed in practice by techniques such as experience replay and target networks.



2025/2026 2025/2026 2025/2026

Figure 13.3: Learning curves for Q-learning and SARSA on cliff-walking (Figure 13.2). Q-learning converges to a higher asymptotic reward (the shorter path) but the policy it learns occasionally falls off the cliff during ϵ-greedy exploration, so its *online* performance during training can be worse than SARSA's despite Q-learning's higher-value final policy.

13.7 Policy Search Methods

Rather than learning a value function and deriving a policy from it, *policy gradient* methods directly parameterize a policy $\pi_\theta(a \mid s)$ and adjust θ to increase the probability of actions that led to higher returns, using the gradient estimator

$$\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t\big]$$

Policy gradient methods handle continuous action spaces naturally and can represent stochastic policies directly, at the cost of typically higher variance in the gradient estimate than value-based methods.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter Summary

- Reinforcement learning solves sequential decision problems without a known transition or reward model, learning instead from experience, and must balance exploration against exploitation.
- Temporal-difference methods (Q-learning, off-policy; SARSA, on-policy) bootstrap value estimates from successive predictions rather than waiting for episode completion.
- Function approximation extends these methods to large or continuous state spaces at some cost to stability; policy gradient methods instead parameterize and directly optimize a policy.

Exercises

13.1 Explain the exploration–exploitation trade-off and describe two strategies (other than ϵ-greedy) for balancing it.

13.2 Derive why Q-learning is off-policy and SARSA is on-policy by comparing their update targets term by term.

13.3 Explain, using Figure 13.2 and Figure 13.3, why on-policy and off-policy methods can converge to qualitatively different policies even on the same MDP, and why their online training performance can rank differently than their final policy quality.

13.4 Explain why function approximation can destabilize Q-learning, and describe how experience replay and target networks address this.

13.5 **Implementation.** Implement tabular Q-learning and SARSA for a gridworld with a cliff or similarly risky region. Reproduce figures similar to Figure 13.2 and Figure 13.3 using your own results.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 14

Language and Perception: From N-grams to Transformers

14.1 Language Models

A language model assigns a probability to a sequence of words, $P(w_1, \ldots, w_n)$, which by the chain rule factors as $\prod_i P(w_i \mid w_1, \ldots, w_{i-1})$. An *n-gram* model approximates this by assuming each word depends only on the preceding $n-1$ words, e.g., a bigram model uses $P(w_i \mid w_{i-1})$, estimated by counting occurrences in a training corpus with smoothing to handle unseen sequences.

Key Idea

Every language model in this chapter, from bigrams to transformers, solves the same problem: estimate $P(w_i \mid \text{context})$. What changes across the chapter is only how much context is used and how that context is represented – a fixed short window of raw words (n-grams), a fixed-size hidden state summarizing an unbounded past (RNNs), or direct, learned access to every previous position at once (attention).

N-gram models are simple and fast but cannot capture dependencies beyond their fixed window, nor generalize well between semantically related words that happen not to co-occur in training – a bigram model that has seen "the cat sat" many times but never "the feline sat" assigns the latter a low probability even though the two sentences are near-synonymous.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

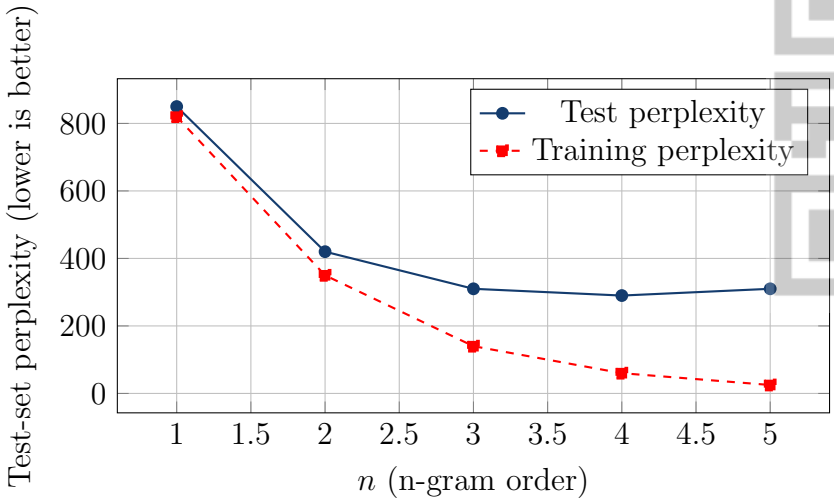


Figure 14.1: Illustrative perplexity as a function of n-gram order. Training perplexity keeps falling as n grows (the model memorizes longer and longer exact phrases), but test perplexity bottoms out and then rises, because higher-order n-grams are seen too rarely in training to estimate reliably – a direct instance of the overfitting pattern from Chapter 11's bias–variance discussion.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

14.2 Word Embeddings

Word embeddings represent each word as a dense vector in a continuous space, learned so that words appearing in similar contexts end up with similar vectors – an operationalization of the distributional hypothesis that a word's meaning is reflected in its usage contexts. Models such as word2vec learn embeddings by training a shallow network to predict a word from its context (or vice versa).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

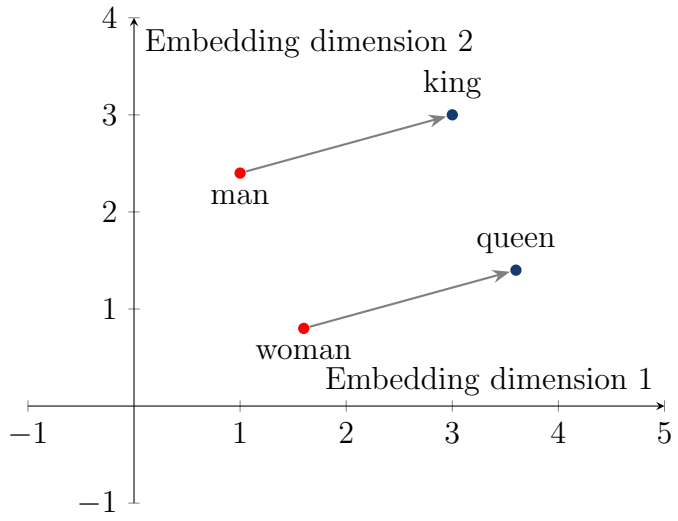


Figure 14.2: A schematic (dimension-reduced) view of word embeddings. The offset vector from "man" to "king" is approximately parallel to the offset from "woman" to "queen," illustrating how relational structure (here, royalty) can emerge as a roughly consistent direction in embedding space purely from co-occurrence statistics, with no explicit relational supervision provided during training.

14.3 Recurrent Networks and Their Limits

Recurrent neural networks (RNNs) process sequences by maintaining a hidden state updated at each time step, $h_t = f(h_{t-1}, x_t)$, in principle allowing dependencies of unbounded length.

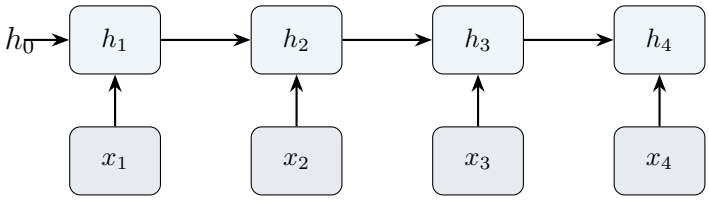


Figure 14.3: An RNN unrolled over four time steps. Each hidden state h_t depends on the current input x_t and the previous hidden state h_{t-1}, so information from x_1 can in principle influence h_4, but only by passing through three intermediate transformations, each of which can attenuate (vanishing gradient) or amplify (exploding gradient) the signal.

In practice, vanilla RNNs struggle with long-range dependencies due to vanishing gradients through many time steps, exactly the phenomenon introduced for deep feedforward networks in Chapter 12 but compounded here by the number of time steps rather than the number of layers. *Long short-term memory* (LSTM) and gated recurrent units address this with gating mechanisms that allow gradients to flow largely unimpeded across many steps when appropriate, using learned gates to decide what to retain, forget, and output at each step.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

14.4 Attention and the Transformer

Key Idea

The *attention* mechanism allows a model to compute, for each position in a sequence, a weighted combination of information from all other positions, with weights determined by learned compatibility between a *query* and each position's *key*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

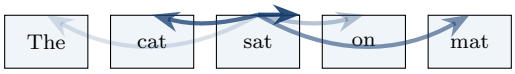


Figure 14.4: Self-attention from the word "sat" (as query) to every word in the sentence (as keys), with line opacity representing attention weight. Unlike the RNN in Figure 14.3, every word is reachable in a single step, with weight determined by learned relevance rather than by sequential distance.

The *transformer* architecture builds entirely on attention (specifically, self-attention layers interleaved with feedforward layers), dispensing with recurrence altogether. Figure 14.5 shows the structure of one transformer block.

2025/2026 2025/2026 2025/2026

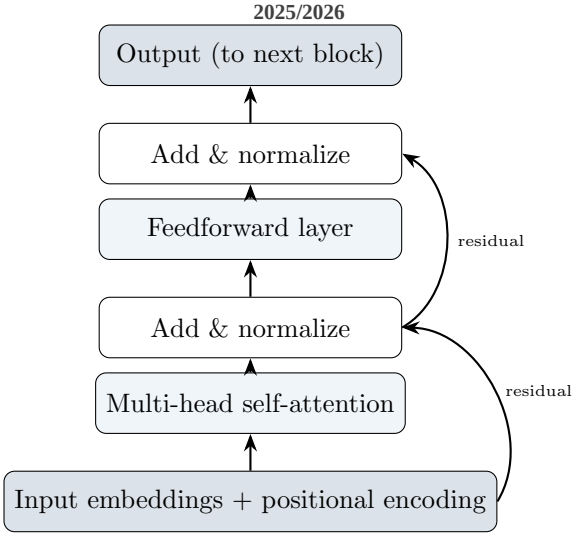


Figure 14.5: One transformer block. Residual ("skip") connections around both the attention and feedforward sublayers, combined with normalization, are what make it practical to stack many such blocks without the vanishing-gradient problems that plague very deep or very long RNNs.

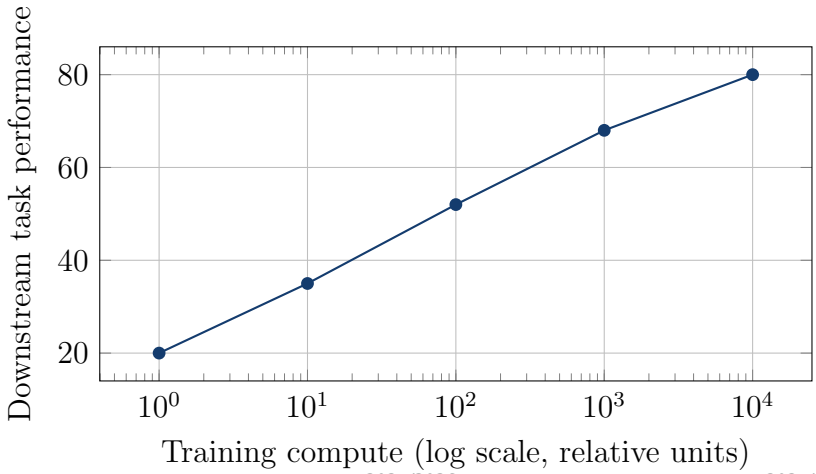
This allows all positions in a sequence to be processed in parallel during training (unlike RNNs, which are inherently sequential) and gives every position direct access to every other position's information regardless of distance. Since attention itself has no

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

notion of sequence order, transformers add *positional encodings* to the input to inject order information.

14.5 Large Language Models

Large language models (LLMs) are transformer-based models trained on massive text corpora to predict the next token in a sequence, using the same factorization from Section 14.1 but with a context window and representational capacity far beyond n-gram or recurrent models.



2025/2026 2025/2026 2025/2026

Figure 14.6: Schematic illustration of an empirical *scaling law*: downstream performance tends to improve smoothly and predictably as training compute (a combination of model size and data) increases, across several orders of magnitude, though the curve eventually flattens and the relationship can vary across task types.

Scaling model size, data, and compute together has produced models capable of few-shot generalization to new tasks specified only by a natural-language prompt, though these models inherit biases present in their training data and can produce fluent but factually incorrect output, a phenomenon commonly called hallucination, motivating careful evaluation before deployment – a theme developed further in Chapter 15.

14.6 Perception: A Brief Overview

Computer vision applies many of the same learned representations introduced in Chapter 12: convolutional networks for image classification and object detection, and increasingly, transformer-based architectures (*vision transformers*) adapted to treat fixed-size image patches as if they were sequence elements, applying exactly the self-attention machinery of Figure 14.4 across patches rather than words. Perception systems share machine

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

learning's core challenge of generalizing beyond the training distribution, which is particularly consequential for physically embodied systems such as autonomous vehicles, where distributional shift can have safety implications (Chapter 15).

Worked Example

Machine translation illustrates the progression in this chapter: early statistical systems relied on n-gram language models and phrase tables; RNN-based encoder–decoder models with attention (an early use of Figure 14.4's mechanism) improved fluency by letting the decoder attend to relevant source words at each step; transformer-based models now dominate the field by replacing recurrence with self-attention throughout, exactly the architecture in Figure 14.5.

14.7 Worked Example: Estimating a Bigram Model

Given the tiny training corpus "the cat sat . the dog sat . the cat ran ." (with sentence boundaries marked by periods), we estimate bigram probabilities by counting: $P(\text{cat} \mid \text{the}) = 2/3$, $P(\text{dog} \mid \text{the}) = 1/3$, $P(\text{sat} \mid \text{cat}) = 1/2$, $P(\text{ran} \mid \text{cat}) = 1/2$. The probability of the novel sentence "the dog ran" is then

$$P(\text{dog} \mid \text{the}) \times P(\text{ran} \mid \text{dog})$$

2025/2026 2025/2026 2025/2026

but $P(\text{ran} \mid \text{dog})$ was never observed in training, so a raw count-based estimate assigns it probability zero, incorrectly implying the sentence is impossible. *Smoothing* (e.g., adding a small count to every possible bigram before normalizing, Laplace smoothing) assigns such unseen bigrams a small nonzero probability instead, trading a little accuracy on frequent bigrams for the ability to assign a sensible probability to any sequence at all.

Chapter Summary

- Language models estimate $P(w_i \mid \text{context})$; n-grams, RNNs, and transformers differ mainly in how much context they use and how it is represented.
- Word embeddings capture distributional meaning as dense vectors; RNNs and LSTMs extend context across time at the cost of sequential computation and vanishing gradients.
- Attention and the transformer architecture allow direct, parallelizable access to all sequence positions, underlying modern large language models and, increasingly, vision models as well.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Exercises

14.1 Estimate bigram probabilities from a small text corpus of your choosing and use them to compute the probability of a short test sentence, including a discussion of how you handle unseen bigrams as in Section 14.7.

14.2 Explain, at a conceptual level, why word embeddings can represent analogies (e.g., king – man + woman ≈ queen) as vector arithmetic, referencing Figure 14.2.

14.3 Explain the vanishing gradient problem in RNNs and how gating in LSTMs mitigates it, connecting your answer to Figure 14.3.

14.4 Explain why transformers can be parallelized during training in a way that RNNs cannot, and why positional encodings are necessary as a consequence.

14.5 Using Figure 14.1, explain why the test-perplexity curve eventually rises even as training perplexity keeps falling, and relate this explicitly to the bias–variance discussion in Chapter 11.

14.6 Explain what a residual connection in Figure 14.5 does mechanically (what is being added to what), and why removing all residual connections would make very deep transformers hard to train.

14.7 **Implementation.** Train a simple word-level bigram or trigram language model on a provided text corpus, and compare its generated text and perplexity against a small pretrained transformer language model on the same corpus.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 15

AI Ethics, Safety, and Governance

15.1 Why Ethics Belongs in a Technical Course

The algorithms in earlier chapters are tools; the consequences of deploying them depend on choices made throughout the pipeline – what data is collected, what objective is optimized, who is affected, and who is accountable when the system errs. Figure 15.1 maps these decision points onto the standard machine learning pipeline from Chapter 11, making explicit that "the model" is only one of several places where a harmful choice can enter the system.

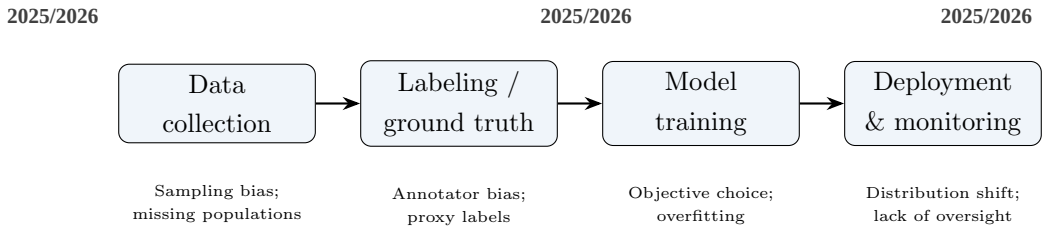


Figure 15.1: The machine learning pipeline of Chapter 11, annotated with a representative failure mode that can be introduced at each stage. A model can be technically correct with respect to its training objective and still cause harm if the harm was introduced earlier in the pipeline, which is why auditing the whole pipeline, not just the trained model, is necessary.

15.2 Bias and Fairness

Machine-learned systems can reproduce or amplify biases present in their training data, in the labels used, or in the choice of objective function, even when no protected attribute is used directly as a feature, since correlated proxy features can reintroduce the same disparity.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Key Idea

There is no single, universally correct measure of algorithmic fairness. Any deployment decision requires an explicit choice among competing definitions, informed by the specific harms and stakeholders involved, not resolved by technical means alone.

Table 15.1: A stylized example showing why equal accuracy across two groups does not imply equal treatment. Both groups have 90% overall accuracy, but Group B suffers a much higher false-negative rate – relevant if a false negative means, e.g., a missed loan approval or missed medical diagnosis.

Group	Overall accuracy	False positive rate	False negative rate
Group A (majority, large sample)	90%	5%	8%
Group B (minority, small sample)	90%	2%	22%

Multiple formal definitions of fairness exist (e.g., demographic parity, equalized odds, predictive parity), and a well-known impossibility result shows that several intuitively desirable fairness criteria cannot all be satisfied simultaneously except in special cases (such as when base rates are equal across groups), meaning fairness work requires an explicit choice of which criterion matters most for a given application, not a purely technical fix.

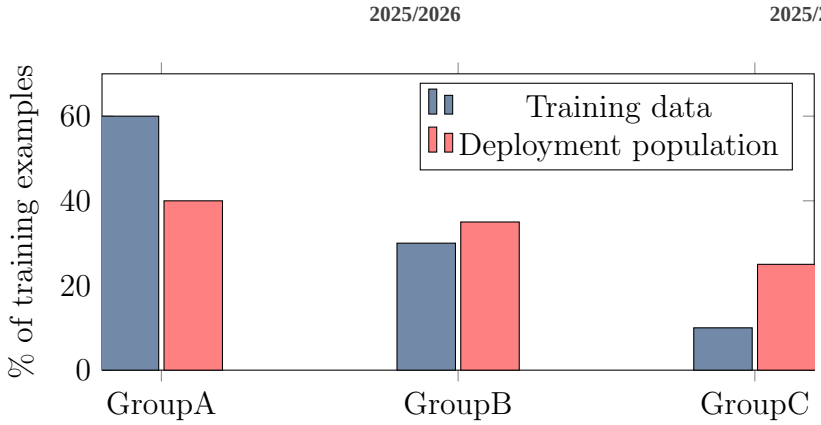


Figure 15.2: A representation gap: the training data's group composition does not match the population the system is actually deployed on. Group C is under-represented by more than half relative to the deployment population, which typically translates into worse model performance for that group even without any explicit protected attribute ever being used as a feature.

15.3 Robustness, Reliability, and Safety

Learned models can fail in ways that differ sharply from human error patterns, including susceptibility to small, often imperceptible input perturbations (adversarial examples)

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

that cause confident misclassification, and silent degradation under distributional shift between training and deployment conditions.

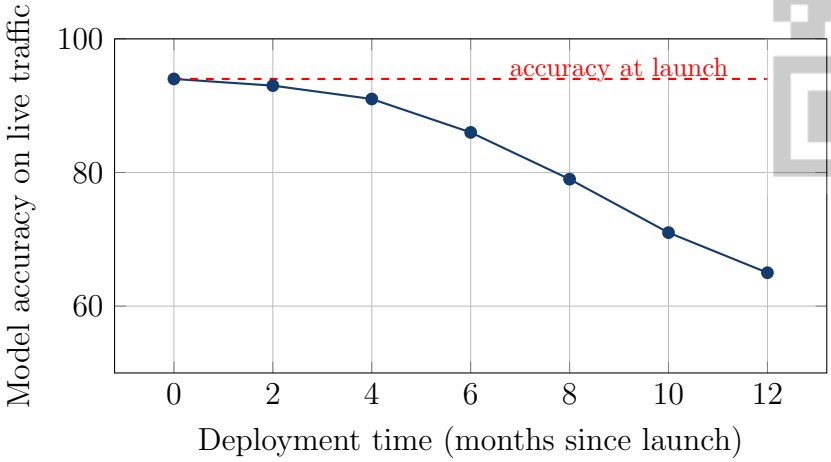


Figure 15.3: Schematic illustration of distributional shift: a model's accuracy on live traffic can degrade steadily after deployment as the real-world data distribution drifts away from the training distribution, even though nothing about the model itself has changed. Monitoring for this kind of drift is a core part of responsible deployment, not a one-time validation step.

For safety-critical applications, this motivates uncertainty quantification (so a system can recognize and flag low-confidence predictions rather than acting on them silently), human oversight, and thorough testing under conditions that resemble real-world deployment rather than only the training distribution.

15.4 Privacy

Training data, especially data scraped from public sources, can contain personal information, and some models can be induced to reveal memorized training examples verbatim. Techniques such as differential privacy add calibrated noise during training to bound how much any single training example can influence the model, providing a mathematical guarantee of limited information leakage at some cost to model accuracy – another instance of a trade-off, this time between privacy and utility, that must be navigated explicitly rather than assumed away.

15.5 Transparency and Explainability

Many high-performing models, particularly deep neural networks, are difficult to interpret directly. *Explainability* techniques (feature attribution, surrogate models, attention visualization such as Figure 14.4 in the previous chapter) attempt to make model decisions more interpretable to affected individuals, auditors, or regulators, though these

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

techniques themselves have known limitations, including cases where explanations do not faithfully reflect the actual computation the model performed.

15.6 Labor, Environmental, and Economic Impact

Training large models consumes substantial computational resources and energy; deploying AI systems can displace or reshape labor in affected industries and, in the case of data-labeling and content-moderation work, can rely on labor performed under difficult conditions. These are not incidental side effects but design-relevant costs that a complete evaluation of a system's benefits should weigh explicitly.

15.7 Governance and Regulation

2025/2026 2025/2026 2025/2026

Governments and standards bodies have begun developing AI-specific regulation and voluntary frameworks addressing risk assessment, transparency requirements, and use restrictions in sensitive domains such as hiring, credit, criminal justice, and healthcare. Because regulatory approaches and requirements continue to evolve rapidly and differ across jurisdictions, students working on real deployments should verify the current legal and institutional requirements applicable to their specific domain and location rather than relying on any fixed summary.

15.8 A Practical Checklist

Bringing together this chapter's themes, Table 15.2 offers a starting checklist for evaluating a proposed AI system – exactly the kind of analysis expected in the ethical-analysis component of the term project (Appendix B).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Table 15.2: A starting checklist for evaluating a deployed or proposed AI system, organized by this chapter's sections.

Axis	Guiding question
Bias/fairness	Which fairness criterion matters most here, and does the training data represent all affected groups adequately?
Robustness	How does performance degrade under distribution shift, and is there a mechanism to detect it after deployment?
Privacy	Could the model leak sensitive information about individuals in its training data?
Transparency	Can an affected person get an explanation for a decision that affects them?
Labor/environmental	What are the compute, data-labeling, and downstream labor costs of this system?
Governance	What regulations or institutional policies apply to this domain and jurisdiction?

Worked Example

A hiring-screening model trained on a company's historical hiring decisions can learn to reproduce whatever biases were present in those past decisions, even after the protected attribute itself is removed from the feature set, if correlated features (school, zip code, employment gaps) carry similar information – illustrating, via Figure 15.1, that the harm originates in the labeling stage (the historical decisions used as ground truth), not in the training algorithm itself.

Chapter Summary

- Harmful outcomes can enter an AI pipeline at any stage – data collection, labeling, training, or deployment – not only inside the model itself.
- No single fairness criterion is universally correct; robustness, privacy, transparency, and labor/environmental costs each require explicit, domain-specific trade-offs rather than a one-size-fits-all technical fix.
- Responsible deployment requires ongoing monitoring (for distributional shift), not only pre-deployment validation, and must account for the current legal and regulatory context.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Exercises

15.1 Using Table 15.1, compute the positive predictive value for each group and explain why equal overall accuracy can coexist with substantially different error profiles.

15.2 Explain the impossibility result concerning fairness criteria using a concrete numerical example with two groups of differing base rates.

15.3 Describe an adversarial example conceptually, and explain why robustness to such examples does not follow automatically from high accuracy on a clean test set.

15.4 Using Figure 15.3, propose two concrete monitoring signals a deployed system could track to detect distributional shift before accuracy visibly degrades.

15.5 Explain, at a conceptual level, what differential privacy guarantees and what it does not guarantee.

15.6 Choose a real or hypothetical deployed AI system and analyze it using Table 15.2, identifying at least one concrete mitigation for each axis.

15.7 For your term project (Appendix B), write a one-page ethical analysis of your chosen application, identifying stakeholders, likely failure modes at each pipeline stage in Figure 15.1, and at least one mitigation you implemented or would recommend.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

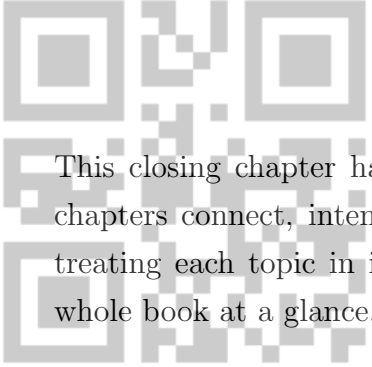
رومانی صبحی رزق جرجس مطر

Chapter 16

Synthesis: How the Pieces Fit Together

2025/2026 2025/2026 2025/2026

16.1 One Course, Several Threads



This closing chapter has no new algorithms; it is a map of how the preceding fifteen chapters connect, intended as a study guide for the final exam and as a corrective to treating each topic in isolation. Figure 16.1 lays out the dependency structure of the whole book at a glance.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

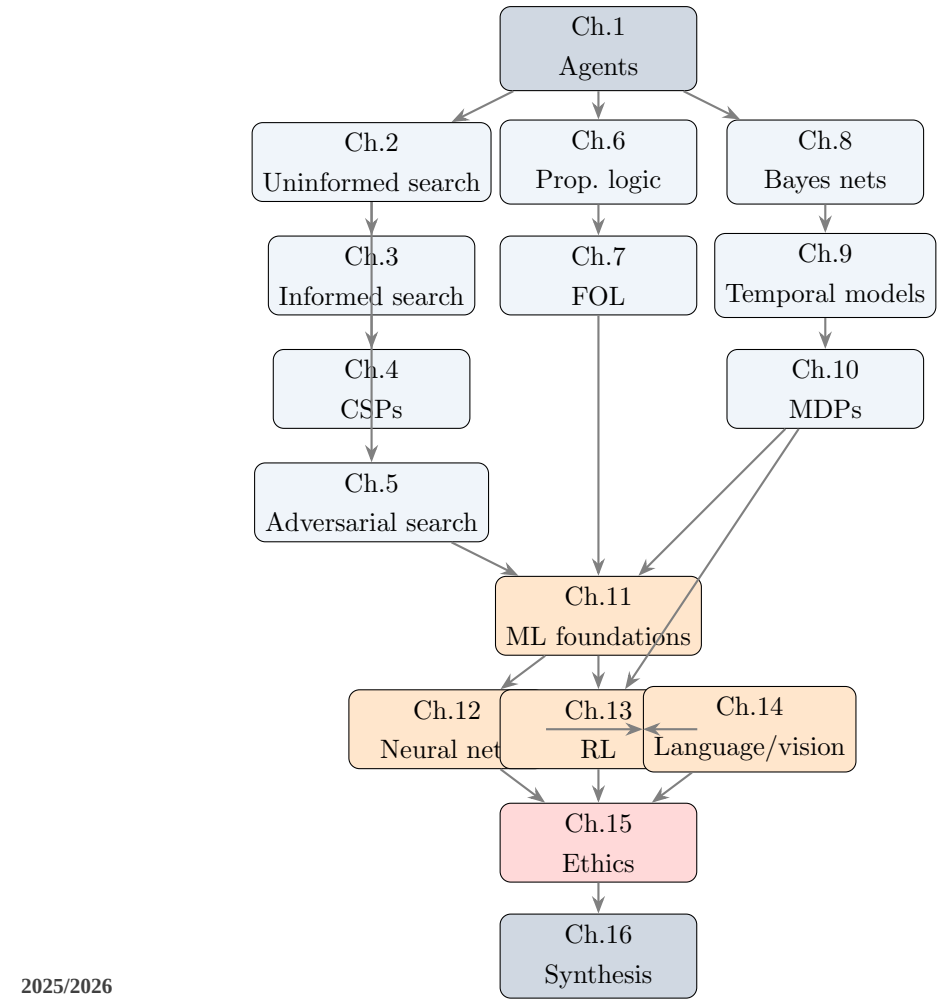


Figure 16.1: Dependency map of the book. The search thread (left, Chapters 2–5), the knowledge/reasoning thread (center, Chapters 6–7), and the uncertainty thread (right, Chapters 8–10) all feed into the learning thread (Chapters 11–14), which in turn motivates the ethics discussion (Chapter 15) that closes the course.

16.2 The Search Thread

Chapters 2–5 are united by a common abstraction: a state space searched by expanding a frontier under some ordering. Uninformed methods order the frontier by depth or accumulated cost; informed methods add a heuristic estimate of remaining cost; adversarial search adds a second, opposing agent whose choices must be accounted for. Constraint satisfaction (Chapter 4) is best understood as search over a factored state representation, which is what makes propagation and variable-ordering heuristics possible in a way that is harder to exploit in unstructured search.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

16.3 The Knowledge and Reasoning Thread

Chapters 6–7 develop logic as an alternative to search-based problem-solving when a problem is more naturally described declaratively: a knowledge base plus an inference procedure, rather than states and actions. The transition from propositional to first-order logic is driven entirely by the need to generalize over objects, at the cost of losing the propositional case's decidability.

16.4 The Uncertainty Thread

Chapters 8–10 replace logical certainty with probability once an agent cannot know the world's state exactly. Bayesian networks (Chapter 8) compress a joint distribution using conditional independence; temporal models (Chapter 9) apply the same idea to sequences of hidden states; and MDPs (Chapter 10) add a decision-maker choosing actions to maximize expected utility under that same uncertainty. Reinforcement learning (Chapter 13) is best understood as solving an MDP when its transition and reward structure are unknown and must be discovered through interaction.

16.5 The Learning Thread

2025/2026 2025/2026 2025/2026

Chapters 11–14 shift from hand-specified models to models fit from data. Chapter 11 establishes the central concern of generalization that recurs everywhere after it: neural networks (Chapter 12) are a particular, highly flexible hypothesis class trained by gradient descent; reinforcement learning (Chapter 13) applies learning to sequential decisions; and language and perception models (Chapter 14) apply the same underlying machinery (learned representations, gradient-based training) to structured, high-dimensional inputs.

Key Idea

A recurring pattern across the whole course: define a formal representation of the problem (states, sentences, distributions, or data), define a notion of optimality or fit with respect to that representation, and design an algorithm that provably or empirically approaches it. Recognizing this pattern is more useful for the exam, and for future coursework, than memorizing any single algorithm in isolation.

16.6 A Cross-Chapter Comparison Table

Table 16.1 makes the recurring pattern from the Key Idea box concrete by naming, for a representative technique from each thread, exactly what plays the role of "representa-

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

tion," "objective," and "algorithm."

Table 16.1: The shared representation/objective/algorithm pattern, instantiated across four representative techniques from the course.

Technique	Representation	Objective	Algorithm
A* (Ch. 3)	States & actions	Minimize path cost	Best-first expansion by $f(n)$
Bayes nets (Ch. 8)	Conditional independencies	Match true joint distribution	Variable elimination / sampling
MDPs (Ch. 10)	States, actions, rewards	Maximize expected discounted return	Value / policy iteration
Neural nets (Ch. 12)	Weighted, layered functions	Minimize training loss	Gradient descent + backpropagation

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

16.7 Full-Course Exam Preparation Table

Table 16.2 lists, for every chapter, the single most important algorithm or result and the one question most likely to appear about it on a comprehensive exam – a compact starting point for review, to be supplemented by the chapter summaries and exercises throughout the book.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Table 16.2: A chapter-by-chapter exam preparation summary.

Chapter	Key result	Likely exam question
1	Rational agent definition	Give a PEAS description and classify an environment.
2	BFS/UCS/DFS/IDS trade-offs	Trace an algorithm and state its complexity.
3	A* optimality	Prove admissibility/consistency; trace A* on a graph.
4	Arc consistency	Trace AC-3; formulate a CSP.
5	Alpha-beta pruning	Trace minimax and alpha-beta on the same tree.
6	Resolution refutation	Convert to CNF and derive the empty clause.
7	Unification	Find a most general unifier; translate English to FOL.
8	Bayesian network factorization	Compute a posterior via Bayes' rule or variable elimination.
9	HMM filtering	Trace one filtering update; explain particle filtering.
10	Bellman equation	Perform one value-iteration backup by hand.
11	Bias–variance trade-off	Explain over/underfitting with a concrete example.
12	Backpropagation	Derive a gradient for a small network by hand.
13	Q-learning vs. SARSA	Explain on-policy vs. off-policy with the cliff example.
14	Self-attention	Explain why transformers parallelize and RNNs do not.
15	Fairness trade-offs	Analyze a scenario using the ethics checklist.
16	Cross-chapter connections	Explain how two techniques from different chapters relate.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

16.8 Where This Leaves You

Chapter 15's discussion of ethics and governance is not a separate topic from the rest of the course but a constraint on how the preceding techniques should be applied: every algorithm in this book makes assumptions (about the environment, about the data, about

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

the objective) that can fail silently in deployment, and evaluating a system honestly means checking those assumptions rather than only checking accuracy on a held-out set drawn from the same distribution as training data.

Beyond this course, natural next steps include a dedicated course in machine learning or deep learning (extending Chapters 11–14), a course in robotics or planning (extending Chapters 2–5 and 10), and a course in natural language processing (extending Chapter 14). The references in the bibliography, particularly Russell and Norvig for breadth and Sutton and Barto, Bishop, or Goodfellow, Bengio, and Courville for depth in specific areas, remain useful well beyond this course.

Chapter Summary

- The course's four threads – search, knowledge/reasoning, uncertainty, and learning – share a common pattern: representation, objective, and algorithm.
- Reinforcement learning connects the uncertainty and learning threads by solving MDPs when their dynamics are unknown; ethics is a constraint on every technique, not a separate topic.
- Recognizing shared structure across chapters is more useful for long-term retention than memorizing any single algorithm in isolation.

2025/2026 2025/2026 2025/2026

Exercises

16.1 Pick any two chapters from Figure 16.1 that are not directly connected by an arrow, and explain one substantive connection between their techniques anyway.

16.2 Reconstruct, from memory, the Bellman optimality equation (Chapter 10) and the Q-learning update (Chapter 13), and explain precisely what changes between them and why.

16.3 Explain how a Bayesian network (Chapter 8) is related to a hidden Markov model (Chapter 9) as a special case, identifying what structural assumption the HMM adds.

16.4 Fill in a fifth row of Table 16.1 for a technique of your choosing from Chapters 2–9, identifying its representation, objective, and algorithm.

16.5 Choose one algorithm from this book and describe how its underlying assumptions (about the environment, about the data distribution, or about the objective) could fail in a real deployment, connecting this to Chapter 15's pipeline diagram (Figure 15.1).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

16.6 As a comprehensive review exercise, redraw Figure 16.1 from memory, and be prepared to explain each arrow.

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

Appendix A

Programming Assignments

رومانی صبحی رزق جرجس مطر

This appendix collects the five programming assignments referenced throughout the book. Each assignment builds on the chapter(s) noted and should be completed individually unless your course states otherwise. General submission requirements: submit runnable source code, a short README describing how to run it, and a brief report (1–2 pages) presenting your results and discussing anything you found surprising or difficult. Appendix D provides a primer on the Python tooling (NumPy, scikit-learn, PyTorch, matplotlib) used across all five assignments.

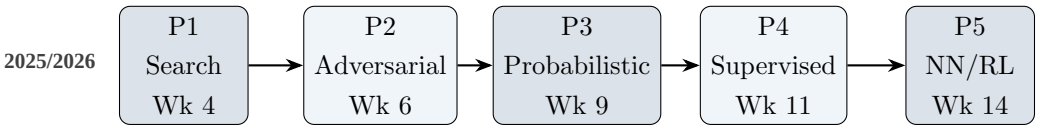


Figure A.1: Timeline of the five programming assignments across the 16-week course, aligned with the chapters they draw on.

A.1 P1: Search Agents (Chapters 2–3)

Objective. Implement and compare BFS, uniform-cost search, and A* search on a pathfinding or puzzle domain (e.g., the 8-puzzle or a grid pathfinding problem with obstacles).

Requirements.

- Implement a general search framework with a pluggable frontier data structure and priority function.
- Implement BFS, UCS, and A* using at least two distinct admissible heuristics.
- On at least five test instances of varying difficulty, report nodes expanded, solution depth, and wall-clock time for each algorithm.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- Discuss the trade-offs observed and relate them to the theoretical complexity results from Chapters 2–3.

Suggested extension. Implement IDA* and compare its memory usage against A* on the hardest instance you can solve with both.

A.2 P2: Adversarial Search (Chapter 5)

Objective. Implement minimax with alpha-beta pruning and a hand-crafted evaluation function for a two-player zero-sum game (e.g., Tic-Tac-Toe, Connect Four, or a simplified Chess variant).

Requirements.

- Implement minimax and alpha-beta pruning as separate, independently verifiable functions that produce identical move choices.
- Design and justify an evaluation function for non-terminal states at your chosen search depth.
- Measure and report nodes expanded with and without pruning, at two or more search depths.

2025/2026 2025/2026 2025/2026

- Play your agent against a random-move baseline and report win rate over at least 50 games.

Suggested extension. Add iterative deepening with a move-ordering heuristic derived from the previous iteration's best moves, and measure its effect on pruning efficiency.

A.3 P3: Probabilistic Inference (Chapters 8–9)

Objective. Implement exact inference in a Bayesian network and a particle filter for a simple temporal model.

Requirements.

- Construct a Bayesian network with at least five nodes for a domain of your choosing, with justified conditional probability tables.
- Implement exact inference by enumeration or variable elimination, and verify at least two query results by hand calculation.
- Implement a particle filter for a simple HMM (e.g., robot localization on a 1D or 2D grid).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- Report how filtering accuracy changes as the number of particles increases, including at least one plot.

Suggested extension. Compare your particle filter's estimates against the exact forward algorithm on a small enough state space to compute both, quantifying the approximation error.

A.4 P4: Supervised Learning (Chapter 11)

Objective. Implement logistic regression and a decision tree classifier from scratch and benchmark them against scikit-learn.

Requirements.

- Implement logistic regression trained by gradient descent (no automatic differentiation libraries).
- Implement a decision tree using information-gain splitting, with a configurable maximum depth or minimum leaf size.
- Evaluate both models on a provided dataset using k-fold cross-validation, reporting accuracy, precision, and recall.
- Compare your implementations against scikit-learn's `LogisticRegression` and `DecisionTreeCla` on the same folds, and explain any performance gap.

2025/2026 2025/2026 2025/2026

Suggested extension. Implement L_2 regularization for your logistic regression and show, empirically, its effect on the bias–variance trade-off as you vary the regularization strength.

A.5 P5: Neural Networks or Reinforcement Learning (Chapters 12–13)

Choose one of the following two options.

Option A: Neural Networks. Implement a feedforward neural network with backpropagation from scratch for a classification task, then reproduce your results using PyTorch. Report training curves, final test accuracy, and a discussion of any differences between your from-scratch implementation and the PyTorch version.

Option B: Reinforcement Learning. Implement tabular Q-learning and SARSA for a gridworld environment with at least one risky region (e.g., a cliff). Compare the learned policies and plot cumulative reward during training for both algorithms under matched hyperparameters.

Requirements (either option).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- A clear description of your chosen task, model or environment, and hyperparameters.
- At least one plot showing learning progress over training.
- A discussion connecting your empirical results to the relevant theory from the corresponding chapter.

A.6 Common Grading Rubric

Unless your instructor provides an assignment-specific rubric, the following general breakdown applies to each of P1–P5.

Table A.1: Default grading rubric applied to each programming assignment.

Component	Description	Weight
Correctness	Algorithm produces correct output on provided and hidden test cases	50%
Empirical analysis	Required plots/tables present, with results correctly interpreted	25%
Code quality	Readable, reasonably efficient, matches submitted report	15%
Report clarity	Discussion is clear and specifically addresses the required questions	10%

2025/3026 2025/2026 2025/2026

A.7 Common Pitfalls Across Assignments

- **Off-by-one errors in goal testing.** Testing for the goal when a node is generated rather than when it is popped for expansion (Chapter 2) can silently return a suboptimal solution in UCS or A*.
- **Forgetting to handle duplicate frontier entries.** When a cheaper path to an already-frontier node is found, the old, more expensive entry must be replaced, not merely supplemented (Chapter 2).
- **Reporting training accuracy instead of held-out accuracy.** This inflates apparent performance and will be penalized under the empirical-analysis rubric component (Chapter 11).
- **Mismatched random seeds between runs used for the report and the final submitted code**, which makes reported numbers irreproducible (see Appendix D's reproducibility checklist).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- **Comparing your implementation against a library baseline using different hyperparameters or preprocessing**, which makes the comparison uninformative rather than a genuine correctness check.



رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

رومانی صبحی رزق جرجس مطر

Appendix B

Term Project Guidelines

B.1 Overview

The term project asks teams of two to three students to select an application problem, apply one or more techniques from this book, and evaluate the result empirically. Projects are expected to go beyond directly re-running a tutorial: some element of the problem formulation, method combination, evaluation design, or domain application should be the team's own contribution.

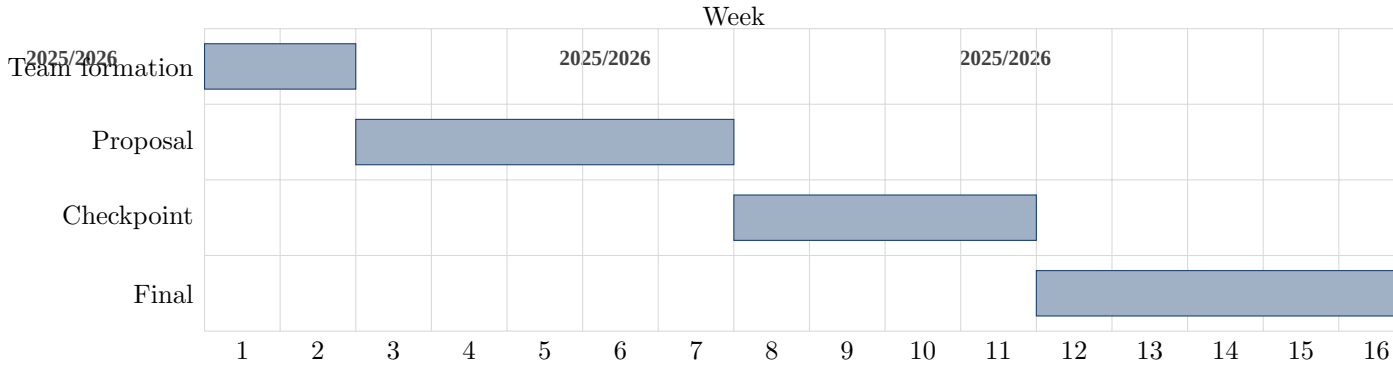


Figure B.1: Term project timeline over the 16-week semester. Team formation and topic selection happen in the first two weeks; the proposal is due at Week 7; the in-lab checkpoint at Week 11; and the final presentation and report at Week 15, matching the weekly schedule in the course syllabus.

B.2 Milestones

Milestone	Deliverable	Weight
Proposal	1–2 page written proposal	—
Checkpoint	In-lab progress demo	—
Final	Presentation (10 min) + report (6–8 pages)	100%

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

B.2.1 Proposal (1–2 pages)

State the problem, why it is interesting, the data or environment you will use, the AI technique(s) you plan to apply (referencing specific chapters of this book), and how you will evaluate success. Identify risks (e.g., data availability, computational cost) and a fallback plan.

B.2.2 Checkpoint

A short in-lab demonstration of progress: at minimum, a working data pipeline or environment setup and a first, even if weak, baseline result. The purpose of the checkpoint is to surface problems (data issues, infeasible scope) while there is still time to adjust.

B.2.3 Final Report and Presentation

The report should include: problem statement and motivation; related work or background (brief); method description with enough detail for a classmate to reproduce your approach; experimental setup; results, including at least one baseline comparison; a discussion of limitations; and an ethical analysis of your application (see Chapter 15) addressing at least bias/fairness and one other axis relevant to your domain. The presentation should cover the same material at a level appropriate for the rest of the class, who will not have read your report in advance.

2025/2026 2025/2026 2025/2026

B.3 Grading Rubric

Component	Weight
Technical execution (correctness, appropriateness of method)	40%
Evaluation rigor (baselines, held-out data, honest reporting of limitations)	25%
Report and presentation clarity	25%
Ethical analysis	10%

B.4 Suggested Project Areas

The list below is illustrative, not exhaustive; teams are encouraged to propose their own topic within the scope of the course.

- **Search/planning**: route planning or scheduling with real-world constraints, comparing informed search or CSP techniques.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- **Game playing**: an agent for a game not covered in class, comparing minimax/alpha-beta against Monte Carlo tree search.
- **Probabilistic reasoning**: a diagnosis or recommendation system built on a Bayesian network learned or elicited for a specific domain.
- **Reinforcement learning**: an agent for a custom or open-source environment, comparing at least two algorithms from Chapter 13.
- **Machine learning application**: a classification or regression task on a real dataset, with careful attention to evaluation methodology and a fairness analysis appropriate to the domain.
- **Language or vision**: a small-scale NLP or computer vision application built on pretrained models, with original evaluation or error analysis.

B.5 Common Pitfalls

- **Scope creep.** Projects that try to do too much often ship nothing that works well. A narrow, well-evaluated project outperforms a broad, poorly-evaluated one under this rubric.
- **No baseline.** A result without a baseline for comparison (even a simple one, such as predicting the majority class) is difficult to interpret and will be marked down under evaluation rigor.
- **Data leakage.** Ensure test data is never used, even indirectly, during model selection or hyperparameter tuning.
- **Skipping the ethical analysis.** This is a required component, not an optional add-on; see Chapter 15's final exercise for guidance on what it should contain.

2025/2026 2025/2026 2025/2026

B.6 Two Worked Project Sketches

To make the abstract guidance above concrete, this section sketches two example projects at the level of detail expected in a proposal.

B.6.1 Example: Route Recommendation with Learned Preferences

Problem. Recommend walking routes in a city that match a user's stated preference (scenic, fastest, or avoiding hills), combining Chapter 3's A* search with a learned cost function.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Approach. Represent the city as a graph (Chapter 2–3) with edge features (length, elevation change, a scenic-score from public data). Learn a weighting over these features from a small set of labeled example routes using linear regression (Chapter 11), then run A* with the learned combination as edge cost.

Evaluation. Compare routes produced by the learned cost function against routes produced by a single hand-picked weighting (baseline) and against user preference labels held out from training.

Ethical analysis. Check whether the scenic-score data source itself has coverage gaps (e.g., systematically less data in lower-income neighborhoods), which could bias the system toward routing pedestrians away from those areas regardless of actual safety or interest.

B.6.2 Example: Cliff-Walking Agent with Risk-Sensitive Policies

Problem. Compare how Q-learning and SARSA (Chapter 13) trade off risk and reward differently in an environment with a costly failure mode.

Approach. Implement the cliff-walking gridworld from Chapter 13, then extend it with a second, riskier shortcut region and a tunable exploration rate ϵ. Train both algorithms across a range of ϵ values.

Evaluation. Plot the learned policies' risk profile (frequency of near-cliff paths chosen) as a function of ϵ, and compare final cumulative reward and worst-case single-episode reward for both algorithms.

2025/2026 2025/2026 2025/2026

Ethical analysis. Discuss, in general terms, why an on-policy algorithm's more conservative learned behavior might be preferable in a safety-critical deployment even at a small cost to average-case performance, connecting to Chapter 15's discussion of robustness.

Key Idea

Notice that both sketches follow the same shape: a clearly stated problem, a method drawn explicitly from named chapters, a baseline for comparison, and an ethical analysis specific to the chosen domain – exactly the four components weighted in the grading rubric above.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

رومانی صبحی رزق جرجس مطر

Appendix C

Mathematical Background

This appendix collects the mathematical tools used throughout the book, for reference and review. It is not a substitute for a full course in linear algebra, calculus, or probability, but should refresh the specific results this book relies on.

C.1 Linear Algebra

C.1.1 Vectors and Dot Products

A vector $x \in \mathbb{R}^n$ is an ordered list of n real numbers. The *dot product* of two vectors x and w is $w^\top x = \sum_{i=1}^{n} w_i x_i$, which appears throughout Chapters 11–14 as the core operation inside linear models, logistic regression, and every layer of a neural network. Geometrically, $w^\top x = \|w\|\|x\| \cos\theta$, where θ is the angle between the vectors, so the dot product is large and positive when two vectors point in similar directions, near zero when they are perpendicular, and negative when they point in opposing directions.

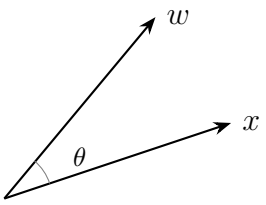


Figure C.1: Two vectors x and w with angle θ between them. The dot product $w^\top x$ measures how aligned the two vectors are, independent of any coordinate system.

C.1.2 Matrices and Matrix-Vector Products

A matrix $W \in \mathbb{R}^{m \times n}$ maps vectors in $\mathbb{R}^n$ to vectors in $\mathbb{R}^m$ via $y = Wx$, where each entry of y is the dot product of a row of W with x. This is exactly the operation computed at every layer of a feedforward network in Chapter 12 ($a^{(l)} = g(W^{(l)} a^{(l-1)} + b^{(l)})$), where $W^{(l)}$ maps the previous layer's activations into the current layer's pre-activation values.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Matrix multiplication is associative but not commutative: $(AB)C = A(BC)$, but in general $AB \neq BA$.

C.1.3 Norms

The L_2 (Euclidean) norm $\|w\|_2 = \sqrt{\sum_i w_i^2}$ measures a vector's length and appears in ridge regression's penalty term $\lambda\|w\|_2^2$ (Chapter 11). The L_1 norm $\|w\|_1 = \sum_i |w_i|$ appears in lasso regression's penalty and tends to produce sparse solutions (many weights exactly zero), unlike the L_2 penalty, which shrinks weights toward zero smoothly without typically forcing them to exactly zero.

C.2 Calculus and Optimization

C.2.1 Derivatives and Gradients

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the *gradient* $\nabla f(x)$ is the vector of partial derivatives $\left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right)$, and it points in the direction of steepest increase of f at x. This is the basis of *gradient descent* (Chapter 11): moving a small step in the direction $-\nabla f(x)$ decreases f locally, and repeating this process converges toward a local minimum.

2025/2026 2025/2026 2025/2026

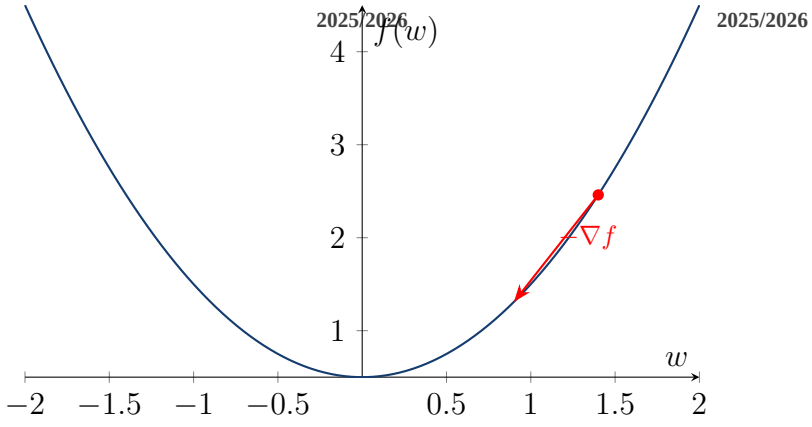


Figure C.2: Gradient descent on $f(w) = w^2 + 0.5$. At the point shown, the negative gradient points back toward the minimum; taking repeated small steps in this direction converges to $w = 0$, the function's minimizer.

C.2.2 The Chain Rule

For composed functions $f(g(x))$, the chain rule gives $\frac{d}{dx} f(g(x)) = f'(g(x))\, g'(x)$. This generalizes to vector and matrix-valued functions and is the single mathematical fact that makes backpropagation (Chapter 12) possible: a deep network is a long composition of layer functions, and the chain rule lets us compute the gradient with respect to any

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

weight by multiplying together the local derivatives along the path from that weight to the output, exactly the recursive $\delta^{(l)}$ computation of Section 12.3.

C.2.3 Convexity

A function is *convex* if the line segment between any two points on its graph lies on or above the graph – equivalently, it has a single global minimum with no other local minima to get stuck in. Linear regression's squared-error loss and logistic regression's cross-entropy loss are both convex in their parameters, which is why gradient descent is guaranteed to find the global optimum for those models. Neural network training losses (Chapter 12) are generally *non-convex*, meaning gradient descent can converge to different local minima depending on initialization – a key practical difference between the two model families.

C.3 Probability Review

2025/2026 2025/2026 2025/2026

C.3.1 Random Variables and Distributions

A random variable is a numerical outcome of a random process. A *discrete* random variable has a probability mass function $P(X = x)$ summing to 1 over its possible values; a *continuous* random variable has a probability density function $p(x)$ integrating to 1, with $P(a \le X \le b) = \int_a^b p(x)\,dx$.

C.3.2 Expectation and Variance

The expectation $\mathbb{E}[X] = \sum_x xP(X = x)$ (or the corresponding integral for continuous variables) is the probability-weighted average value of X, and appears throughout the book wherever we speak of "expected utility" (Chapter 10) or "expected return" (Chapter 13). Variance $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$ measures spread around the mean and underlies the bias–variance decomposition of Chapter 11.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

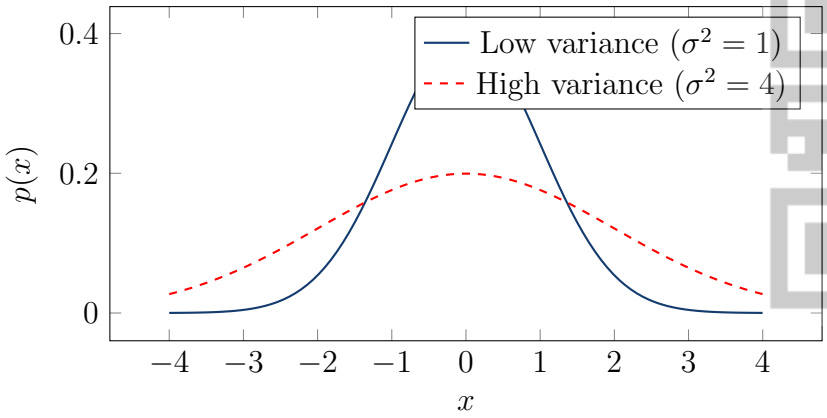


Figure C.3: Two Gaussian distributions with the same mean but different variances. Variance quantifies how spread out a distribution is around its expectation, independent of where that expectation is located.

C.3.3 The Chain Rule of Probability

For any random variables, $P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1})$. This identity underlies both the Bayesian network factorization of Chapter 8 (where conditional independence lets many terms in this product simplify) and the language modeling factorization of Chapter 14 (where x_i is the i-th word in a sequence).

C.4 Dimensionality Reduction: A Brief Look at PCA

Many datasets used in Chapters 11–12 have far more features than are actually needed to describe the underlying structure. *Principal component analysis* (PCA) finds a small number of new axes (linear combinations of the original features) that capture as much of the data's variance as possible, and is frequently used both to visualize high-dimensional data in two or three dimensions and to reduce the input dimensionality before training a simpler downstream model.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

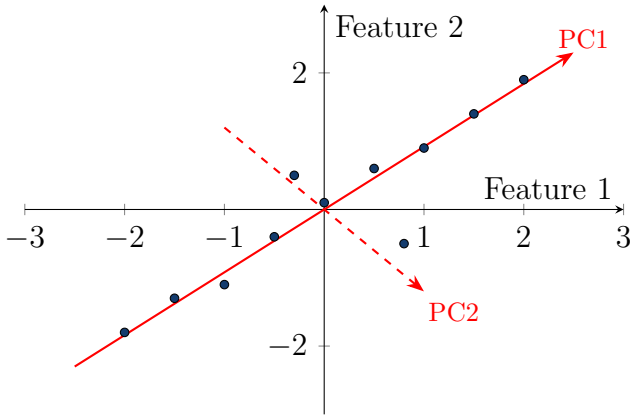


Figure C.4: PCA on a 2-D dataset with correlated features. The first principal component (PC1) is the direction of greatest variance; the second (PC2), orthogonal to the first, captures most of what remains. Projecting onto PC1 alone would preserve most of the structure visible in this data while reducing it to a single dimension.

رومانی صبحی رزق جرجس مطر

Mathematically, PCA finds the eigenvectors of the data's covariance matrix, ordered by their corresponding eigenvalues (the variance captured along each direction); this is the one place in the book where eigenvalues, rather than just dot products and gradients, play a central role, and a first linear algebra course's treatment of eigendecomposition is sufficient background.

2025/2026 2025/2026 2025/2026

C.5 Big-O Notation Review

Asymptotic notation describes how an algorithm's resource use grows with input size, ignoring constant factors and lower-order terms. We say $f(n) = O(g(n))$ if f grows no faster than a constant multiple of g for large n; this is the language used throughout the book's complexity claims (e.g., $O(b^{d+1})$ for breadth-first search in Chapter 2).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

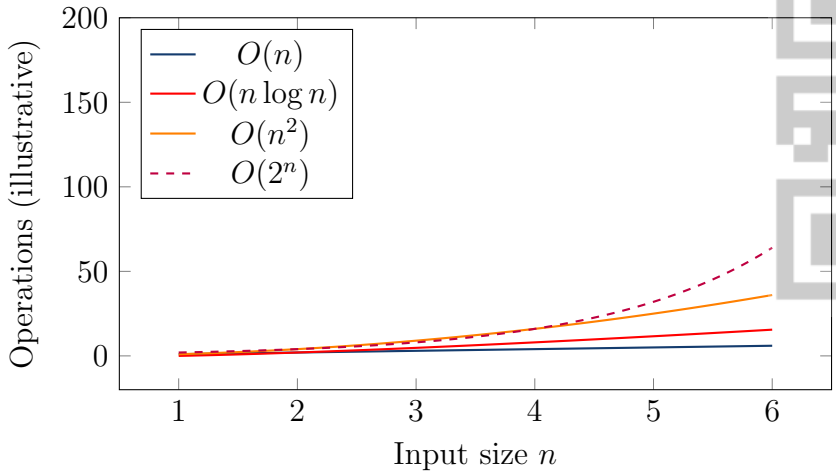


Figure C.5: Growth rates of common complexity classes. Exponential growth, $O(2^n)$ or more generally $O(b^n)$, is what makes exhaustive search over large state spaces (Chapters 2–5) infeasible without pruning or heuristics, since it quickly dwarfs every polynomial alternative regardless of the constant factors involved.

C.6 Notation Summary

Table C.1 collects notation used across multiple chapters, supplementing the algorithm- and formula-specific notation already collected in Quick Reference Sheet II at the back of the book.

2025/2026 2025/2026 2025/2026

Table C.1: Common mathematical notation used throughout this book.

Notation	Meaning
$\mathbb{R}^n$	the set of n-dimensional real vectors
$w^\top x$	dot product of vectors w and x
$\|w\|_2$, $\|w\|_1$	Euclidean and Manhattan norms of w
$\nabla f(x)$	gradient of f at x
$\mathbb{E}[X]$, $\mathrm{Var}(X)$	expectation and variance of X
$\arg\max_x f(x)$	the value of x that maximizes $f(x)$
$O(\cdot)$	asymptotic (big-O) upper bound on growth rate
$\odot$	elementwise (Hadamard) product

Exercises

C.1 Compute $w^\top x$ for $w = (1, 2, -1)$ and $x = (3, 0, 4)$, and state whether the angle between them is acute, obtuse, or right, without computing the angle directly.

C.2 Explain why ridge regression's L_2 penalty tends to shrink weights smoothly while

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

lasso's L_1 penalty tends to produce exactly zero weights. (A geometric argument based on the shape of the constraint region is sufficient.)

C.3 Using the chain rule, derive $\frac{d}{dx}\sigma(wx)$ where σ is the logistic sigmoid, and relate your answer to the gradient computations in Chapter 11.

C.4 Explain why convexity of the loss function matters for the reliability of gradient descent, using Figure C.2 as a reference, and explain what could go wrong on a non-convex loss.

C.5 Verify that the two Gaussian densities in Figure C.3 each integrate to 1 (a conceptual, not symbolic-integration, justification is sufficient), and explain what changing only the mean (not the variance) would do to each curve.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Appendix D

Python and Tooling Primer

رومانی صبحی رزق جرجس مطر

This appendix is a quick-start reference for the software tools used in the programming assignments (Appendix A). It assumes basic programming experience but not prior exposure to the specific scientific Python stack used in this course.

D.1 Environment Setup

All assignments use Python 3.11 or later. We recommend a virtual environment to keep course dependencies isolated from other projects:

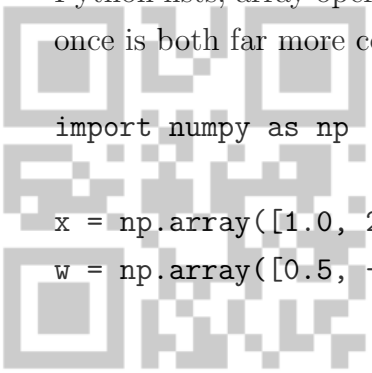
2025/2026 2025/2026 2025/2026

```
python3 -m venv ai-course-env
source ai-course-env/bin/activate     # on Windows: ai-course-env\Scripts\activate
pip install numpy scikit-learn torch matplotlib jupyter
```

If departmental GPU resources are unavailable, Google Colab's free tier provides a preconfigured environment with these packages already installed, and is sufficient for every assignment in this book except the largest optional extensions.

D.2 NumPy Essentials

NumPy arrays are the basic data structure for all numerical code in this course. Unlike Python lists, array operations are vectorized: applying an operation to an entire array at once is both far more concise and far faster than looping in pure Python.



```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
w = np.array([0.5, -1.0, 2.0])
```

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg


```
dot = w @ x                    # dot product, Appendix C.1
Wx  = W @ x                    # matrix-vector product, for a matrix W

grad_step = w - 0.01 * gradient   # one step of gradient descent
```

Key Idea

Whenever you find yourself writing a Python `for` loop over the elements of a NumPy array, pause and ask whether the same computation can be expressed as a vectorized array operation instead. Vectorized code is typically 10–100× faster and, once the idiom becomes familiar, easier to read.

D.3 Working with scikit-learn

Every scikit-learn model shares the same basic interface: construct the model, call `.fit(X, y)` on training data, then call `.predict(X_test)` or `.score(X_test, y_test)`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=5)   # 5-fold CV, Chapter 11
print(scores.mean(), scores.std())
```

This uniform interface is what makes it straightforward to compare your own from-scratch implementations (Programming Assignment P4) against a library baseline: as long as both expose `.fit` and `.predict`, the same evaluation code works for either.

D.4 A Minimal PyTorch Training Loop

Every neural network training loop in this course (Chapters 12–13) follows the same basic pattern: forward pass, compute loss, backward pass, update weights, repeat.

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128), nn.ReLU(),
    nn.Linear(128, 10)
)
```

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg


```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    for x_batch, y_batch in dataloader:
        optimizer.zero_grad()
        logits = model(x_batch)          # forward pass
        loss = loss_fn(logits, y_batch)
        loss.backward()                  # backprop, Chapter 12
        optimizer.step()                 # gradient descent update
```

The call to `loss.backward()` is where PyTorch performs automatic differentiation, computing exactly the backpropagation gradients derived by hand in Chapter 12, but for an arbitrarily constructed network, without the programmer ever writing out the chain rule explicitly.

D.5 Plotting with Matplotlib

Nearly every figure in this book showing a learning curve, comparison bar chart, or convergence plot can be reproduced with a few lines of matplotlib, and your assignment reports are expected to include similar plots.

2025/2026 2025/2026 2025/2026

```
import matplotlib.pyplot as plt

plt.plot(epochs, train_loss, label="train")
plt.plot(epochs, val_loss, label="validation")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.savefig("training_curve.png", dpi=150)
```

D.6 Reproducibility Checklist

Before submitting any assignment, verify the following, since reproducibility issues are among the most common (and most easily avoided) sources of lost points:

- Random seeds are set explicitly (`np.random.seed(...)`, `torch.manual_seed(...)`) so results can be reproduced.
- Your code runs from a clean environment using only the packages listed in your README.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

- Reported numbers in your written report match what your submitted code actually outputs when run.
- Any data files your code depends on are either included, publicly downloadable via a script, or clearly documented as requiring the course-provided dataset.

D.7 Debugging Numerical Code

Numerical bugs are often silent – the code runs without error but produces wrong answers – which makes them harder to catch than syntax errors. Useful habits: check array shapes at each step (`x.shape`) since shape mismatches are the single most common source of silently wrong results in vectorized code; verify gradients numerically for any hand-derived backpropagation code (Chapter 12) using finite differences on a small test case before trusting them at scale; and start with a tiny synthetic dataset where you know the correct answer in advance, rather than debugging directly on the full assignment dataset.

D.8 Version Control for Assignments

Every assignment in this course benefits from basic version control, even when working alone. A minimal workflow:

2025/2026 2025/2026 2025/2026

```
git init
git add .
git commit -m "Initial working BFS implementation"
# ... continue working ...
git add .
git commit -m "Add A* with Manhattan distance heuristic"
```

For team-based work (the term project, Appendix B), a shared remote repository (GitHub, GitLab, or your institution's equivalent) with frequent small commits makes it far easier to recover from a broken change, divide work by feature, and produce a clear record of who contributed what – often required documentation for team-based grading.

Key Idea

Commit early and often, with a message describing *what changed and why*, not just "fixed bug" or "updates." A commit history that reads like a changelog is far more useful when something breaks two weeks later than a single commit containing an entire assignment's worth of untracked changes.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

D.9 Common Error Messages and What They Usually Mean

Table D.1 lists errors that come up repeatedly across the programming assignments in Appendix A, to save debugging time on issues that are usually quick to diagnose once recognized.

Table D.1: Frequently encountered errors and their typical causes.

Error	Typical cause
`ValueError: shapes (n,) and (m,) not aligned`	A matrix or vector has the wrong dimensions for the operation attempted; print `.shape` on every operand.
Loss becomes `NaN` during training	Learning rate too high, or an unguarded log(0) in a cross-entropy loss; check for near-zero predictions before taking a log.
Model trains but validation accuracy never improves	Possible data leakage check reversed (training on validation data), or the loss and evaluation metric do not actually correspond to the same task.
Search algorithm runs forever	Missing or incorrect explored-set check (Chapter 2), causing infinite re-expansion of the same states.
A* returns a suboptimal path	Heuristic is not actually admissible for the specific action costs used, or the goal test fires on generation rather than expansion.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Exercises

D.1 Rewrite the following loop as a single vectorized NumPy expression: computing the squared Euclidean distance from a point p to every row of a matrix X.

D.2 Explain what `optimizer.zero_grad()` does in the PyTorch training loop above, and what would go wrong across multiple training steps if it were omitted.

D.3 Using the reproducibility checklist, identify two ways a submitted assignment could produce different results when re-run by a grader, even though the code itself was not changed.

D.4 Implement a numerical gradient check (using finite differences) for a simple function of your choosing, and use it to verify a hand-written analytical gradient.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Appendix E

Solutions and Hints to Selected Exercises

رومانی صبحی رزق جرجس مطر

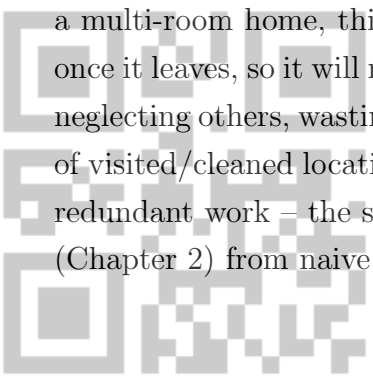
This appendix provides worked solutions or detailed hints for two or three representative exercises from each chapter, chosen to illustrate the kind of reasoning expected throughout the exercise sets. Remaining exercises are left for independent work, discussion sections, or graded homework.

Chapter 1: What Is AI?

2025/2026 2025/2026 2025/2026

Exercise 1.2 (rationality without omniscience). A rational agent crossing a street looks both ways, sees no oncoming traffic within stopping distance, and proceeds. If it is then struck by a vehicle running a red light at high speed from an obscured angle, the agent's decision to cross was still rational: rationality is judged against the information available *at decision time*, and the agent had no way to perceive the oncoming violator. An unlucky outcome does not imply an irrational decision, just as a lucky outcome from a reckless decision (crossing without looking, and happening not to be hit) does not imply the decision was rational.

Exercise 1.6 (model-based vs. simple reflex). A simple reflex vacuum agent using only "if dirt sensed, vacuum; if bumped, turn" performs adequately in a single small room because every location is within sensor range often enough that reactive rules suffice. In a multi-room home, this same agent has no way to know it has already cleaned a room once it leaves, so it will repeatedly wander back into already-clean rooms while potentially neglecting others, wasting battery. A model-based agent that tracks a simple internal map of visited/cleaned locations can instead prioritize unvisited rooms, substantially reducing redundant work – the same internal-state idea that distinguishes BFS-with-explored-set (Chapter 2) from naive tree search.



UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter 2: Uninformed Search

Exercise 2.2 (UCS optimality proof sketch). Suppose for contradiction that UCS returns a solution with cost $C > C^*$, the true optimal cost. Since UCS expands nodes in non-decreasing order of $g(n)$, and all step costs are strictly positive, every node on the optimal path has $g(n) \leq C^* < C$, so every such node would have been expanded (or at least placed on the frontier with cost $\leq C^*$) before the returned goal node. This contradicts UCS returning a higher-cost solution first. With negative step costs, this argument breaks: a node's $g(n)$ is no longer monotonically non-decreasing along a path, so a node reached via a long path with negative-cost edges could have arbitrarily low $g(n)$, and UCS's ordering assumption fails entirely (in fact, negative cycles make the problem ill-posed, since cost can be driven to $-\infty$).

Exercise 2.5 (tree vs. graph search). Consider a state space that is a single cycle of three states $A \to B \to C \to A$, with no goal state reachable. Tree search re-expands A, B, C, A, B, C, ... forever, since it never remembers that A has been visited before. Graph search, maintaining an explored set, expands A, B, C once each, finds no goal, exhausts the frontier, and correctly reports failure.

Exercise 2.6 (bidirectional search preconditions). Bidirectional search requires an explicit, known goal state (so a backward search frontier can be initialized), reversible actions (so the backward search can generate valid predecessor states), and an efficient test for whether the two frontiers intersect. A route-finding problem satisfies all three easily (roads are typically bidirectional, the destination is known, and city membership in each frontier can be checked with a hash set). An open-ended text-generation search problem generally fails the first condition, since there is no single well-defined goal state to search backward from.

Chapter 3: Informed Search

Exercise 3.1 (max of two admissible heuristics). If $h_1(n) \leq h^*(n)$ and $h_2(n) \leq h^*(n)$ for the true cost-to-goal $h^*(n)$, then $\max(h_1(n), h_2(n)) \leq h^*(n)$ as well, since the max of two quantities each individually at most $h^*(n)$ cannot exceed $h^*(n)$. So the max is admissible. The sum $h_1(n) + h_2(n)$ is *not* in general admissible: e.g., if $h^*(n) = 5$ and $h_1(n) = h_2(n) = 4$ (both admissible individually), their sum is 8, which overestimates.

Exercise 3.2 (Manhattan distance is consistent). For an 8-puzzle move sliding one tile into the blank, exactly one tile moves a Manhattan distance of exactly 1 closer to or farther from its goal position, and all other tiles' distances are unchanged. So $h(n) - h(n')$ is either $+1$ (move helped) or -1 (move hurt), and the step cost $c(n, a, n') = 1$ in all cases. The consistency condition $h(n) \leq c(n, a, n') + h(n')$ becomes $h(n) \leq 1 + h(n')$, i.e., $h(n) - h(n') \leq 1$, which holds since the actual change is at most $+1$ in magnitude.

Exercise 3.3 (consistency and re-expansion). Consistency guarantees $f(n) = g(n) + h(n)$ is non-decreasing along every path from the root. If A* graph search later finds a cheaper path to a node n already in the explored set, that new path would have to arrive with a lower $g(n)$, hence lower $f(n)$ than when n was first expanded – but since n was expanded only when its f-value was the minimum on the frontier, and f-values along any single path never decrease, no later-discovered path to n can have a smaller f-value than the one that caused n's original expansion. This is precisely why consistency (not just admissibility) is the condition required for A* graph search's optimality.

Chapter 4: Constraint Satisfaction

Exercise 4.1 (Australia map coloring). Variables: WA, NT, SA, Q, NSW, V, T (Australian states/territories). Domains: {Red, Green, Blue} each. Constraints: adjacent regions must differ (WA–NT, WA–SA, NT–SA, NT–Q, SA–Q, SA–NSW, SA–V, Q–NSW, NSW–V; Tasmania has no mainland neighbors). A 2-coloring is impossible because the mainland constraint graph contains a triangle (e.g., SA, NT, Q are mutually adjacent), and a triangle requires 3 distinct colors – no 2-coloring of a graph containing an odd cycle (including a triangle) exists.

Chapter 5: Adversarial Search

2025/2026 2025/2026 2025/2026

Exercise 5.3 (move ordering and pruning). Alpha-beta's amount of pruning depends on how quickly tight α/β bounds are established. If the best move at a node is examined first, subsequent siblings are compared against an already-strong bound and are pruned more often; if the best move is examined last, little or no pruning occurs before it is found. Because chess has a much higher branching factor than Tic-Tac-Toe, the difference between well-ordered and poorly-ordered search compounds multiplicatively with depth, making move ordering (e.g., trying captures and previously-good moves first) practically essential for chess engines but nearly irrelevant for exhaustively-searchable games like Tic-Tac-Toe.

Chapter 6: Propositional Logic

Exercise 6.1 (resolution derivation). $(P \Rightarrow Q) \wedge (Q \Rightarrow R)$ in CNF is $(\neg P \vee Q) \wedge (\neg Q \vee R)$. Resolving these two clauses on $Q/\neg Q$ gives $(\neg P \vee R)$, which is exactly the CNF form of $P \Rightarrow R$. This matches the general pattern in Figure 6's resolution refutation: chaining implications via resolution is precisely how hypothetical syllogism is derived as a special case of the single resolution rule.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter 7: First-Order Logic

Exercise 7.2 (unification). Unify Knows(John, x) with Knows(y, Mother(y)): matching the first arguments gives y = John; matching the second arguments then requires x = Mother(John). The most general unifier is $\{y/\text{John}, x/\text{Mother(John)}\}$, and applying it to both sentences yields the identical sentence Knows(John, Mother(John)).

Chapter 8: Bayesian Networks

Exercise 8.2 (rare disease, accurate test). Suppose a disease has prior $P(D) = 0.001$, and a test has true-positive rate $P(+ \mid D) = 0.99$ and false-positive rate $P(+ \mid \neg D) = 0.05$. By Bayes' rule,

$$P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+ \mid D)P(D) + P(+ \mid \neg D)P(\neg D)} = \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.05 \times 0.999} \approx 0.019$$

Despite the test's high accuracy, only about 2% of positive results are true positives, because the disease is so rare that the much larger population of healthy people, even at a low false-positive rate, produces more false positives in absolute terms than the small diseased population produces true positives. This is exactly the pattern shown in Figure 8's rare-disease chart in Chapter 8.

2025/2026 2025/2026 2025/2026

Chapter 9: Temporal Models

Exercise 9.1 (rejection sampling vs. likelihood weighting). Rejection sampling generates full samples and discards any inconsistent with the evidence; if the evidence event has probability, say, 0.001, then roughly 999 out of every 1000 samples are wasted. Likelihood weighting instead fixes the evidence variables directly during sampling and weights each (never-discarded) sample by how likely that evidence was given the sampled values of its parents, so no samples are wasted regardless of how rare the evidence is – at the cost of requiring more samples to reduce variance when evidence occurs deep in the sampling order, since weights can become very small.

Chapter 10: MDPs

Exercise 10.1 (utility axioms). The transitivity axiom states that if an agent prefers A to B and B to C, it must prefer A to C. An agent violating transitivity (preferring $A > B > C > A$ in a cycle) can be money-pumped: starting with C, it would pay to trade up to B, then pay to trade up to A, then pay to trade back down to C, ending

رومانی صبحی رزق برجس مطر

where it started but strictly poorer – a sequence of trades each individually rational under the agent's stated preferences but collectively guaranteed to lose money, showing why rational preference must be transitive.

Chapter 11: Machine Learning Foundations

Exercise 11.2 (gradient of squared error). For $\text{Loss}(w) = \sum_i (y_i - w^\top x_i)^2$,

$$\nabla_w \text{Loss}(w) = \sum_i 2(y_i - w^\top x_i)(-x_i) = -2\sum_i (y_i - w^\top x_i)\, x_i$$

One gradient descent step is $w \leftarrow w - \alpha \nabla_w \text{Loss}(w) = w + 2\alpha \sum_i (y_i - w^\top x_i)\, x_i$, moving w in the direction that reduces the residual $(y_i - w^\top x_i)$ for each training example, weighted by how large that residual currently is.

Chapter 12: Neural Networks

Exercise 12.1 (perceptron cannot represent XOR). XOR outputs 1 exactly when its two binary inputs differ. Plotting the four input points $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$ shows that no single straight line can separate the two positive points from the two negative points: the positive points are on one diagonal and the negative points on the other, so any line separating them on one side necessarily misclassifies at least one point on the other diagonal. A hidden layer resolves this by first mapping the inputs into a new feature space (via a nonlinear transformation) in which the four points *are* linearly separable – geometrically, "bending" the input space before applying a linear decision boundary.

Chapter 13: Reinforcement Learning

Exercise 13.2 (off-policy vs. on-policy). Q-learning's update target uses $\max_{a'} Q(s', a')$ – the value of the *best* available next action, regardless of what the exploration policy actually does next. SARSA's target uses $Q(s', a')$ for the action a' *actually selected* by the current (ϵ-greedy) policy. Because Q-learning's target ignores the possibility of an exploratory misstep, it learns the value of the optimal policy; because SARSA's target incorporates the actual (occasionally exploratory, occasionally risky) behavior, it learns the value of the policy actually being executed, including its own exploration risk.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Chapter 14: Language and Perception

Exercise 14.4 (parallelism in transformers). An RNN's hidden state at time t depends on its hidden state at time $t-1$, creating a strict sequential dependency that prevents computing all time steps simultaneously during training. A transformer's self-attention computes, for every position, a function of all positions at once (via matrix operations over the whole sequence), with no such step-by-step dependency, so all positions can be processed in parallel on suitable hardware. Because attention itself is permutation-invariant (shuffling the input positions would shuffle the output identically), positional encodings must be added explicitly so the model can distinguish "the cat sat" from "sat the cat."

Chapter 15: Ethics and Governance

Exercise 15.2 (fairness impossibility, concrete numbers). Using Table 15.1's two groups with different false-negative rates (8% vs. 22%) at equal overall accuracy (90% each), achieving *equalized odds* (equal false-negative and false-positive rates across groups) would require changing the decision threshold for at least one group, which then breaks *demographic parity* (equal positive-prediction rates) if the groups' underlying qualification rates differ – concretely illustrating why satisfying one fairness criterion can require violating another whenever base rates differ across groups.

2025/2026 2025/2026 2025/2026

Chapter 16: Synthesis

Exercise 16.3 (Bayes nets and HMMs). A hidden Markov model is a Bayesian network with a very specific, repeating structure: a chain of hidden state nodes $X_1 \to X_2 \to \cdots \to X_T$ (encoding the Markov property that each state depends only on the previous one), with each X_t additionally pointing to an observed evidence node E_t (encoding that evidence depends only on the current state). The general Bayesian network machinery of Chapter 8 (joint distribution as a product of conditionals) applies directly; the HMM's added structural assumption is simply that this particular chain-shaped graph, repeated identically at every time step, is what the network looks like.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Appendix F

End-to-End Case Study: Building a Maze-Solving Agent

رومانی صبحی رزق جرجس مطر

This appendix walks through a single running example – an agent that must navigate a maze to collect a reward – across the search, probabilistic, and learning techniques introduced throughout the book. The goal is to show, concretely, how the same problem can be approached with progressively more sophisticated tools as its assumptions are relaxed, and to serve as a template for the kind of integrated thinking expected in the term project (Appendix B).

2025/2026 2025/2026 2025/2026

F.1 Stage 1: Known Maze, Deterministic Movement

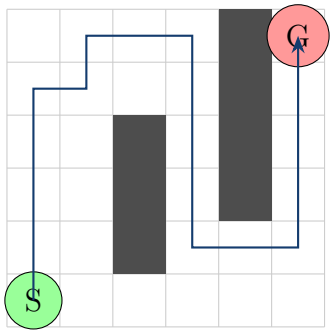


Figure F.1: Stage 1: a fully known, deterministic maze. A* with Manhattan distance (Chapter 3) finds the shortest path around both walls in milliseconds; there is no uncertainty to reason about yet.

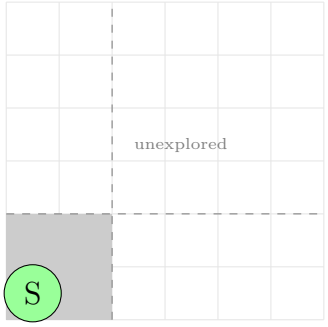
With the maze layout known in advance and movement deterministic (every action succeeds exactly as intended), this is a direct application of Chapter 3's A* search: states are grid cells, actions are the four compass directions, the transition model is deterministic, and the goal test checks whether the current cell is the goal cell. The heuristic

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

(Manhattan distance to the goal) is admissible because it never overestimates the true number of moves required, even accounting for walls.

F.2 Stage 2: Unknown Maze, Must Be Explored

Now suppose the agent does not know the maze layout in advance and can only observe walls in cells adjacent to its current position. This is no longer a pure search problem in the Chapter 2–3 sense, since the transition model (which cells are passable) is not fully known up front.



2025/2026 2025/2026 2025/2026

Figure F.2: Stage 2: the agent has only explored the shaded region near its start. A model-based reflex agent (Chapter 1) that maintains a partial map and re-plans with A* every time new wall information is observed – an approach sometimes called "repeated A*" – handles this setting well as long as the maze does not change while being explored.

Handling this properly requires exactly the model-based reflex agent architecture of Chapter 1: the agent maintains an internal map (its belief about which cells are open or walled), updates that map as it observes new cells, and re-invokes A* on its current best map estimate whenever a planned path turns out to be blocked.

F.3 Stage 3: Noisy Sensors, Uncertain Position

Now suppose the agent's own position sensor is noisy – it receives a reading that is usually, but not always, correct. The agent must now maintain a *belief state* over its own position, exactly as in Chapter 9's temporal models and Chapter 10's POMDP discussion, rather than assuming it knows its location exactly.

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

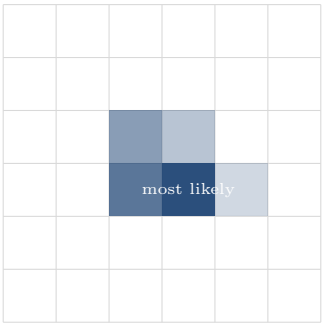


Figure F.3: Stage 3: a particle-filter-style belief over the agent's true position (Chapter 9), with opacity representing probability mass. The agent should choose actions that make sense across this whole distribution – for instance, favoring an action safe under all likely positions – rather than acting as if the single most likely cell were certainly correct.

Concretely, the agent would run a particle filter (Chapter 9), propagating position hypotheses forward under its motion model and reweighting them against noisy sensor readings, then choose actions using an MDP or POMDP formulation (Chapter 10) rather than the pure deterministic search of Stage 1.

F.4 Stage 4: Unknown Reward Structure, Learned by Trial and Error

2025/2026 2025/2026 2025/2026

Finally, suppose the agent does not even know where the reward is or what the maze looks like in advance, and must learn purely from experience – exactly the reinforcement learning setting of Chapter 13.

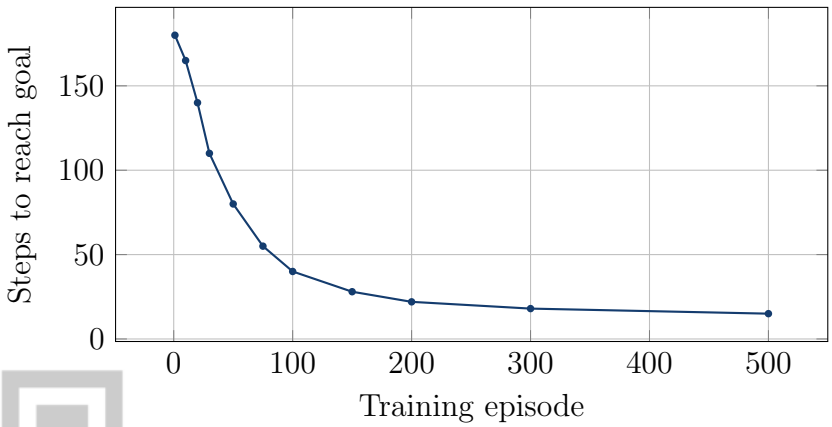


Figure F.4: Stage 4: learning curve for a Q-learning agent (Chapter 13) discovering the maze and the reward location purely through trial-and-error exploration. Early episodes involve extensive random wandering; as the action-value estimates improve, the agent converges toward a near-optimal number of steps, approaching (but not necessarily matching exactly) the Stage-1 A* solution length, since some residual exploration continues under an ϵ-greedy policy.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

F.5 What This Progression Illustrates

Key Idea

The same physical problem (navigate a maze to a reward) is solved by four different techniques in this appendix, chosen entirely by which assumptions about the environment can and cannot be made: full knowledge and determinism favor plain search (Stage 1); partial knowledge favors a model-based agent that re-plans (Stage 2); sensor uncertainty favors probabilistic state estimation (Stage 3); and unknown dynamics/rewards favor learning from experience (Stage 4). Choosing the right technique starts with correctly diagnosing which of these assumptions actually holds in your problem.

This progression mirrors the book's overall structure (Chapters 2–5, then 8–10, then 11–14) and is a useful diagnostic to apply to your own term project: identify which stage your problem most resembles before committing to a specific algorithm, since applying Stage-1 tools (plain search) to a Stage-3 or Stage-4 problem (with real uncertainty or unknown dynamics) is a common and avoidable source of a project's mid-semester technical difficulties.

Exercises

2025/2026 2025/2026 2025/2026

F.1 For a maze-solving agent with a*known* but stochastic transition model (e.g., 10% chance any move fails and the agent stays in place), explain which stage's technique from this appendix best applies, and why.

F.2 Sketch, at a high level, how you would combine Stage 2's map-building with Stage 4's reinforcement learning if both the maze layout and the reward location are initially unknown.

F.3 Using Figure F.4, explain why the learning curve flattens but does not reach exactly the Stage-1 optimal step count, and connect your answer to the exploration–exploitation trade-off from Chapter 13.

F.4 Choose a term-project idea of your own and write one paragraph identifying which of the four stages in this appendix it most closely resembles, and why.

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Glossary

رومانی صبحی رزق جرجس مطر

Admissible heuristic
A heuristic that never overestimates the true cost to the goal (Ch. 3).

Agent Anything that perceives its environment through sensors and acts upon it through actuators (Ch. 1).

Arc consistency
A local consistency condition on a CSP: every value in a variable's domain has a compatible value in each neighbor's domain (Ch. 4).

Attention A mechanism computing a weighted combination of information across sequence positions, weighted by learned relevance (Ch. 14).

2025/2026 2025/2026 2025/2026

Backpropagation
An efficient algorithm for computing gradients in a neural network via the chain rule, working backward from the output layer (Ch. 12).

Bayes' rule The identity $P(h \mid e) = P(e \mid h)P(h)/P(e)$, used to update beliefs given evidence (Ch. 8).

Bayesian network
A directed acyclic graph encoding conditional independence assumptions among random variables, together with conditional probability tables (Ch. 8).

Bias–variance trade-off
The decomposition of prediction error into systematic error from an overly simple model (bias) and sensitivity to the training sample (variance) (Ch. 11).



Bidirectional search
Simultaneous forward and backward search that meets in the middle, reducing time complexity roughly to $O(b^{d/2})$ (Ch. 2).

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Conditional independence

A and B are conditionally independent given C if $P(A, B \mid C) = P(A \mid C)P(B \mid C)$ (Ch. 8).

Consistent heuristic

A heuristic satisfying $h(n) \leq c(n, a, n') + h(n')$ for every edge; implies admissibility (Ch. 3).

Constraint satisfaction problem

A problem defined by variables, domains, and constraints on allowable value combinations (Ch. 4).

Convolutional neural network

A neural network using shared, spatially local filters, well suited to grid-structured data such as images (Ch. 12).

Cross-validation

An evaluation technique that partitions data into folds, training on some and testing on the rest, to obtain a more reliable performance estimate (Ch. 11).

Discount factor (γ)

2025/2026 2025/2026 2025/2026

A factor in $[0, 1)$ that down-weights future rewards in a Markov decision process (Ch. 10).

Entailment $KB \models \alpha$ holds if α is true in every model in which KB is true (Ch. 6).

Expectimax A generalization of minimax that averages over chance-node outcomes weighted by probability, rather than taking a strict min or max (Ch. 5).

Expected utility

The probability-weighted average utility of an action's possible outcomes (Ch. 10).

Filtering Computing the current belief about a hidden state given all evidence observed so far, in a temporal probabilistic model (Ch. 9).

First-order logic

A logic extending propositional logic with objects, relations, functions, and quantifiers (Ch. 7).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Forward chaining
An inference procedure that repeatedly applies rules whose premises are already known, until no new facts follow (Ch. 6).

Frontier The set of nodes generated but not yet expanded during search (Ch. 2).

Generalization A model's ability to perform well on data it was not trained on, the central goal of machine learning (Ch. 11).

Gradient descent
An iterative optimization method that moves parameters in the direction of steepest decrease of a loss function (Ch. 11).

Heuristic function
An estimate of the cost from a given node to the nearest goal, used to guide informed search (Ch. 3).

Hidden Markov model
A temporal probabilistic model with a single discrete hidden state variable evolving under the Markov property (Ch. 9).

Horn clause A clause with at most one positive literal, a restriction that makes propositional and first-order inference tractable via forward or backward chaining (Ch. 6–7).

2025/2026 2025/2026 2025/2026

Markov decision process
A model of sequential decision-making under uncertainty, defined by states, actions, transition probabilities, and rewards (Ch. 10).

Markov property
The assumption that the current state depends only on a bounded (typically immediately preceding) history (Ch. 9).

Minimax A decision rule for adversarial games that assumes the opposing player will always choose the move worst for the maximizing player (Ch. 5).

Monte Carlo tree search
A search algorithm that grows a game tree using repeated random simulations rather than exhaustive expansion, well suited to very high branching factors (Ch. 5).

Overfitting Fitting a model too closely to training data, capturing noise rather than signal, at the expense of generalization (Ch. 11).

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Particle filter An approximate inference technique representing a belief state with a weighted population of samples, updated by propagation, reweighting, and resampling (Ch. 9).

PEAS A framework for specifying a task environment: Performance measure, Environment, Actuators, Sensors (Ch. 1).

Policy A mapping from states to actions, either in an MDP (Ch. 10) or learned via reinforcement learning (Ch. 13).

Q-learning An off-policy temporal-difference algorithm that learns the optimal action-value function directly from experience (Ch. 13).

Rational agent An agent that selects, for every percept sequence, the action expected to maximize its performance measure given available information (Ch. 1).

Regularization A technique that discourages overly complex models (e.g., large weights) to reduce overfitting (Ch. 11–12).

Reinforcement learning
Learning a policy for sequential decision-making from experience, without a given transition or reward model (Ch. 13).

Resolution An inference rule that derives new clauses from clauses containing complementary literals; refutation-complete for propositional and first-order logic (Ch. 6–7).

SARSA An on-policy temporal-difference algorithm that learns the value of the policy actually being followed, including its exploration (Ch. 13).

Self-attention Attention applied within a single sequence, where each position attends to every other position in that same sequence (Ch. 14).

Temporal-difference learning
A learning method that updates value estimates using the difference between successive predictions, without waiting for a final outcome (Ch. 13).

Transformer A neural architecture built entirely on attention layers, dispensing with recurrence, enabling parallel training and direct long-range dependency modeling (Ch. 14).

Unification Finding a substitution that makes two first-order logic expressions syntactically identical (Ch. 7).

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Utility function

A function mapping outcomes to real-valued preferences, generalizing binary goal satisfaction (Ch. 1, 10).

Value iteration An algorithm that computes the optimal value function for an MDP by repeatedly applying the Bellman update (Ch. 10).

Variable elimination

An exact inference algorithm for Bayesian networks that sums out hidden variables one at a time, reusing intermediate factors (Ch. 8).

رومانی صبحی رزق جرجس مطر

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Quick Reference Sheet I: Algorithms

Algorithm	Idea	Complete?	Optimal?
BFS	Expand shallowest node (FIFO)	Yes	Yes*
DFS	Expand deepest node (LIFO)	No	No
Uniform-cost	Expand lowest $g(n)$	Yes	Yes
Iterative deepening	DFS with growing depth limit	Yes	Yes*
Greedy best-first	Expand lowest $h(n)$	No	No
A*	Expand lowest $f(n) = g(n) + h(n)$	Yes	Yes†
Minimax	Max/min alternation over game tree	—	Yes (vs. optimal opp.)
Alpha-beta	Minimax + pruning ($\alpha \geq \beta$)	—	Yes (same result as minimax)
Backtracking (CSP)	Assign + check constraints, undo on failure	Yes	—
AC-3	Enforce arc consistency, propagate domain cuts	—	—
Min-conflicts	Local search reassigning conflicted variables	No	—

* under uniform step costs. † with an admissible (tree) or consistent (graph) heuristic.

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Algorithm	**Core update / rule**
Resolution	From $(\ell \vee \alpha)$ and $(\neg\ell \vee \beta)$, derive $(\alpha \vee \beta)$
Forward chaining	Apply Horn-clause rules whose premises are known, repeat to fixpoint
Value iteration	$U_{i+1}(s) \leftarrow \max_a \sum_{s'} P(s' \mid s,a)[R(s,a,s') + \gamma U_i(s')]$
Policy iteration	Alternate: evaluate current policy exactly, then act greedily w.r.t. its value
Q-learning (off-policy)	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
SARSA (on-policy)	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$
Gradient descent	$w \leftarrow w - \alpha \nabla_w \text{Loss}(w)$
Backpropagation	$\delta^{(l)} = (W^{(l+1)\top}\delta^{(l+1)}) \odot g'(z^{(l)})$, propagated backward
Particle filter	Propagate particles $\rightarrow$ reweight by evidence likelihood $\rightarrow$ resample

Search complexity (branching factor b, solution depth d, max depth m): BFS/UCS: $O(b^{d+1})$ time and space. DFS: $O(b^m)$ time, $O(bm)$ space. IDS: $O(b^d)$ time, $O(bd)$ space. Bidirectional: $O(b^{d/2})$.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg



Quick Reference Sheet II: Formulas and Notation

رومانی صبحی رزق جرجس مطر

Probability

$$P(a \mid b) = \frac{P(a \wedge b)}{P(b)} \qquad \text{Bayes' rule: } P(h \mid e) = \frac{P(e \mid h)P(h)}{P(e)}$$

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b) \quad \text{Cond. independence: } P(a, b \mid c) = P(a \mid c)P(b \mid c)$$

Bayesian network joint distribution: $P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid \text{Parents}(X_i))$
HMM filtering update: $P(X_{t+1} \mid e_{1:t+1}) \propto P(e_{t+1} \mid X_{t+1}) \sum_{x_t} P(X_{t+1} \mid x_t) P(x_t \mid e_{1:t})$

2025/2026 2025/2026 2025/2026

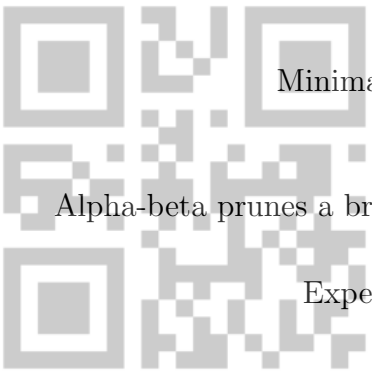
Decision Theory and MDPs

$$\text{Expected utility: } EU(a) = \sum_{s} P(s \mid a)\, U(s)$$

$$\text{Bellman equation: } U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, U^{\pi}(s')$$

$$\text{Bellman optimality: } U^{*}(s) = R(s) + \gamma \max_{a} \sum_{s'} P(s' \mid s, a)\, U^{*}(s')$$

Games



$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{terminal} \\ \max_a \text{Minimax}(\text{Result}(s,a)) & \text{MAX} \\ \min_a \text{Minimax}(\text{Result}(s,a)) & \text{MIN} \end{cases}$$

Alpha-beta prunes a branch when $\alpha \geq \beta$.

$$\text{Expectimax}(s) = \sum_{r} P(r)\, \text{Expectimax}(\text{Result}(s, r))$$

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Machine Learning

Squared error: $\text{Loss}(w) = \sum_i (y_i - w^\top x_i)^2$ Logistic: $P(y{=}1 \mid x) = \sigma(w^\top x) = \frac{1}{}$

Information gain: $\text{Gain}(A) = \text{Entropy}(S) - \sum_v \frac{|S_v|}{|S|}\text{Entropy}(S_v)$ Entropy: $H(S) = -\sum_i p_i \log_2$

Neural Networks

Forward: $a^{(l)} = g(W^{(l)}a^{(l-1)} + b^{(l)})$ Output error: $\delta^{(L)} = \nabla_a \text{Loss} \odot g'(z^{(L)})$

Backprop error: $\delta^{(l)} = (W^{(l+1)\top}\delta^{(l+1)}) \odot g'(z^{(l)})$ Weight grad: $\nabla_{W^{(l)}}\text{Loss} = \delta^{(l)}a^{(l-1)\top}$

Attention: $\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$

2025/2026 2025/2026 2025/2026

Reinforcement Learning

TD update: $U(s) \leftarrow U(s) + \alpha[R(s) + \gamma U(s') - U(s)]$

Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$

SARSA: $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big]$

Policy gradient: $\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t\big]$

رومانی صبحی رزق جرجس مطر

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Common Notation

Symbol	Meaning
s, s'	current state, next state
a	action
$P(x \mid y)$	probability of x given y
γ	discount factor, $0 \leq \gamma < 1$
α	learning rate (or, in game trees, the MAX bound)
b, d, m	branching factor, solution depth, maximum depth
$h(n), g(n), f(n)$	heuristic estimate, cost so far, $f = g + h$
π	policy
$U(s), Q(s, a)$	state value, action value
w, θ	model parameters / weights

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Bibliography

[1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson, 2021.

[2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[4] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] C. M. Bishop and H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.

[6] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[7] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[8] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd edition. Springer, 2009.

[9] D. Jurafsky and J. H. Martin. *Speech and Language Processing*, 3rd edition (draft). 2023.

[10] D. L. Poole and A. K. Mackworth. *Artificial Intelligence: Foundations of Computational Agents*, 3rd edition. Cambridge University Press, 2023.

[11] R. Szeliski. *Computer Vision: Algorithms and Applications*, 2nd edition. Springer, 2022.

[12] A. Vaswani et al. Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

2025/2026 2025/2026 2025/2026

UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg UG_31240275@ics.tanta.edu.eg

Index

رومانی صبحی رزق جرجس مطر

Admissible heuristic, 127
A* search, 17
Agent, 127
Arc consistency, 127
Attention, 76, 127

Backpropagation, 68, 127
Bayes' rule, 44, 127
Bayesian network, 127
Bellman equation, 57
Bias–variance trade-off, 127
Bidirectional search, 127

Conditional independence, 128
Consistent heuristic, 128
Constraint satisfaction problem, 128
Convolutional neural network, 128
Cross-validation, 128

Discount factor, 128

Entailment, 128
Expected utility, 128
Expectimax, 128

Fairness, 84
Filtering, 128
First-order logic, 128
Forward chaining, 129
Frontier, 129

Generalization, 129
Gradient descent, 129

Heuristic function, 129
Hidden Markov model, 129
Horn clause, 129

Markov decision process, 129
Markov property, 129
Minimax, 129
Monte Carlo tree search, 129

Overfitting, 129

Particle filter, 130
PEAS, 130
Policy, 130

Q-learning, 130

Rational agent, 1, 130
Regularization, 130
Reinforcement learning, 130
Resolution, 130

SARSA, 130
Search: frontier and explored set, 10
Self-attention, 130

Temporal model, 51
Temporal-difference learning, 73, 130
Transformer, 130

Unification, 130
Utility function, 131

Value iteration, 131
Variable elimination, 131

2025/2026 2025/2026 2025/2026